From SVMs to Uncertainty-Aware Classification: A Detailed Treatment of Gaussian and Gaussian Mixture Sample Uncertainty

Sushrut Gaikwad

April 20, 2026

Abstract

These notes provide a self-contained, detailed exposition of the Support Vector Machine with Gaussian Sample Uncertainty (SVM-GSU), as proposed by Tzelepis et al. (2017). We begin with the basics of the standard linear SVM and carefully build up to the SVM-GSU formulation, explaining every mathematical step in a way that is accessible to readers with an undergraduate-level background in linear algebra, calculus, and probability. We then develop a novel extension, SVM with Gaussian Mixture Uncertainty (SVM-GMU), which handles non-Gaussian uncertainties by modeling each training example's uncertainty as a mixture of Gaussians. This allows the classifier to handle complex, multimodal, and non-elliptical uncertainty shapes (e.g., banana-shaped contours) while preserving all the desirable properties of the original framework: closed-form loss, closed-form gradients, convexity, and efficient stochastic gradient descent optimization.

Contents

Notation

Throughout this document, we use the following conventions:

- n denotes the number of training examples.
- d denotes the dimensionality of the feature space (i.e., the number of features per example).
- Vectors are **lowercase, bold, non-italic**: $\mathbf{w}$, $\mathbf{x}$, $\mathbf{z}$.
- Greek vectors are **bold upright**: $\boldsymbol{\mu}$, $\boldsymbol{\sigma}$, $\boldsymbol{\theta}$.
- Matrices are **bold upright uppercase**: $\boldsymbol{\Sigma}$, $\boldsymbol{\Lambda}$.
- Scalars are italic: λ, b, d, n.
- In the standard SVM (Part I), $\mathbf{x}_i$ denotes the fixed feature vector of the i-th example.
- In SVM-GSU and SVM-GMU (Parts III and IV), $\boldsymbol{\mu}_i$ denotes the mean of the i-th example's uncertainty distribution, and $\mathbf{x}_i$ denotes a random vector drawn from that distribution: $\mathbf{x}_i \sim \mathcal{N}(\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$.

Part I
Background and the Standard Linear SVM

1 What Is Classification?

Suppose you are a doctor trying to decide whether a tumor is malignant or benign based on some measurements (e.g., size, shape, texture). You have access to historical data, that is, past patients for whom the diagnosis is known. Each patient's measurements form a *data point*, and the diagnosis (malignant or benign) is the *label*. The goal of *classification* is to learn a rule from this historical data that can predict the label of a new, unseen patient.

More formally, we have a *training set* of n examples:

$$\mathcal{X} = \Big\{(\mathbf{x}_i,\, y_i) :\ \mathbf{x}_i \in \mathbb{R}^d,\ y_i \in \{+1, -1\},\ i = 1, \dots, n\Big\}, \tag{1}$$

where:

- $\mathbf{x}_i \in \mathbb{R}^d$ is the *feature vector* of the i-th example. It is a d-dimensional column vector. Each entry represents some measured quantity (a "feature"). For example, if $d = 3$, then $\mathbf{x}_i = \begin{bmatrix} x_{i,1} & x_{i,2} & x_{i,3} \end{bmatrix}^\intercal$ might contain the tumor's diameter, perimeter, and average intensity.
- $y_i \in \{+1, -1\}$ is the *class label*. We use $+1$ for one class (say, malignant) and -1 for the other (benign).

The classifier's job is to find a function that, given a new $\mathbf{x}$, predicts whether $y = +1$ or $y = -1$.

2 The Idea of a Linear Classifier

The simplest classifier is a *linear classifier*. In two dimensions, this is just a straight line that separates the two classes. In three dimensions, it is a plane. In d dimensions, it is a *hyperplane*. A hyperplane in $\mathbb{R}^d$ is defined by a weight vector $\mathbf{w} \in \mathbb{R}^d$ and a bias (or intercept) $b \in \mathbb{R}$:

$$H : \quad \mathbf{w}^\intercal \mathbf{x} + b = 0. \tag{2}$$

Let us unpack this notation:

- $\mathbf{w}^\intercal \mathbf{x}$ is the *dot product* (or inner product) of $\mathbf{w}$ and $\mathbf{x}$:

  $$\mathbf{w}^\intercal \mathbf{x} = \sum_{j=1}^{d} w_j\, x_j = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d. \tag{3}$$

  This gives a single real number that measures how much $\mathbf{x}$ "aligns" with $\mathbf{w}$.
- b shifts the hyperplane. Without b, the hyperplane would always pass through the origin.

The hyperplane H divides $\mathbb{R}^d$ into two *halfspaces*:

$$\text{Positive side:} \quad \mathbf{w}^\intercal \mathbf{x} + b > 0, \tag{4}$$

$$\text{Negative side:} \quad \mathbf{w}^\intercal \mathbf{x} + b < 0. \tag{5}$$

A linear classifier predicts the label of a new point $\mathbf{x}_t$ as:

$$\hat{y}_t = \operatorname{sign}(\mathbf{w}^\intercal \mathbf{x}_t + b)\,, \tag{6}$$

where "sign$(\cdot)$" is the sign function: it returns $+1$ if the argument is positive and -1 if negative.

3 The Margin: Why Not Just Any Separating Line?

If the data is *linearly separable* (i.e., a hyperplane exists that perfectly separates the two classes), there are infinitely many such hyperplanes. Which one should we choose? Intuitively, we want the hyperplane that is as far as possible from the nearest data points of both classes. This "buffer zone" is called the *margin*. A larger margin means the classifier is more robust: small perturbations in the data are less likely to cause misclassification.

3.1 Deriving the Distance from a Point to a Hyperplane

Let us derive the distance formula from scratch. Consider a hyperplane $H : \mathbf{w}^\intercal \mathbf{x} + b = 0$ and a point $\mathbf{x}_0 \in \mathbb{R}^d$. The signed distance from $\mathbf{x}_0$ to H is the length of the shortest vector from $\mathbf{x}_0$ to any point on H.

Let $\mathbf{x}_H$ be the point on H closest to $\mathbf{x}_0$. The vector $\mathbf{x}_0 - \mathbf{x}_H$ must be perpendicular to H. Since $\mathbf{w}$ is the normal vector to H, we have $\mathbf{x}_0 - \mathbf{x}_H = t\mathbf{w}$ for some scalar $t \in \mathbb{R}$. Thus $\mathbf{x}_H = \mathbf{x}_0 - t\mathbf{w}$. Since $\mathbf{x}_H$ lies on H, we have $\mathbf{w}^\intercal \mathbf{x}_H + b = 0$. Substituting:

$$\mathbf{w}^\intercal(\mathbf{x}_0 - t\mathbf{w}) + b = 0 \implies \mathbf{w}^\intercal \mathbf{x}_0 + b - t\|\mathbf{w}\|^2 = 0 \implies t = \frac{\mathbf{w}^\intercal \mathbf{x}_0 + b}{\|\mathbf{w}\|^2}. \tag{7}$$

The distance is $\|\mathbf{x}_0 - \mathbf{x}_H\| = |t| \cdot \|\mathbf{w}\|$:

$$\text{distance}(\mathbf{x}_0, H) = \frac{|\mathbf{w}^\intercal \mathbf{x}_0 + b|}{\|\mathbf{w}\|}, \tag{8}$$

where $\|\mathbf{w}\| = \sqrt{\mathbf{w}^\intercal \mathbf{w}} = \sqrt{\sum_j w_j^2}$ is the Euclidean norm (length) of $\mathbf{w}$. The Support Vector Machine (SVM) is the classifier that *maximizes* this margin.

4 The SVM Optimization Problem

In practice, data may not be perfectly separable. The standard *soft-margin SVM* allows some misclassifications but penalizes them. The optimization problem is:

$$\min_{\mathbf{w}, b} \quad \mathcal{J}_{\text{SVM}}(\mathbf{w}, b) = \underbrace{\frac{\lambda}{2}\|\mathbf{w}\|^2}_{\text{regularization}} + \underbrace{\frac{1}{n}\sum_{i=1}^{n} \max(0,\ 1 - y_i(\mathbf{w}^\intercal \mathbf{x}_i + b))}_{\text{average hinge loss over training set}}. \tag{9}$$

Let us understand each term.

4.1 The Regularization Term: $\frac{\lambda}{2}\|\mathbf{w}\|^2$

This is the *regularization* term. Here $\lambda > 0$ is a user-chosen parameter that controls how much we care about keeping $\|\mathbf{w}\|$ small (equivalently, keeping the margin large, since the geometric margin equals $1/\|\mathbf{w}\|$). A larger λ means we prioritize a wider margin even if it means more misclassifications.

4.2 The Hinge Loss: $\max(0,\ 1 - y_i(\mathbf{w}^\intercal \mathbf{x}_i + b))$

This is the *hinge loss*, often written as $h(t) = \max(0, 1 - t)$ where $t = y_i(\mathbf{w}^\intercal \mathbf{x}_i + b)$. The quantity $t_i = y_i(\mathbf{w}^\intercal \mathbf{x}_i + b)$ is called the *functional margin* of point i. Let us see what happens:

- **Correctly classified and far from boundary** ($t_i \geq 1$): The hinge loss is zero. The point contributes no penalty. These are the "easy" points.
- **Correctly classified but close to boundary** ($0 < t_i < 1$): The hinge loss is $1 - t_i > 0$. The point is correctly classified but falls within the margin. It is penalized mildly.
- **Misclassified** ($t_i < 0$): The hinge loss is $1 - t_i > 1$. The point is on the wrong side of the boundary and is penalized heavily. The further away it is on the wrong side, the larger the penalty.

The SVM objective eq. (9) balances two competing goals:

1. **Wide margin** (small $\|\mathbf{w}\|$): Controlled by the regularization term.
2. **Few misclassifications** (small hinge loss): Controlled by the data-fitting term.

The parameter λ trades off between these two goals.

4.3 A Note on Convexity

The SVM objective eq. (9) is a *convex function*. This is a crucial property. A function is convex if, roughly speaking, it is "bowl-shaped", that is, it has no local minima other than the global minimum. This means:

- Any local minimum is also the global minimum.
- We can find the optimal $\mathbf{w}$ and b efficiently using gradient-based optimization.

Convexity follows because $\|\mathbf{w}\|^2$ is convex, $\max(0, 1 - t)$ is convex in t (it is piecewise linear), and $t = y_i(\mathbf{w}^\intercal \mathbf{x}_i + b)$ is linear in $(\mathbf{w}, b)$. A sum of convex functions is convex.

Part II
The Problem: What If Our Data Is Uncertain?

5 Why Data Uncertainty Matters

In the standard SVM formulation above, each training example $\mathbf{x}_i$ is treated as a *fixed, known point* in $\mathbb{R}^d$. But in practice, this is often not realistic:

1. **Measurement noise:** Every measurement instrument has some imprecision. The true value is not exactly $\mathbf{x}_i$ but something "close to" $\mathbf{x}_i$, with some uncertainty.
2. **Feature extraction artifacts:** In computer vision, natural language processing, or audio processing, the features are computed via complex pipelines. The extracted features are noisy approximations of the underlying signal.

3. **Aggregated data:** Sometimes $\mathbf{x}_i$ is the *average* of many measurements. For example, in a medical dataset, the reported value might be the average of multiple cell measurements, and the standard deviation is also available.

4. **Data augmentation modeling:** An image can be shifted, rotated, or scaled by small amounts. The set of images produced by these transforms can be modeled as a distribution centered at the original image.

In all these cases, we know (or can estimate) not just a single point $\mathbf{x}_i$, but a *probability distribution* around $\mathbf{x}_i$ that describes where the "true" data point might lie.

6 Why the Standard SVM Fails to Use This Information

The standard SVM takes only the point $\mathbf{x}_i$ (typically the mean) and ignores any associated uncertainty. Consider two scenarios:

- **Scenario A:** A data point $\mathbf{x}_i$ lies very close to the decision boundary, but we are very certain about its position (tiny uncertainty).
- **Scenario B:** The same point $\mathbf{x}_i$ lies close to the boundary, but we are very *uncertain* about its position (large uncertainty that extends across the boundary).

The standard SVM treats these two scenarios identically. But clearly, in Scenario B, there is a significant probability that the true data point actually lies on the wrong side of the boundary. A good classifier should account for this.

Part III
SVM with Gaussian Sample Uncertainty (SVM-GSU)

This part presents the method proposed by Tzelepis et al. (2017). Each training example is modeled not as a fixed point, but as a Gaussian distribution.

Roadmap for Part III

We will proceed in five steps:

1. **Define** the new training set, replacing fixed points with Gaussians (section 7).
2. **Write down** the objective function containing an integral we cannot yet evaluate (section 8).
3. **Evaluate** that integral in closed form using a key theorem (section 9).
4. **Substitute** the result back to obtain the full, explicit objective function.
5. **Differentiate** the objective to get closed-form gradients for optimization (section 10).

At each stage, we will display the objective function so that you always see the bigger picture.

7 The New Training Set

In the standard SVM, the i-th training example was a fixed vector $\mathbf{x}_i$. Now, we make a conceptual shift: what was previously a fixed point is now replaced by a *probability distribution* describing where the true data point might lie. Specifically, we model each training example as a Gaussian distribution with a known mean and covariance.

To make this shift clear in our notation, we use $\boldsymbol{\mu}_i$ to denote the *mean* (center) of the i-th example's distribution, and $\mathbf{x}_i$ to denote a *random vector* drawn from that distribution. The training set becomes:

$$\mathcal{X}' = \Big\{(\boldsymbol{\mu}_i,\, \boldsymbol{\Sigma}_i,\, y_i) : \ \boldsymbol{\mu}_i \in \mathbb{R}^d,\ \boldsymbol{\Sigma}_i \in \mathbb{S}^d_{++},\ y_i \in \{+1, -1\},\ i = 1, \ldots, n\Big\}, \tag{10}$$

where:

- $\boldsymbol{\mu}_i \in \mathbb{R}^d$ is the *mean vector* of the i-th example. This is the "center" of the uncertainty distribution, that is, the best estimate of where the data point lies.
- $\boldsymbol{\Sigma}_i \in \mathbb{S}^d_{++}$ is the *covariance matrix* of the i-th example. It is a $d \times d$ symmetric positive definite matrix that describes the shape and extent of the uncertainty. Here, $\mathbb{S}^d_{++}$ denotes the set of all $d \times d$ symmetric positive definite matrices.
- $y_i \in \{+1, -1\}$ is the class label, as before.

We then define a random vector $\mathbf{x}_i$ for each training example, and assume it follows a multivariate Gaussian (normal) distribution:

$$\mathbf{x}_i \sim \mathcal{N}(\boldsymbol{\mu}_i,\, \boldsymbol{\Sigma}_i)\,. \tag{11}$$

This notation reads naturally: "the i-th data point $\mathbf{x}_i$ is drawn from a Gaussian with mean $\boldsymbol{\mu}_i$ and covariance $\boldsymbol{\Sigma}_i$." The mean $\boldsymbol{\mu}_i$ plays the role that the fixed point $\mathbf{x}_i$ played in the standard SVM, but now we also have the covariance $\boldsymbol{\Sigma}_i$ encoding the uncertainty around it.

7.1 What Is a Multivariate Gaussian Distribution?

For readers who may be less familiar, let us briefly review this concept. A random vector $\mathbf{x} \in \mathbb{R}^d$ follows a multivariate Gaussian distribution with mean $\boldsymbol{\mu}$ and covariance $\boldsymbol{\Sigma}$ if its probability density function (PDF) is:

$$f_{\mathbf{x}}(\mathbf{x}) = \frac{1}{(2\pi)^{d/2}\,|\boldsymbol{\Sigma}|^{1/2}} \exp\!\Big(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^{\intercal}\boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\Big)\,, \tag{12}$$

where:

- $|\boldsymbol{\Sigma}|$ is the determinant of $\boldsymbol{\Sigma}$.
- $\boldsymbol{\Sigma}^{-1}$ is the inverse of $\boldsymbol{\Sigma}$.
- The quantity $(\mathbf{x} - \boldsymbol{\mu})^{\intercal}\boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})$ is called the *Mahalanobis distance* squared. It measures how far $\mathbf{x}$ is from the mean $\boldsymbol{\mu}$, taking into account the shape of the distribution.

The *iso-density contours* (curves where $f_{\mathbf{x}}$ is constant) are *ellipsoids* centered at $\boldsymbol{\mu}$. The shape and orientation of these ellipsoids are determined by $\boldsymbol{\Sigma}$:

- If $\boldsymbol{\Sigma} = \sigma^2\mathbf{I}$ (a scalar times the identity matrix), the contours are *spheres*. The uncertainty is the same in every direction. This is called *isotropic* uncertainty.

- If $\boldsymbol{\Sigma}$ is diagonal (but with different diagonal entries), the contours are axis-aligned ellipsoids. The uncertainty differs along the coordinate axes.
- If $\boldsymbol{\Sigma}$ is a general positive definite matrix, the contours are arbitrarily oriented ellipsoids. This is *anisotropic* uncertainty.

7.2 A Useful Property: Linear Transformations of Gaussians

We will need the following standard fact. If $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ and $\mathbf{a} \in \mathbb{R}^d$ is a fixed vector, then the scalar $\mathbf{a}^\intercal \mathbf{x}$ follows a univariate Gaussian:

$$\mathbf{a}^\intercal \mathbf{x} \sim \mathcal{N}(\mathbf{a}^\intercal \boldsymbol{\mu},\ \mathbf{a}^\intercal \boldsymbol{\Sigma} \mathbf{a})\,. \tag{13}$$

Proof. The mean is $\mathbb{E}\left[\mathbf{a}^\intercal \mathbf{x}\right] = \mathbf{a}^\intercal \mathbb{E}[\mathbf{x}] = \mathbf{a}^\intercal \boldsymbol{\mu}$. The variance is

$$\mathrm{Var}\left(\mathbf{a}^\intercal \mathbf{x}\right) = \mathbb{E}\left[(\mathbf{a}^\intercal \mathbf{x} - \mathbf{a}^\intercal \boldsymbol{\mu})^2\right] = \mathbb{E}\left[\mathbf{a}^\intercal (\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\intercal \mathbf{a}\right] = \mathbf{a}^\intercal \boldsymbol{\Sigma} \mathbf{a}. \tag{14}$$

Gaussianity is preserved under linear transformations (a well-known property of the Gaussian family).

8 The Key Idea: Expected Hinge Loss

Now comes the central idea of SVM-GSU. Instead of computing the hinge loss at the fixed point $\boldsymbol{\mu}_i$ (as the standard SVM would), we compute the *expected value* (average) of the hinge loss over all possible positions of the i-th training example, weighted by the probability of each position under the Gaussian $\mathcal{N}(\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$.

8.1 Why Take the Expected Value?

Think of it this way: the i-th training example is not a single point but a "cloud" of possible points, described by the Gaussian $\mathcal{N}(\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$. Each possible point $\mathbf{x}$ in this cloud would incur a hinge loss of $\max(0, 1 - y_i(\mathbf{w}^\intercal \mathbf{x} + b))$. The expected hinge loss averages this over the entire cloud, weighting each point by how likely it is.

Points that are more probable (near the mean $\boldsymbol{\mu}_i$) contribute more to the expected loss. Points that are less probable (far from $\boldsymbol{\mu}_i$) contribute less. This is exactly the right thing to do if we believe the data comes from this distribution.

8.2 The SVM-GSU Objective Function (Integral Form)

Recall that the standard SVM objective (eq. (9)) was:

$$\mathcal{J}_{\mathrm{SVM}}(\mathbf{w}, b) = \underbrace{\frac{\lambda}{2}\|\mathbf{w}\|^2}_{\text{regularization}} + \frac{1}{n}\sum_{i=1}^{n} \underbrace{\max(0,\ 1 - y_i(\mathbf{w}^\intercal \mathbf{x}_i + b))}_{\text{hinge loss at } \mathbf{x}_i}.$$

The SVM-GSU objective replaces the hinge loss at the fixed point $\mathbf{x}_i$ with the *expected* hinge loss over the Gaussian $\mathcal{N}(\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$. The regularization term stays the same:

$$\mathcal{J}_{\mathrm{GSU}}(\mathbf{w}, b) = \underbrace{\frac{\lambda}{2}\|\mathbf{w}\|^2}_{\text{regularization}} + \frac{1}{n}\sum_{i=1}^{n} \underbrace{\int_{\mathbb{R}^d} \max(0,\ 1 - y_i(\mathbf{w}^\intercal \mathbf{x} + b))\, f_{\mathbf{x}_i}(\mathbf{x})\ \mathrm{d}\mathbf{x}}_{\mathcal{L}_{\mathrm{GSU}\,i}(\mathbf{w},b) \,=\, \text{expected hinge loss for example } i}, \tag{15}$$

where $f_{\mathbf{x}_i}(\mathbf{x})$ is the Gaussian PDF of the i-th example (eq. (12) with $\boldsymbol{\mu} = \boldsymbol{\mu}_i$ and $\boldsymbol{\Sigma} = \boldsymbol{\Sigma}_i$). Note that $\mathbf{x}$ in the integral is a *dummy variable of integration*. It ranges over all possible positions in $\mathbb{R}^d$.

We can write the loss for the i-th example more compactly using expectation notation:

$$\mathcal{L}_{\mathrm{GSU}i}(\mathbf{w}, b) = \mathbb{E}_{\mathbf{x}_i \sim \mathcal{N}(\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)}[\max(0,\ 1 - y_i(\mathbf{w}^\intercal \mathbf{x}_i + b))]\,. \tag{16}$$

SVM-GSU Objective - Integral Form (eq. (15))

$$\mathcal{J}_{\mathrm{GSU}}(\mathbf{w}, b) = \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_{i=1}^{n} \underbrace{\int_{\mathbb{R}^d} \max(0,\ 1 - y_i(\mathbf{w}^\intercal \mathbf{x} + b))\, f_{\mathbf{x}_i}(\mathbf{x})\ \mathrm{d}\mathbf{x}}_{\mathcal{L}_{\mathrm{GSU}i}(\mathbf{w},b)\,=\,\text{expected hinge loss for example } i}\,.$$

The unknowns are $\mathbf{w}$ and b. Everything else ($\boldsymbol{\mu}_i$, $\boldsymbol{\Sigma}_i$, y_i) is given data. The integral $\mathcal{L}_{GSUi}(\mathbf{w}, b)$ is not yet in a usable form. We evaluate it next.

9 Evaluating the Loss in Closed Form

The integral $\mathcal{L}_{\mathrm{GSU}i}(\mathbf{w}, b)$ in eq. (15) looks daunting. Remarkably, it can be evaluated in *closed form*, meaning we can write down an explicit formula without needing to compute the integral numerically.

9.1 Splitting the Integral Over a Halfspace

The hinge loss $\max(0, 1 - y_i(\mathbf{w}^\intercal \mathbf{x} + b))$ is zero when $y_i(\mathbf{w}^\intercal \mathbf{x} + b) \geq 1$, and equals $1 - y_i(\mathbf{w}^\intercal \mathbf{x} + b)$ when $y_i(\mathbf{w}^\intercal \mathbf{x} + b) < 1$. Therefore, the integral is only nonzero over the *halfspace*:

$$\Omega_i = \left\{\mathbf{x} \in \mathbb{R}^d :\ y_i(\mathbf{w}^\intercal \mathbf{x} + b) \leq 1\right\}. \tag{17}$$

This is the set of points that are either misclassified or lie within the margin. So the loss becomes:

$$\mathcal{L}_{\mathrm{GSU}i}(\mathbf{w}, b) = \int_{\Omega_i} [1 - y_i(\mathbf{w}^\intercal \mathbf{x} + b)]\, f_{\mathbf{x}_i}(\mathbf{x})\ \mathrm{d}\mathbf{x}\,. \tag{18}$$

This integral involves integrating a *linear function* times a *Gaussian PDF* over a *halfspace*. This is a well-known type of integral that can be evaluated analytically.

9.2 The Closed-Form Result (Theorem 1 from the Paper)

The paper proves (in Appendix A) the following key result. We state it here, and provide a full proof in Appendix A.

Theorem 9.1 (Gaussian-Weighted Linear Integral Over a Halfspace)**.** *Let $\mathbf{x} \in \mathbb{R}^d$ be a random vector with $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$. Let H be the hyperplane $\mathbf{a}^\intercal \mathbf{x} + b' = 0$, which divides $\mathbb{R}^d$ into two halfspaces,*

$$\Omega^{\pm} = \left\{\mathbf{x} \in \mathbb{R}^d : \mathbf{a}^\intercal \mathbf{x} + b' \gtrless 0\right\}. \tag{19}$$

Then the integrals

$$I^{\pm}(\mathbf{a}, b') \equiv \int_{\Omega^{\pm}} (\mathbf{a}^\intercal \mathbf{x} + b')\, f_{\mathbf{x}}(\mathbf{x})\ \mathrm{d}\mathbf{x} \tag{20}$$

evaluate to

$$I^{\pm}(\mathbf{a}, b') = \frac{d_{\boldsymbol{\mu}}}{2}\left(1 \pm \operatorname{erf}\left(\frac{d_{\boldsymbol{\mu}}}{d_{\boldsymbol{\Sigma}}}\right)\right) \pm \frac{d_{\boldsymbol{\Sigma}}}{2\sqrt{\pi}} \exp\left(-\frac{d_{\boldsymbol{\mu}}^2}{d_{\boldsymbol{\Sigma}}^2}\right), \tag{21}$$

where

$$d_{\boldsymbol{\mu}} = \mathbf{a}^\intercal \boldsymbol{\mu} + b', \tag{22}$$
$$d_{\boldsymbol{\Sigma}} = \sqrt{2\,\mathbf{a}^\intercal \boldsymbol{\Sigma}\,\mathbf{a}}, \tag{23}$$

and $\operatorname{erf}(\cdot)$ *is the* error function, *defined as:*

$$\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2}\ \mathrm{d}t\,. \tag{24}$$

9.3 What Is the Error Function?

The error function $\operatorname{erf}(x)$ is closely related to the cumulative distribution function (CDF) of the standard normal distribution in the following way:

$$\Phi(x) = \frac{1}{2}\left(1 + \operatorname{erf}\left(\frac{x}{\sqrt{2}}\right)\right).$$

This error function maps $\mathbb{R}$ to $(-1, 1)$, with the following properties:

- $\operatorname{erf}(0) = 0$.
- $\lim_{x\to\infty} \operatorname{erf}(x) = 1$.
- $\lim_{x\to-\infty} \operatorname{erf}(x) = -1$.
- It is an odd function: $\operatorname{erf}(-x) = -\operatorname{erf}(x)$.

It appears naturally whenever we integrate Gaussian functions over half-lines or halfspaces. It is available as a standard function in essentially all numerical computing libraries.

An important identity we will use repeatedly is the derivative of the error function:

$$\frac{\mathrm{d}}{\mathrm{d}x}\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} e^{-x^2}. \tag{25}$$

This follows directly from eq. (24) by the fundamental theorem of calculus.

9.4 Applying the Theorem to Obtain the SVM-GSU Loss

To apply theorem 9.1 to our loss function eq. (18), we need to match the notation. The halfspace Ω_i is defined by $y_i(\mathbf{w}^\intercal \mathbf{x} + b) \le 1$. Rearranging:

$$y_i(\mathbf{w}^\intercal \mathbf{x} + b) \le 1 \iff y_i \mathbf{w}^\intercal \mathbf{x} + y_i b - 1 \le 0 \iff -y_i \mathbf{w}^\intercal \mathbf{x} + (1 - y_i b) \ge 0. \tag{26}$$

The integrand is $[1 - y_i(\mathbf{w}^\intercal \mathbf{x} + b)]$, which equals $(-y_i \mathbf{w})^\intercal \mathbf{x} + (1 - y_i b)$. Comparing with the theorem's integrand $\mathbf{a}^\intercal \mathbf{x} + b'$ over the halfspace $\Omega^+ = \{\mathbf{x} : \mathbf{a}^\intercal \mathbf{x} + b' \ge 0\}$, we identify:

$$\mathbf{a} = -y_i \mathbf{w}, \qquad b' = 1 - y_i b. \tag{27}$$

Substituting into eqs. (22) and (23):

$$\begin{aligned} d_{\boldsymbol{\mu}_i} \equiv d_{\boldsymbol{\mu}} &= (-y_i \mathbf{w})^\intercal \boldsymbol{\mu}_i + (1 - y_i b) = -y_i \mathbf{w}^\intercal \boldsymbol{\mu}_i + 1 - y_i b \\ &= 1 - y_i(\mathbf{w}^\intercal \boldsymbol{\mu}_i + b), \end{aligned} \tag{28}$$

$$\begin{aligned} d_{\boldsymbol{\Sigma}_i} \equiv d_{\boldsymbol{\Sigma}} &= \sqrt{2\,(-y_i \mathbf{w})^\intercal \boldsymbol{\Sigma}_i\,(-y_i \mathbf{w})} = \sqrt{2\,y_i^2\,\mathbf{w}^\intercal \boldsymbol{\Sigma}_i\,\mathbf{w}} \\ &= \sqrt{2\,\mathbf{w}^\intercal \boldsymbol{\Sigma}_i\,\mathbf{w}}, \end{aligned} \tag{29}$$

where in the last step we used $y_i^2 = 1$ (since $y_i \in \{+1, -1\}$). Notice that $d_{\boldsymbol{\Sigma}_i}$ does not depend on y_i or on b. Applying the "+" branch of eq. (21), the closed-form loss for the i-th training example is:

$$\mathcal{L}_{\mathrm{GSU}\,i}(\mathbf{w}, b) = \frac{d_{\boldsymbol{\mu}_i}}{2}\left[\operatorname{erf}\left(\frac{d_{\boldsymbol{\mu}_i}}{d_{\boldsymbol{\Sigma}_i}}\right) + 1\right] + \frac{d_{\boldsymbol{\Sigma}_i}}{2\sqrt{\pi}} \exp\left(-\frac{d_{\boldsymbol{\mu}_i}^2}{d_{\boldsymbol{\Sigma}_i}^2}\right). \tag{30}$$

9.5 The Full Objective with the Closed-Form Loss

Now we substitute eq. (30) back into the objective eq. (15). The integral has been eliminated and the objective is now a fully explicit function of $\mathbf{w}$ and b:

$$\mathcal{J}_{\mathrm{GSU}}(\mathbf{w}, b) = \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_{i=1}^{n} \underbrace{\left[\frac{d_{\boldsymbol{\mu}_i}}{2}\left(\operatorname{erf}\left(\frac{d_{\boldsymbol{\mu}_i}}{d_{\boldsymbol{\Sigma}_i}}\right) + 1\right) + \frac{d_{\boldsymbol{\Sigma}_i}}{2\sqrt{\pi}} \exp\left(-\frac{d_{\boldsymbol{\mu}_i}^2}{d_{\boldsymbol{\Sigma}_i}^2}\right)\right]}_{\mathcal{L}_{\mathrm{GSU}\,i}(\mathbf{w},b)}, \tag{31}$$

where $d_{\boldsymbol{\mu}_i} = 1 - y_i(\mathbf{w}^\intercal \boldsymbol{\mu}_i + b)$ and $d_{\boldsymbol{\Sigma}_i} = \sqrt{2\,\mathbf{w}^\intercal \boldsymbol{\Sigma}_i\, \mathbf{w}}$.

SVM-GSU Objective - Closed Form (eq. (31))

$$\mathcal{J}_{\mathrm{GSU}}(\mathbf{w}, b) = \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_{i=1}^{n} \underbrace{\left[\frac{d_{\boldsymbol{\mu}_i}}{2}\left(\operatorname{erf}\left(\frac{d_{\boldsymbol{\mu}_i}}{d_{\boldsymbol{\Sigma}_i}}\right) + 1\right) + \frac{d_{\boldsymbol{\Sigma}_i}}{2\sqrt{\pi}} \exp\left(-\frac{d_{\boldsymbol{\mu}_i}^2}{d_{\boldsymbol{\Sigma}_i}^2}\right)\right]}_{\mathcal{L}_{\mathrm{GSU}\,i}(\mathbf{w},b)}$$

The only unknowns are $\mathbf{w}$ and b. The quantities $d_{\boldsymbol{\mu}_i}$ and $d_{\boldsymbol{\Sigma}_i}$ are shorthand that depend on $\mathbf{w}$, b, and the given data ($\boldsymbol{\mu}_i$, $\boldsymbol{\Sigma}_i$, y_i). No integrals remain. This is what we minimize.

9.6 Understanding the Two Quantities

Let us build intuition for the two key quantities $d_{\boldsymbol{\mu}_i}$ and $d_{\boldsymbol{\Sigma}_i}$ appearing in the objective:

- $d_{\boldsymbol{\mu}_i} = 1 - y_i(\mathbf{w}^\intercal \boldsymbol{\mu}_i + b)$:
  - This is exactly the hinge loss argument of the *mean* $\boldsymbol{\mu}_i$ in the standard SVM. If the mean is correctly classified and lies outside the margin, $d_{\boldsymbol{\mu}_i} < 0$. If the mean lies inside the margin or is misclassified, $d_{\boldsymbol{\mu}_i} > 0$.
- $d_{\boldsymbol{\Sigma}_i} = \sqrt{2\,\mathbf{w}^\intercal \boldsymbol{\Sigma}_i\, \mathbf{w}}$:
  - This measures the *spread of the uncertainty in the direction of* $\mathbf{w}$. Using the linear-transformation property discussed in section 7.2, if $\mathbf{x}_i \sim \mathcal{N}(\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$, then $\mathbf{w}^\intercal \mathbf{x}_i \sim \mathcal{N}(\mathbf{w}^\intercal \boldsymbol{\mu}_i, \mathbf{w}^\intercal \boldsymbol{\Sigma}_i \mathbf{w})$. So $\mathbf{w}^\intercal \boldsymbol{\Sigma}_i \mathbf{w}$ is exactly the variance of the projection of $\mathbf{x}_i$ onto the direction $\mathbf{w}$, and $d_{\boldsymbol{\Sigma}_i}$ is $\sqrt{2}$ times the standard deviation. This captures how much the uncertainty "matters" for classification, specifically, how much the Gaussian is spread out in the direction perpendicular to the decision boundary (since $\mathbf{w}$ is the normal vector to the hyperplane).

Remark 9.1 (Sanity check)**.** *Look at eq. (31). The first term is the regularizer, identical to standard SVM. The second term is a sum over training examples, where each contribution depends on two things: how far the mean $\boldsymbol{\mu}_i$ is from the decision boundary (via $d_{\boldsymbol{\mu}_i}$) and how spread out the uncertainty is in the classification-relevant direction (via $d_{\boldsymbol{\Sigma}_i}$). If a training example has large uncertainty that straddles the boundary, its loss contribution will be large, even if its mean is on the correct side.*

9.7 Limiting Behavior: Recovering Standard SVM

What happens in eq. (31) when the uncertainty vanishes, i.e., $\boldsymbol{\Sigma}_i \to \mathbf{0}$? Let us verify that SVM-GSU recovers the standard SVM hinge loss in this limit. As $\boldsymbol{\Sigma}_i \to \mathbf{0}$, we have $d_{\boldsymbol{\Sigma}_i} = \sqrt{2\mathbf{w}^\intercal \boldsymbol{\Sigma}_i \mathbf{w}} \to 0^+$.

Case 1: $d_{\boldsymbol{\mu}_i} > 0$ (mean is inside margin or misclassified). Then $\frac{d_{\boldsymbol{\mu}_i}}{d_{\boldsymbol{\Sigma}_i}} \to +\infty$, so we get $\operatorname{erf}\left(\frac{d_{\boldsymbol{\mu}_i}}{d_{\boldsymbol{\Sigma}_i}}\right) \to 1$. The first term becomes

$$\frac{d_{\boldsymbol{\mu}_i}}{2}(1+1) = d_{\boldsymbol{\mu}_i}. \tag{32}$$

For the second term, $\exp\left(-\frac{d^2_{\boldsymbol{\mu}_i}}{d^2_{\boldsymbol{\Sigma}_i}}\right) \to 0$ faster than any polynomial, and $d_{\boldsymbol{\Sigma}_i} \to 0$, so the product $\frac{d_{\boldsymbol{\Sigma}_i}}{2\sqrt{\pi}} \exp\left(-\frac{d^2_{\boldsymbol{\mu}_i}}{d^2_{\boldsymbol{\Sigma}_i}}\right) \to 0$. Thus $\mathcal{L}_{\mathrm{GSU}i} \to d_{\boldsymbol{\mu}_i} = 1 - y_i(\mathbf{w}^\intercal \boldsymbol{\mu}_i + b) > 0$.

Case 2: $d_{\boldsymbol{\mu}_i} < 0$ (mean is correctly classified, outside margin). Then $\frac{d_{\boldsymbol{\mu}_i}}{d_{\boldsymbol{\Sigma}_i}} \to -\infty$, so $\operatorname{erf}\left(\frac{d_{\boldsymbol{\mu}_i}}{d_{\boldsymbol{\Sigma}_i}}\right) \to -1$. The first term becomes

$$\frac{d_{\boldsymbol{\mu}_i}}{2}(-1+1) = 0. \tag{33}$$

The second term again vanishes. Thus $\mathcal{L}_{\mathrm{GSU}i} \to 0$.

Conclusion. In both cases, $\mathcal{L}_{\mathrm{GSU}i} \to \max(0, d_{\boldsymbol{\mu}_i}) = \max(0, 1 - y_i(\mathbf{w}^\intercal \boldsymbol{\mu}_i + b))$, which is exactly the standard SVM hinge loss evaluated at the mean. This confirms that **SVM-GSU is a strict generalization of the standard SVM**. The standard SVM is recovered as a special case when all covariance matrices tend to zero.

10 Gradients in Closed Form

To minimize the objective $\mathcal{J}_{\mathrm{GSU}}(\mathbf{w}, b)$ (i.e., eq. (31)) using gradient descent, we need its partial derivatives with respect to $\mathbf{w}$ and b. We derive them below in full detail.

10.1 Setup

We restate the objective:

$$\mathcal{J}_{\mathrm{GSU}}(\mathbf{w}, b) = \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_{i=1}^{n} \underbrace{\left[\frac{d_{\boldsymbol{\mu}_i}}{2}\left(\operatorname{erf}\left(\frac{d_{\boldsymbol{\mu}_i}}{d_{\boldsymbol{\Sigma}_i}}\right) + 1\right) + \frac{d_{\boldsymbol{\Sigma}_i}}{2\sqrt{\pi}} \exp\left(-\frac{d^2_{\boldsymbol{\mu}_i}}{d^2_{\boldsymbol{\Sigma}_i}}\right)\right]}_{\mathcal{L}_{\mathrm{GSU}i}(\mathbf{w}, b)}, \tag{34}$$

where

$$\begin{aligned} d_{\boldsymbol{\mu}_i} &= 1 - y_i(\mathbf{w}^\intercal \boldsymbol{\mu}_i + b), \\ d_{\boldsymbol{\Sigma}_i} &= \sqrt{2\, \mathbf{w}^\intercal \boldsymbol{\Sigma}_i\, \mathbf{w}}. \end{aligned}$$

We observe:

- $\mathcal{L}_{\text{GSU}i}$ depends on $\mathbf{w}$ through *both* $d_{\boldsymbol{\mu}_i}$ (linearly, via $\mathbf{w}^\intercal \boldsymbol{\mu}_i$) and $d_{\boldsymbol{\Sigma}_i}$ (via the quadratic form $\mathbf{w}^\intercal \boldsymbol{\Sigma}_i \mathbf{w}$).
- $\mathcal{L}_{\text{GSU}i}$ depends on b only through $d_{\boldsymbol{\mu}_i}$.

We will therefore use the chain rule. Let $r_i \equiv \frac{d_{\boldsymbol{\mu}_i}}{d_{\boldsymbol{\Sigma}_i}}$ for brevity.

10.2 Step 1: Compute $\frac{\partial \mathcal{L}_{\text{GSU}i}}{\partial d_{\mu_i}}$

We differentiate eq. (30) with respect to $d_{\boldsymbol{\mu}_i}$, treating $d_{\boldsymbol{\Sigma}_i}$ as a constant (the dependence of $d_{\boldsymbol{\Sigma}_i}$ on $\mathbf{w}$ will be handled separately). The loss is

$$\mathcal{L}_{\text{GSU}i} = \underbrace{\frac{d_{\boldsymbol{\mu}_i}}{2} \left(\operatorname{erf}(r_i) + 1\right)}_{\text{term A}} + \underbrace{\frac{d_{\boldsymbol{\Sigma}_i}}{2\sqrt{\pi}} \exp\left(-r_i^2\right)}_{\text{term B}}. \tag{35}$$

Derivative of term A. By the product rule, treating $r_i = \frac{d_{\boldsymbol{\mu}_i}}{d_{\boldsymbol{\Sigma}_i}}$:

$$\begin{aligned}
\frac{\partial \mathrm{A}}{\partial d_{\boldsymbol{\mu}_i}} &= \frac{1}{2}(\operatorname{erf}(r_i) + 1) + \frac{d_{\boldsymbol{\mu}_i}}{2} \cdot \frac{\partial}{\partial d_{\boldsymbol{\mu}_i}} \operatorname{erf}(r_i) \\
&= \frac{1}{2}(\operatorname{erf}(r_i) + 1) + \frac{d_{\boldsymbol{\mu}_i}}{2} \cdot \operatorname{erf}'(r_i) \cdot \frac{\partial r_i}{\partial d_{\boldsymbol{\mu}_i}}.
\end{aligned} \tag{36}$$

Using eq. (25), $\operatorname{erf}'(r_i) = \frac{2}{\sqrt{\pi}} e^{-r_i^2}$, and $\frac{\partial r_i}{\partial d_{\boldsymbol{\mu}_i}} = \frac{1}{d_{\boldsymbol{\Sigma}_i}}$:

$$\frac{\partial \mathrm{A}}{\partial d_{\boldsymbol{\mu}_i}} = \frac{1}{2}(\operatorname{erf}(r_i) + 1) + \frac{d_{\boldsymbol{\mu}_i}}{2} \cdot \frac{2}{\sqrt{\pi}} e^{-r_i^2} \cdot \frac{1}{d_{\boldsymbol{\Sigma}_i}} = \frac{1}{2}(\operatorname{erf}(r_i) + 1) + \frac{d_{\boldsymbol{\mu}_i} e^{-r_i^2}}{\sqrt{\pi}\, d_{\boldsymbol{\Sigma}_i}}. \tag{37}$$

Derivative of term B. Here $d_{\boldsymbol{\Sigma}_i}$ is treated as constant and only r_i depends on $d_{\boldsymbol{\mu}_i}$:

$$\frac{\partial \mathrm{B}}{\partial d_{\boldsymbol{\mu}_i}} = \frac{d_{\boldsymbol{\Sigma}_i}}{2\sqrt{\pi}} \cdot e^{-r_i^2} \cdot (-2r_i) \cdot \frac{\partial r_i}{\partial d_{\boldsymbol{\mu}_i}} = \frac{d_{\boldsymbol{\Sigma}_i}}{2\sqrt{\pi}} \cdot e^{-r_i^2} \cdot (-2r_i) \cdot \frac{1}{d_{\boldsymbol{\Sigma}_i}} = -\frac{r_i\, e^{-r_i^2}}{\sqrt{\pi}}. \tag{38}$$

Substituting $r_i = \frac{d_{\boldsymbol{\mu}_i}}{d_{\boldsymbol{\Sigma}_i}}$:

$$\frac{\partial \mathrm{B}}{\partial d_{\boldsymbol{\mu}_i}} = -\frac{d_{\boldsymbol{\mu}_i}\, e^{-r_i^2}}{\sqrt{\pi}\, d_{\boldsymbol{\Sigma}_i}}. \tag{39}$$

Sum. Adding the two derivatives, the terms $\frac{d_{\boldsymbol{\mu}_i} e^{-r_i^2}}{\sqrt{\pi} d_{\boldsymbol{\Sigma}_i}}$ and $-\frac{d_{\boldsymbol{\mu}_i} e^{-r_i^2}}{\sqrt{\pi} d_{\boldsymbol{\Sigma}_i}}$ cancel:

$$\boxed{\frac{\partial \mathcal{L}_{\text{GSU}i}}{\partial d_{\boldsymbol{\mu}_i}} = \frac{1}{2}(\operatorname{erf}(r_i) + 1).} \tag{40}$$

This is a remarkably clean result.

10.3 Step 2: Compute $\frac{\partial \mathcal{L}_{\text{GSU}i}}{\partial d_{\boldsymbol{\Sigma}_i}}$

Now we differentiate with respect to $d_{\boldsymbol{\Sigma}_i}$, treating $d_{\boldsymbol{\mu}_i}$ as constant.

Derivative of term A. Only r_i depends on $d_{\boldsymbol{\Sigma}_i}$ here, since the coefficient $\frac{d_{\boldsymbol{\mu}_i}}{2}$ is constant. Using $\frac{\partial r_i}{\partial d_{\boldsymbol{\Sigma}_i}} = -\frac{d_{\boldsymbol{\mu}_i}}{d^2_{\boldsymbol{\Sigma}_i}}$:

$$\frac{\partial \mathrm{A}}{\partial d_{\boldsymbol{\Sigma}_i}} = \frac{d_{\boldsymbol{\mu}_i}}{2} \cdot \operatorname{erf}'(r_i) \cdot \frac{\partial r_i}{\partial d_{\boldsymbol{\Sigma}_i}} = \frac{d_{\boldsymbol{\mu}_i}}{2} \cdot \frac{2}{\sqrt{\pi}} e^{-r_i^2} \cdot \left(-\frac{d_{\boldsymbol{\mu}_i}}{d^2_{\boldsymbol{\Sigma}_i}}\right) = -\frac{d^2_{\boldsymbol{\mu}_i}\, e^{-r_i^2}}{\sqrt{\pi}\, d^2_{\boldsymbol{\Sigma}_i}}. \tag{41}$$

Derivative of term B. By the product rule:

$$\begin{aligned}
\frac{\partial \mathrm{B}}{\partial d_{\boldsymbol{\Sigma}_i}} &= \frac{1}{2\sqrt{\pi}} e^{-r_i^2} + \frac{d_{\boldsymbol{\Sigma}_i}}{2\sqrt{\pi}} \cdot e^{-r_i^2} \cdot (-2r_i) \cdot \frac{\partial r_i}{\partial d_{\boldsymbol{\Sigma}_i}} \\
&= \frac{e^{-r_i^2}}{2\sqrt{\pi}} + \frac{d_{\boldsymbol{\Sigma}_i}}{2\sqrt{\pi}} \cdot e^{-r_i^2} \cdot (-2r_i) \cdot \left(-\frac{d_{\boldsymbol{\mu}_i}}{d^2_{\boldsymbol{\Sigma}_i}}\right) \\
&= \frac{e^{-r_i^2}}{2\sqrt{\pi}} + \frac{r_i d_{\boldsymbol{\mu}_i}\, e^{-r_i^2}}{\sqrt{\pi}\, d_{\boldsymbol{\Sigma}_i}} \\
&= \frac{e^{-r_i^2}}{2\sqrt{\pi}} + \frac{d^2_{\boldsymbol{\mu}_i}\, e^{-r_i^2}}{\sqrt{\pi}\, d^2_{\boldsymbol{\Sigma}_i}},
\end{aligned} \tag{42}$$

where in the last step we used $r_i = \frac{d_{\boldsymbol{\mu}_i}}{d_{\boldsymbol{\Sigma}_i}}$.

Sum. Adding, the terms $-\frac{d^2_{\boldsymbol{\mu}_i} e^{-r_i^2}}{\sqrt{\pi} d^2_{\boldsymbol{\Sigma}_i}}$ and $+\frac{d^2_{\boldsymbol{\mu}_i} e^{-r_i^2}}{\sqrt{\pi} d^2_{\boldsymbol{\Sigma}_i}}$ cancel:

$$\boxed{\frac{\partial \mathcal{L}_{\mathrm{GSU}i}}{\partial d_{\boldsymbol{\Sigma}_i}} = \frac{e^{-r_i^2}}{2\sqrt{\pi}}.} \tag{43}$$

10.4 Step 3: Compute $\frac{\partial d_{\mu_i}}{\partial \mathbf{w}}$ and $\frac{\partial d_{\mu_i}}{\partial b}$

Since $d_{\boldsymbol{\mu}_i} = 1 - y_i(\mathbf{w}^\intercal \boldsymbol{\mu}_i + b) = 1 - y_i \mathbf{w}^\intercal \boldsymbol{\mu}_i - y_i b$:

$$\frac{\partial d_{\boldsymbol{\mu}_i}}{\partial \mathbf{w}} = -y_i \boldsymbol{\mu}_i, \tag{44}$$

$$\frac{\partial d_{\boldsymbol{\mu}_i}}{\partial b} = -y_i. \tag{45}$$

10.5 Step 4: Compute $\frac{\partial d_{\boldsymbol{\Sigma}_i}}{\partial \mathbf{w}}$

We have $d_{\boldsymbol{\Sigma}_i} = \sqrt{2\mathbf{w}^\intercal \boldsymbol{\Sigma}_i \mathbf{w}}$. First, note that for a symmetric matrix $\boldsymbol{\Sigma}_i$,

$$\frac{\partial}{\partial \mathbf{w}} (\mathbf{w}^\intercal \boldsymbol{\Sigma}_i \mathbf{w}) = (\boldsymbol{\Sigma}_i + \boldsymbol{\Sigma}_i^\intercal)\mathbf{w} = 2\boldsymbol{\Sigma}_i \mathbf{w}. \tag{46}$$

Therefore, by the chain rule:

$$\frac{\partial d_{\boldsymbol{\Sigma}_i}}{\partial \mathbf{w}} = \frac{1}{2} (2\mathbf{w}^\intercal \boldsymbol{\Sigma}_i \mathbf{w})^{-1/2} \cdot 2 \cdot 2\boldsymbol{\Sigma}_i \mathbf{w} = \frac{2\boldsymbol{\Sigma}_i \mathbf{w}}{\sqrt{2\mathbf{w}^\intercal \boldsymbol{\Sigma}_i \mathbf{w}}} = \frac{2\boldsymbol{\Sigma}_i \mathbf{w}}{d_{\boldsymbol{\Sigma}_i}}. \tag{47}$$

Also, $d_{\boldsymbol{\Sigma}_i}$ does not depend on b:

$$\frac{\partial d_{\boldsymbol{\Sigma}_i}}{\partial b} = \mathbf{0}. \tag{48}$$

10.6 Step 5: Assemble the Gradient with Respect to $\mathbf{w}$

By the chain rule:

$$\frac{\partial \mathcal{L}_{\mathrm{GSU}i}}{\partial \mathbf{w}} = \frac{\partial \mathcal{L}_{\mathrm{GSU}i}}{\partial d_{\boldsymbol{\mu}_i}} \cdot \frac{\partial d_{\boldsymbol{\mu}_i}}{\partial \mathbf{w}} + \frac{\partial \mathcal{L}_{\mathrm{GSU}i}}{\partial d_{\boldsymbol{\Sigma}_i}} \cdot \frac{\partial d_{\boldsymbol{\Sigma}_i}}{\partial \mathbf{w}}. \tag{49}$$

Substituting eqs. (40), (43), (44) and (47):

$$\begin{aligned} \frac{\partial \mathcal{L}_{\mathrm{GSU}i}}{\partial \mathbf{w}} &= \frac{1}{2}(\operatorname{erf}(r_i)+1)\cdot(-y_i\boldsymbol{\mu}_i) + \frac{e^{-r_i^2}}{2\sqrt{\pi}} \cdot \frac{2\boldsymbol{\Sigma}_i\mathbf{w}}{d_{\boldsymbol{\Sigma}_i}} \\ &= \frac{e^{-r_i^2}}{\sqrt{\pi}\, d_{\boldsymbol{\Sigma}_i}}\boldsymbol{\Sigma}_i\mathbf{w} - \frac{y_i}{2}[\operatorname{erf}(r_i)+1]\,\boldsymbol{\mu}_i. \end{aligned} \tag{50}$$

Substituting back $r_i = \frac{d_{\boldsymbol{\mu}_i}}{d_{\boldsymbol{\Sigma}_i}}$ and adding the regularizer's gradient $\lambda\mathbf{w}$:

$$\boxed{\frac{\partial \mathcal{J}_{\mathrm{GSU}}}{\partial \mathbf{w}} = \lambda\mathbf{w} + \frac{1}{n}\sum_{i=1}^{n}\left[\frac{\exp\left(-\frac{d_{\boldsymbol{\mu}_i}^2}{d_{\boldsymbol{\Sigma}_i}^2}\right)}{\sqrt{\pi}\, d_{\boldsymbol{\Sigma}_i}}\boldsymbol{\Sigma}_i\mathbf{w} - \frac{y_i}{2}\left(\operatorname{erf}\left(\frac{d_{\boldsymbol{\mu}_i}}{d_{\boldsymbol{\Sigma}_i}}\right)+1\right)\boldsymbol{\mu}_i\right].} \tag{51}$$

The two terms inside the sum have clear interpretations:

1. The first term, involving $\boldsymbol{\Sigma}_i\mathbf{w}$, arises from the dependence of $d_{\boldsymbol{\Sigma}_i}$ on $\mathbf{w}$. It has no analog in the standard SVM and is the new contribution due to uncertainty.
2. The second term, involving $\boldsymbol{\mu}_i$, is analogous to the gradient of the hinge loss in standard SVM, but modulated by a "soft" factor $\frac{1}{2}(\operatorname{erf}(\cdot)+1)$ that smoothly transitions between 0 and 1 instead of the hard step function in standard SVM.

10.7 Step 6: Assemble the Gradient with Respect to b

Since $d_{\boldsymbol{\Sigma}_i}$ does not depend on b (eq. (48)), only the $d_{\boldsymbol{\mu}_i}$ pathway contributes:

$$\frac{\partial \mathcal{L}_{\mathrm{GSU}i}}{\partial b} = \frac{\partial \mathcal{L}_{\mathrm{GSU}i}}{\partial d_{\boldsymbol{\mu}_i}} \cdot \frac{\partial d_{\boldsymbol{\mu}_i}}{\partial b} = \frac{1}{2}(\operatorname{erf}(r_i)+1)\cdot(-y_i) = -\frac{y_i}{2}(\operatorname{erf}(r_i)+1). \tag{52}$$

Summing over all examples (the regularizer does not depend on b):

$$\boxed{\frac{\partial \mathcal{J}_{\mathrm{GSU}}}{\partial b} = -\frac{1}{n}\sum_{i=1}^{n}\frac{y_i}{2}\left[\operatorname{erf}\left(\frac{d_{\boldsymbol{\mu}_i}}{d_{\boldsymbol{\Sigma}_i}}\right)+1\right].} \tag{53}$$

Remark 10.1 (Note on the original paper's gradient expressions)**.** *The gradient expressions given in the original paper (Eqs. (7)–(8) of Tzelepis et al. (2017)) appear to contain transcription errors. Specifically, their Eq. (7) omits the factor y_i in the term multiplying $\boldsymbol{\mu}_i$ (their $\mathbf{x}_i$), and their Eq. (8) omits both the factor y_i and the factor $\frac{1}{2}$ from the bias gradient. Our expressions eqs. (51) and (53) follow from a careful application of the chain rule, as shown above, and can be verified against numerical (finite-difference) gradients.*

10.8 Computational Cost

The key computational cost per training example is:

- $\mathbf{w}^\intercal \boldsymbol{\mu}_i$: an inner product, $\mathcal{O}(d)$. Same as standard SVM.
- $\mathbf{w}^\intercal \boldsymbol{\Sigma}_i \mathbf{w}$: a quadratic form. $\mathcal{O}(d^2)$ for a general covariance matrix, but $\mathcal{O}(d)$ if $\boldsymbol{\Sigma}_i$ is diagonal (the common case in practice).

Remark 10.2 (Sanity check)**.** *Both* $\mathbf{w}^\intercal \boldsymbol{\mu}_i$ *and* $\mathbf{w}^\intercal \boldsymbol{\Sigma}_i \mathbf{w}$ *need to be computed only* once *per example per iteration, since they appear in both the loss and the gradient formulas. So the cost of SVM-GSU with diagonal covariances is essentially the same as the cost of standard SVM.*

11 Convexity of the SVM-GSU Objective

Proposition 11.1. *The SVM-GSU objective* $\mathcal{J}_{GSU}(\mathbf{w}, b)$ *given by eq.* (31) *is strongly convex with respect to* $(\mathbf{w}, b)$*.*

Proof. We break the objective into parts and use standard preservation-of-convexity results.

Regularizer. The function $(\mathbf{w}, b) \mapsto \frac{\lambda}{2}\|\mathbf{w}\|^2$ is strongly convex in $\mathbf{w}$ (its Hessian is $\lambda \mathbf{I}$, positive definite) and constant (hence convex) in b. Therefore, it is strongly convex in $(\mathbf{w}, b)$ jointly.

Each per-example loss. Consider the integrand at a fixed $\mathbf{x}$:

$$h_i(\mathbf{w}, b; \mathbf{x}) = \max(0, 1 - y_i(\mathbf{w}^\intercal \mathbf{x} + b)). \tag{54}$$

The argument inside the max, $1 - y_i(\mathbf{w}^\intercal \mathbf{x} + b)$, is an affine (hence convex) function of $(\mathbf{w}, b)$. The function $\max(0, \cdot)$ is convex and non-decreasing. The composition of a convex non-decreasing function with a convex function is convex. Hence $h_i(\cdot, \cdot; \mathbf{x})$ is convex in $(\mathbf{w}, b)$ for every fixed $\mathbf{x}$.

The per-example loss is

$$\mathcal{L}_{\mathrm{GSU}i}(\mathbf{w}, b) = \int_{\mathbb{R}^d} h_i(\mathbf{w}, b; \mathbf{x}) f_{\mathbf{x}_i}(\mathbf{x}) \, \mathrm{d}\mathbf{x}\,. \tag{55}$$

This is an integral of convex functions against a non-negative weight $f_{\mathbf{x}_i}(\mathbf{x})$. Equivalently, for any $\alpha \in [0, 1]$ and any $(\mathbf{w}_1, b_1), (\mathbf{w}_2, b_2)$:

$$\begin{aligned}
\mathcal{L}_{\mathrm{GSU}i}(\alpha(\mathbf{w}_1, b_1) + (1-\alpha)(\mathbf{w}_2, b_2)) &= \int h_i(\alpha(\mathbf{w}_1, b_1) + (1-\alpha)(\mathbf{w}_2, b_2); \mathbf{x})\, f_{\mathbf{x}_i}(\mathbf{x}) \, \mathrm{d}\mathbf{x} \\
&\le \int [\alpha h_i(\mathbf{w}_1, b_1; \mathbf{x}) + (1-\alpha) h_i(\mathbf{w}_2, b_2; \mathbf{x})]\, f_{\mathbf{x}_i}(\mathbf{x}) \, \mathrm{d}\mathbf{x} \\
&= \alpha\, \mathcal{L}_{\mathrm{GSU}i}(\mathbf{w}_1, b_1) + (1-\alpha)\, \mathcal{L}_{\mathrm{GSU}i}(\mathbf{w}_2, b_2).
\end{aligned} \tag{56}$$

So each $\mathcal{L}_{\mathrm{GSU}i}$ is convex.

Conclusion. A non-negative combination of convex functions is convex, so the average $\frac{1}{n}\sum_i \mathcal{L}_{\mathrm{GSU}i}$ is convex. Adding the strongly convex regularizer yields a strongly convex objective, which therefore has a unique global minimum. □

12 The SGD Solver (Pegasos-Style)

The SVM-GSU optimization problem is solved using a stochastic gradient descent (SGD) algorithm inspired by the Pegasos algorithm. At each iteration, instead of computing the gradient using the entire training set (which can be expensive for large n), we randomly sample a small subset ("mini-batch") and compute an approximate gradient.

12.1 Algorithm Description

Algorithm 1 presents the stochastic sub-gradient descent procedure for solving the SVM-GSU optimization problem.

Algorithm 1 Stochastic sub-gradient descent for SVM-GSU (Pegasos-style).

Require: Training set $\mathcal{X}'$, regularization parameter $\lambda > 0$, number of iterations $T \in \mathbb{N}$, mini-batch size $k \in \mathbb{N}$.

Ensure: Learned parameters $\mathbf{w}^{(T+1)}$, $b^{(T+1)}$.

1: Initialize $b^{(1)} \leftarrow 0$ and $\mathbf{w}^{(1)}$ such that $\|\mathbf{w}^{(1)}\| \leq 1/\sqrt{\lambda}$.
2: **for** $t = 1, 2, \ldots, T$ **do**
3: Sample a random mini-batch $\mathcal{X}_t \subseteq \mathcal{X}'$ with $|\mathcal{X}_t| = k$.
4: Set learning rate $\eta_t \leftarrow 1/\lambda t$.
5: Compute $\frac{\partial \mathcal{J}_{\mathrm{GSU}}}{\partial \mathbf{w}}$ and $\frac{\partial \mathcal{J}_{\mathrm{GSU}}}{\partial b}$ using eqs. (51) and (53) on $\mathcal{X}_t$.
6: $\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \eta_t \frac{\partial \mathcal{J}_{\mathrm{GSU}}}{\partial \mathbf{w}}$ ▷ Gradient step on $\mathbf{w}$
7: $\mathbf{w}^{(t+1)} \leftarrow \min\left(1, \frac{1}{\sqrt{\lambda}\,\|\mathbf{w}^{(t+1)}\|}\right) \mathbf{w}^{(t+1)}$ ▷ Project onto $\|\mathbf{w}\| \leq 1/\sqrt{\lambda}$
8: $b^{(t+1)} \leftarrow b^{(t)} - \eta_t \frac{\partial \mathcal{J}_{\mathrm{GSU}}}{\partial b}$ ▷ Gradient step on b
9: **end for**
10: **return** $\mathbf{w}^{(T+1)}$, $b^{(T+1)}$

13 Learning in Linear Subspaces

When the dimensionality d is very large, but most of the uncertainty (variance) of each Gaussian lies in a low-dimensional subspace, we can speed up computation by projecting into that subspace. For each training example i, we perform eigendecomposition of $\mathbf{\Sigma}_i$ and keep the top d_i eigenvectors preserving a fraction p of the total variance, giving a projection matrix $\mathbf{P}_i \in \mathbb{R}^{d_i \times d}$. The projected quantities $\mathbf{z}_i = \mathbf{P}_i \boldsymbol{\mu}_i$ and $\mathbf{\Sigma}_i^z = \mathbf{P}_i \mathbf{\Sigma}_i \mathbf{P}_i^\intercal$ (which is diagonal) yield the same loss form as eq. (30) in the lower-dimensional space.

14 To Sample or Not to Sample?

One might wonder: why not just draw many samples from each Gaussian and train a standard SVM? A concentration inequality by Cirel'son et al. (2006) shows:

$$P\left(\left|\mathcal{L}_{\mathrm{GSU}i} - \tilde{\mathcal{L}}_{\mathrm{GSU}_{i,N}}\right| \geq r\right) \leq 2\exp\left(-\frac{r^2}{2\|\mathbf{w}\|^2}\right), \tag{57}$$

where $\tilde{\mathcal{L}}_{\mathrm{GSU}_{i,N}}$ is the sample-based approximation using N samples. The bound on the right-hand side depends on $\|\mathbf{w}\|^2$. Although the Pegasos algorithm constrains $\|\mathbf{w}\|$ to a ball of radius $1/\sqrt{\lambda}$, the fundamental difficulty with sampling in high-dimensional spaces is that Gaussian distributions spread their probability mass more thinly as the dimensionality grows. Consequently, far more samples are needed to adequately represent the distribution in higher dimensions. In experiments, approximately 10^3 samples per Gaussian were needed in 2D and 5×10^4 in 3D to match SVM-GSU's hyperplane. In high-dimensional problems (d in the hundreds or thousands), sampling becomes infeasible, making the closed-form approach essential. Section 30 makes this dimensional scaling quantitative on purpose-built d-dimensional Gaussian-mixture datasets, measuring how the sample budget a standard SVM needs in order to match SVM-GMU grows with d.

Part IV

Extension: SVM with Gaussian Mixture Uncertainty (SVM-GMU)

Roadmap for Part IV

We will proceed in four steps, mirroring the structure of Part III:

1. **Motivate** why Gaussian uncertainty is not always enough (section 15).
2. **Write down** the objective function with a GMM density inside the integral (section 18).
3. **Swap** the sum and integral (linearity of integration), recognizing each term as an SVM-GSU loss we already know how to evaluate.
4. **Substitute** the SVM-GSU closed form to get the fully explicit SVM-GMU objective, then **differentiate** it.

As before, we will display the evolving objective at each stage.

15 Motivation: Why Go Beyond Gaussian Uncertainty?

The SVM-GSU framework assumes that the uncertainty of each training example is described by a single multivariate Gaussian. While this is flexible (it allows anisotropic, example-specific uncertainty), Gaussian distributions have a fundamental limitation: their iso-density contours are always *ellipsoids*. This means:

- The uncertainty is always *unimodal* (one peak).
- The shape is always *ellipsoidal*. Complex shapes like banana-shaped, crescent-shaped, or multi-lobed uncertainty regions cannot be captured.
- The distribution is always *symmetric* around its mean. Skewed uncertainty is not representable.

16 Gaussian Mixture Models (GMMs) as Universal Approximators

The solution is to model the uncertainty as a *Gaussian Mixture Model* (GMM). A GMM is a weighted sum of several Gaussian distributions:

$$f_{\mathbf{x}_i}(\mathbf{x}) = \sum_{m=1}^{M_i} \pi_i^{(m)} \mathcal{N}\Big(\mathbf{x};\ \boldsymbol{\mu}_i^{(m)},\ \boldsymbol{\Sigma}_i^{(m)}\Big)\,, \tag{58}$$

where M_i is the number of components, $\pi_i^{(m)} \geq 0$ are mixing weights with $\sum_m \pi_i^{(m)} = 1$, and each component has mean $\boldsymbol{\mu}_i^{(m)} \in \mathbb{R}^d$ and covariance $\boldsymbol{\Sigma}_i^{(m)} \in \mathbb{S}^d_{++}$.

GMMs are *universal density approximators*: given enough components, a GMM can approximate any continuous probability distribution to arbitrary accuracy.

17 The New Training Set for SVM-GMU

The new training set considering Gaussian mixture uncertainty is then:

$$\mathcal{X}'' = \left\{ \left(\left\{ \pi_i^{(m)},\ \boldsymbol{\mu}_i^{(m)},\ \boldsymbol{\Sigma}_i^{(m)} \right\}_{m=1}^{M_i},\ y_i \right) :\ i = 1, \ldots, n \right\}. \tag{59}$$

18 Deriving the SVM-GMU Objective Function

18.1 Stage 1: The Objective with the GMM Integral

Just as in SVM-GSU, we replace the hinge loss with its expected value. The objective has the same structure as eq. (15), but with the GMM density $f_{\mathbf{x}_i}$ from eq. (58):

$$\mathcal{J}_{\text{GMU}}(\mathbf{w}, b) = \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_{i=1}^{n} \underbrace{\int_{\mathbb{R}^d} \max(0,\ 1 - y_i(\mathbf{w}^\intercal \mathbf{x} + b))\, f_{\mathbf{x}_i}(\mathbf{x})\ \mathrm{d}\mathbf{x}}_{\mathcal{L}_{\text{GMU}\,i}(\mathbf{w},b)}\,. \tag{60}$$

SVM-GMU Objective - Stage 1: Integral Form (eq. (60))

$$\mathcal{J}_{\text{GMU}}(\mathbf{w}, b) = \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_{i=1}^{n} \underbrace{\int_{\mathbb{R}^d} \max(0,\ 1 - y_i(\mathbf{w}^\intercal \mathbf{x} + b))\, f_{\mathbf{x}_i}(\mathbf{x})\ \mathrm{d}\mathbf{x}}_{\mathcal{L}_{\text{GMU}\,i}(\mathbf{w},b)}\,.$$

Same structure as SVM-GSU (eq. (15)), but $f_{\mathbf{x}_i}$ is now a GMM, not a single Gaussian. The integral $\mathcal{L}_{GMUi}(\mathbf{w}, b)$ is not yet tractable.

18.2 Stage 2: Substituting the GMM Density and Swapping Sum and Integral

In eq. (60), we now substitute $f_{\mathbf{x}_i}(\mathbf{x})$ with the GMM density from eq. (58):

$$\mathcal{J}_{\text{GMU}}(\mathbf{w}, b) = \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_{i=1}^{n} \underbrace{\int_{\mathbb{R}^d} \max(0,\ 1 - y_i(\mathbf{w}^\intercal \mathbf{x} + b)) \left[\sum_{m=1}^{M_i} \pi_i^{(m)} \mathcal{N}\Big(\mathbf{x};\ \boldsymbol{\mu}_i^{(m)},\ \boldsymbol{\Sigma}_i^{(m)}\Big) \right] \mathrm{d}\mathbf{x}}_{\mathcal{L}_{\text{GMU}\,i}(\mathbf{w},b)}\,. \tag{61}$$

Justification for swapping sum and integral. We use the linearity of the integral. The integrand is a finite sum of non-negative integrable terms (the hinge loss is non-negative, each Gaussian PDF is non-negative, and each $\pi_i^{(m)} \geq 0$), so Tonelli's theorem justifies exchanging the finite sum and the integral:

$$\mathcal{J}_{\mathrm{GMU}}(\mathbf{w}, b) = \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_{i=1}^{n} \underbrace{\sum_{m=1}^{M_i} \pi_i^{(m)} \underbrace{\int_{\mathbb{R}^d} \max(0,\ 1 - y_i(\mathbf{w}^\top \mathbf{x} + b))\, \mathcal{N}\Big(\mathbf{x};\ \boldsymbol{\mu}_i^{(m)}, \boldsymbol{\Sigma}_i^{(m)}\Big)\, \mathrm{d}\mathbf{x}}_{\mathcal{L}_{\mathrm{GSU}_i^{(m)}}(\mathbf{w},b) = \text{SVM-GSU loss for component } m}}_{\mathcal{L}_{\mathrm{GMU}i}(\mathbf{w},b)}. \tag{62}$$

Each integral $\mathcal{L}_{\mathrm{GSU}_i^{(m)}}(\mathbf{w}, b)$ is *exactly* the SVM-GSU loss (eq. (30)) for a Gaussian with mean $\boldsymbol{\mu}_i^{(m)}$ and covariance $\boldsymbol{\Sigma}_i^{(m)}$. We already know its closed form.

SVM-GMU Objective - Stage 2: Mixture Form (eq. (62))

$$\mathcal{J}_{\mathrm{GMU}}(\mathbf{w}, b) = \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_{i=1}^{n} \underbrace{\sum_{m=1}^{M_i} \pi_i^{(m)} \underbrace{\int_{\mathbb{R}^d} \max(0,\ 1 - y_i(\mathbf{w}^\top \mathbf{x} + b))\, \mathcal{N}\Big(\mathbf{x};\ \boldsymbol{\mu}_i^{(m)}, \boldsymbol{\Sigma}_i^{(m)}\Big)\, \mathrm{d}\mathbf{x}}_{\mathcal{L}_{\mathrm{GSU}_i^{(m)}}(\mathbf{w},b) = \text{SVM-GSU loss for component } m}}_{\mathcal{L}_{\mathrm{GMU}i}(\mathbf{w},b)},$$

Each $\mathcal{L}_{GSU_i^{(m)}}(\mathbf{w}, b)$ is still an integral, but one we already evaluated in closed form in section 9.

18.3 Stage 3: The Fully Explicit SVM-GMU Objective

Applying the closed-form loss eq. (30) to each component, with:

$$d_{\boldsymbol{\mu}_i}^{(m)} = 1 - y_i \left(\mathbf{w}^\top \boldsymbol{\mu}_i^{(m)} + b \right), \tag{63}$$

$$d_{\boldsymbol{\Sigma}_i}^{(m)} = \sqrt{2\, \mathbf{w}^\top \boldsymbol{\Sigma}_i^{(m)} \mathbf{w}}, \tag{64}$$

we get the component loss:

$$\mathcal{L}_{\mathrm{GSU}_i^{(m)}}(\mathbf{w}, b) = \frac{d_{\boldsymbol{\mu}_i}^{(m)}}{2} \left[\mathrm{erf}\left(\frac{d_{\boldsymbol{\mu}_i}^{(m)}}{d_{\boldsymbol{\Sigma}_i}^{(m)}} \right) + 1 \right] + \frac{d_{\boldsymbol{\Sigma}_i}^{(m)}}{2\sqrt{\pi}} \exp\left(-\frac{\left(d_{\boldsymbol{\mu}_i}^{(m)}\right)^2}{\left(d_{\boldsymbol{\Sigma}_i}^{(m)}\right)^2} \right). \tag{65}$$

The total loss for the i-th example is the weighted sum:

$$\mathcal{L}_{\mathrm{GMU}i}(\mathbf{w}, b) = \sum_{m=1}^{M_i} \pi_i^{(m)}\, \mathcal{L}_{\mathrm{GSU}_i^{(m)}}(\mathbf{w}, b). \tag{66}$$

And the **full SVM-GMU objective** is:

$$\mathcal{J}_{\mathrm{GMU}}(\mathbf{w}, b) = \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_{i=1}^{n} \underbrace{\sum_{m=1}^{M_i} \pi_i^{(m)} \underbrace{\left[\frac{d_{\boldsymbol{\mu}_i}^{(m)}}{2} \left(\mathrm{erf}\left(\frac{d_{\boldsymbol{\mu}_i}^{(m)}}{d_{\boldsymbol{\Sigma}_i}^{(m)}} \right) + 1 \right) + \frac{d_{\boldsymbol{\Sigma}_i}^{(m)}}{2\sqrt{\pi}} \exp\left(-\frac{(d_{\boldsymbol{\mu}_i}^{(m)})^2}{(d_{\boldsymbol{\Sigma}_i}^{(m)})^2} \right) \right]}_{\mathcal{L}_{\mathrm{GSU}_i^{(m)}}(\mathbf{w},b)}}_{\mathcal{L}_{\mathrm{GMU}i}(\mathbf{w},b)}. \tag{67}$$

SVM-GMU Objective - Stage 3: Fully Explicit Closed Form (eq. (67))

$$\mathcal{J}_{\text{GMU}}(\mathbf{w}, b) = \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_{i=1}^{n} \underbrace{\sum_{m=1}^{M_i} \pi_i^{(m)} \underbrace{\left[\frac{d_{\boldsymbol{\mu}_i}^{(m)}}{2}\left(\operatorname{erf}\left(\frac{d_{\boldsymbol{\mu}_i}^{(m)}}{d_{\boldsymbol{\Sigma}_i}^{(m)}}\right)+1\right) + \frac{d_{\boldsymbol{\Sigma}_i}^{(m)}}{2\sqrt{\pi}}\exp\left(-\frac{(d_{\boldsymbol{\mu}_i}^{(m)})^2}{(d_{\boldsymbol{\Sigma}_i}^{(m)})^2}\right)\right]}_{\mathcal{L}_{\text{GSU}}{}_i^{(m)}(\mathbf{w},b)}}_{\mathcal{L}_{\text{GMU}\,i}(\mathbf{w},b)}$$

The only unknowns are still $\mathbf{w}$ *and* b*. Compared to SVM-GSU (eq. (31)), the only structural difference is the extra sum over components* m *with weights* $\pi_i^{(m)}$*. No integrals remain. This is what we minimize.*

Remark 18.1 (Sanity check)**.** *Setting* $M_i = 1$ *and* $\pi_i^{(1)} = 1$ *for all* i *collapses the inner sum to a single term, and eq. (67) reduces exactly to the SVM-GSU objective eq. (31). Further setting* $\boldsymbol{\Sigma}_i^{(1)} \to \mathbf{0}$ *recovers the standard SVM. The nesting is: Standard SVM* $\subset$ *SVM-GSU* $\subset$ *SVM-GMU.*

19 Gradients of the SVM-GMU Objective

We differentiate the closed-form objective eq. (67). Since it has the form

$$\frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_i\sum_m \pi_i^{(m)}\mathcal{L}_{\text{GSU}}{}_i^{(m)}(\mathbf{w}, b),$$

and differentiation is linear, each gradient is the regularizer's contribution plus a weighted sum of the per-component gradients. Each per-component gradient is exactly the SVM-GSU gradient from eqs. (51) and (53), applied to the component parameters $\boldsymbol{\mu}_i^{(m)}$ and $\boldsymbol{\Sigma}_i^{(m)}$.

19.1 Derivation of the Gradient with Respect to w

Starting from the gradient of a single SVM-GSU component loss (derived in section 10):

$$\frac{\partial \mathcal{L}_{\text{GSU}}{}_i^{(m)}}{\partial \mathbf{w}} = \frac{\exp\left(-\frac{\left(d_{\boldsymbol{\mu}_i}^{(m)}\right)^2}{\left(d_{\boldsymbol{\Sigma}_i}^{(m)}\right)^2}\right)}{\sqrt{\pi}\, d_{\boldsymbol{\Sigma}_i}^{(m)}}\boldsymbol{\Sigma}_i^{(m)}\mathbf{w} - \frac{y_i}{2}\left[\operatorname{erf}\left(\frac{d_{\boldsymbol{\mu}_i}^{(m)}}{d_{\boldsymbol{\Sigma}_i}^{(m)}}\right)+1\right]\boldsymbol{\mu}_i^{(m)}. \tag{68}$$

By linearity of differentiation applied to eq. (67):

$$\frac{\partial \mathcal{J}_{\text{GMU}}}{\partial \mathbf{w}} = \lambda\mathbf{w} + \frac{1}{n}\sum_{i=1}^{n}\sum_{m=1}^{M_i}\pi_i^{(m)}\frac{\partial \mathcal{L}_{\text{GSU}}{}_i^{(m)}}{\partial \mathbf{w}}. \tag{69}$$

Substituting:

$$\boxed{\frac{\partial \mathcal{J}_{\text{GMU}}}{\partial \mathbf{w}} = \lambda\mathbf{w} + \frac{1}{n}\sum_{i=1}^{n}\sum_{m=1}^{M_i}\pi_i^{(m)}\left[\frac{\exp\left(-\frac{\left(d_{\boldsymbol{\mu}_i}^{(m)}\right)^2}{\left(d_{\boldsymbol{\Sigma}_i}^{(m)}\right)^2}\right)}{\sqrt{\pi}\, d_{\boldsymbol{\Sigma}_i}^{(m)}}\boldsymbol{\Sigma}_i^{(m)}\mathbf{w} - \frac{y_i}{2}\left(\operatorname{erf}\left(\frac{d_{\boldsymbol{\mu}_i}^{(m)}}{d_{\boldsymbol{\Sigma}_i}^{(m)}}\right)+1\right)\boldsymbol{\mu}_i^{(m)}\right].} \tag{70}$$

19.2 Derivation of the Gradient with Respect to b

Similarly, the per-component bias gradient is

$$\frac{\partial \mathcal{L}_{\mathrm{GSU}}{}_i^{(m)}}{\partial b} = -\frac{y_i}{2}\left[\mathrm{erf}\left(\frac{d_{\boldsymbol{\mu}_i}^{(m)}}{d_{\boldsymbol{\Sigma}_i}^{(m)}}\right) + 1\right]. \tag{71}$$

Summing:

$$\boxed{\frac{\partial \mathcal{J}_{\mathrm{GMU}}}{\partial b} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{m=1}^{M_i}\pi_i^{(m)}\,\frac{y_i}{2}\left[\mathrm{erf}\left(\frac{d_{\boldsymbol{\mu}_i}^{(m)}}{d_{\boldsymbol{\Sigma}_i}^{(m)}}\right) + 1\right].} \tag{72}$$

Remark 19.1 (Sanity check)**.** *Compare eq. (70) with the SVM-GSU gradient eq. (51). The structure is identical; each term inside the brackets has the same form, but now there is an additional weighted sum over components m. If $M_i = 1$ and $\pi_i^{(1)} = 1$, the extra sum collapses to a single term and we recover the SVM-GSU gradient exactly.*

20 Convexity of the SVM-GMU Objective

Proposition 20.1. *$\mathcal{J}_{GMU}(\mathbf{w}, b)$ is strongly convex with respect to $(\mathbf{w}, b)$.*

Proof. By theorem 11.1 and the reasoning in its proof, each per-component loss $\mathcal{L}_{\mathrm{GSU}}{}_i^{(m)}(\mathbf{w}, b)$ is convex in $(\mathbf{w}, b)$. Since $\pi_i^{(m)} \geq 0$, the mixture loss

$$\mathcal{L}_{\mathrm{GMU}i}(\mathbf{w}, b) = \sum_{m=1}^{M_i}\pi_i^{(m)}\,\mathcal{L}_{\mathrm{GSU}}{}_i^{(m)}(\mathbf{w}, b) \tag{73}$$

is a non-negative weighted sum of convex functions, hence convex. The average over training examples is therefore also convex. Adding the strongly convex regularizer $\frac{\lambda}{2}\|\mathbf{w}\|^2$ yields a strongly convex objective with a unique global minimum. □

21 The SGD Algorithm for SVM-GMU

The Pegasos-style SGD extends directly from algorithm 1. The only structural change is that gradient computation now sums over M_i components per example using eqs. (70) and (72). Per-example cost is $\mathcal{O}(M_i \cdot d)$ with diagonal covariances, compared to $\mathcal{O}(d)$ for SVM-GSU. The full algorithm is given in algorithm 2.

Algorithm 2 Stochastic sub-gradient descent for SVM-GMU.

Require: Training set $\mathcal{X}''$ of GMM-annotated examples, regularization parameter $\lambda > 0$, number of iterations $T \in \mathbb{N}$, mini-batch size $k \in \mathbb{N}$.
Ensure: Learned parameters $\mathbf{w}^{(T+1)}$, $b^{(T+1)}$.

Initialize $b^{(1)} \leftarrow 0$ and $\mathbf{w}^{(1)}$ such that $\|\mathbf{w}^{(1)}\| \leq 1/\sqrt{\lambda}$.
for $t = 1, 2, \ldots, T$ **do**
  Sample a random mini-batch $\mathcal{X}_t \subseteq \mathcal{X}''$ with $|\mathcal{X}_t| = k$.
  Set learning rate $\eta_t \leftarrow 1/\lambda t$.
  Initialize gradient accumulators $\mathbf{g}_{\mathbf{w}} \leftarrow \mathbf{0}$, $g_b \leftarrow 0$.
  for each example i in $\mathcal{X}_t$ **do**
    for each component $m = 1, \ldots, M_i$ **do**
      Compute $d_{\boldsymbol{\mu}_i}^{(m)}$ and $d_{\boldsymbol{\Sigma}_i}^{(m)}$ using eqs. (63) and (64).
      Accumulate $\mathbf{g}_{\mathbf{w}} \leftarrow \mathbf{g}_{\mathbf{w}} + \pi_i^{(m)} \cdot$ (per-component contribution from eq. (70)).
      Accumulate $g_b \leftarrow g_b + \pi_i^{(m)} \cdot$ (per-component contribution from eq. (72)).
    end for
  end for
  Add regularizer gradient: $\mathbf{g}_{\mathbf{w}} \leftarrow \mathbf{g}_{\mathbf{w}} + \lambda \mathbf{w}^{(t)}$.
  $\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \eta_t \, \mathbf{g}_{\mathbf{w}}$ ▷ Gradient step on $\mathbf{w}$
  $\mathbf{w}^{(t+1)} \leftarrow \min\left(1, \frac{1}{\sqrt{\lambda}\,\|\mathbf{w}^{(t+1)}\|}\right) \mathbf{w}^{(t+1)}$ ▷ Project onto $\|\mathbf{w}\| \leq 1/\sqrt{\lambda}$
  $b^{(t+1)} \leftarrow b^{(t)} - \eta_t \, g_b$ ▷ Gradient step on b
end for
return $\mathbf{w}^{(T+1)}$, $b^{(T+1)}$

22 Limiting and Special Cases

- $M_i = 1$ for all i: recovers SVM-GSU.
- $M_i = 1$ and $\boldsymbol{\Sigma}_i \to \mathbf{0}$: recovers standard SVM.
- Shared means ($\boldsymbol{\mu}_i^{(m)} = \boldsymbol{\mu}_i$ for all m): models heavier-than-Gaussian tails via a scale mixture.
- Isotropic components ($\boldsymbol{\Sigma}_i^{(m)} = (\sigma_i^{(m)})^2 \mathbf{I}$): $d_{\boldsymbol{\Sigma}_i}^{(m)} = \sqrt{2}\,\sigma_i^{(m)} \|\mathbf{w}\|$.

23 Extension to Linear Subspace Learning

Two strategies:

1. per-component subspaces via eigendecomposition of each $\boldsymbol{\Sigma}_i^{(m)}$, or
2. a shared subspace per example.

For strategy (2), we can use the weighted mean covariance $\bar{\boldsymbol{\Sigma}}_i = \sum_m \pi_i^{(m)} \boldsymbol{\Sigma}_i^{(m)}$, or the full mixture covariance given by the *law of total covariance*:

$$\mathrm{Cov}(\mathbf{x}_i) = \bar{\boldsymbol{\Sigma}}_i + \sum_{m=1}^{M_i} \pi_i^{(m)} \left(\boldsymbol{\mu}_i^{(m)} - \bar{\boldsymbol{\mu}}_i\right)\left(\boldsymbol{\mu}_i^{(m)} - \bar{\boldsymbol{\mu}}_i\right)^\intercal, \qquad \bar{\boldsymbol{\mu}}_i = \sum_m \pi_i^{(m)} \boldsymbol{\mu}_i^{(m)}. \tag{74}$$

The first term captures the average within-component variance; the second captures the variance of the means. This is the correct covariance of a random vector drawn from the GMM.

24 Summary Table

Table 1: Comparison of the standard SVM, SVM-GSU, and the proposed SVM-GMU.

Property	**Standard SVM**	**SVM-GSU**	**SVM-GMU**
Uncertainty model	None (point)	Gaussian	Gaussian mixture
Uncertainty shape	—	Ellipsoidal	Arbitrary
Multimodal	—	No	Yes
Components per example	—	1	$M_i \geq 1$
Closed-form loss	Yes	Yes	Yes
Closed-form gradients	Yes	Yes	Yes
Convex objective	Yes	Yes	Yes
SGD applicable	Yes	Yes	Yes
Per-example cost (diagonal $\mathbf{\Sigma}$)	$\mathcal{O}(d)$	$\mathcal{O}(d)$	$\mathcal{O}(M_i \cdot d)$
Degenerates to standard SVM	N/A	Yes ($\mathbf{\Sigma}_i \to \mathbf{0}$)	Yes ($M_i = 1$, $\mathbf{\Sigma}_i \to \mathbf{0}$)

Part V

Experiments and Empirical Results

This part presents six experiments carried out with the author's Python implementation of SVM-GMU. The first (section 25) introduces a small two-dimensional dataset with deliberately non-Gaussian per-sample uncertainty and shows how the learned SVM-GMU boundary differs from the standard SVM boundary. The second (section 27) empirically verifies the claim of section 14: a standard SVM trained on samples drawn from the per-sample GMMs converges, as the number of samples grows, to the SVM-GMU boundary obtained in closed form on the original data. The third (section 28) isolates the contribution of the mixture structure itself by comparing SVM-GMU against SVM-GSU fitted on a moment-matched single-Gaussian approximation of the same GMMs. The fourth (section 29) takes the realistic next step: rather than assuming the per-sample GMMs are given, it draws samples from each true GMM, re-estimates a GMM per sample by Expectation Maximization with the component count chosen by BIC, and shows that the SVM-GMU boundary fit on the estimated mixtures converges to the closed-form reference as the sample size grows. The fifth (section 30) returns to the scaling argument of section 14 and makes it quantitative: it embeds the dataset in progressively higher dimensions and measures how the number of samples a standard SVM needs in order to match the closed-form SVM-GMU boundary grows with the dimensionality. The sixth (section 31) leaves synthetic data behind and tests the central claim on real handwritten-digit images: it constructs genuinely non-Gaussian (two-lobed) per-image uncertainty by data augmentation and shows that SVM-GMU classifies the images more accurately than SVM-GSU, than an isotropic-uncertainty SVM, and than a standard SVM, with the advantage growing as the uncertainty becomes more pronounced.

Reporting protocol. The experiments that rely on random sampling or stochastic optimization (sections 27 and 29 to 31) are each repeated over R independent random seeds, all spawned from a single master seed so that the entire sweep is reproducible. For these we report every metric as its *median* over the R seeds, accompanied by an inter-quartile band spanning the 25th to the 75th percentile (so the band holds the middle half of the seed-to-seed values, and a narrow band signals a result that barely changes from seed to seed), rather than the value of a single run. By contrast, sections 26 and 28 fit only the closed-form SVM-GMU and SVM-GSU objectives. These objectives are strongly convex (theorem 11.1), so the full-batch Pegasos optimizer converges to the *unique* global minimizer regardless of its random initialization; the learned boundaries are therefore seed-independent, as we confirm numerically in the corresponding sections.

The first four experiments share a common dataset, which we describe in detail in section 25 and then reuse throughout; the high-dimensional study of section 30 instead uses its own purpose-built family of d-dimensional Gaussian-mixture datasets, since its question requires uncertainty that genuinely occupies $\mathbb{R}^d$.

25 The `close_separable` Dataset

25.1 Overview

We construct a dataset with $n = 6$ training examples in $d = 2$ dimensions. Three examples belong to class $+1$ and three to class -1. Each example's uncertainty is modeled as a Gaussian mixture with $M_i \in \{5, 6\}$ components, chosen so that the iso-density contours of the mixtures trace out *banana*- and *crescent*-shaped curves. The two class regions are placed close enough to each other that the orientation of the uncertainty has a visible effect on the learned decision boundary, while still being linearly separable at the level of the observed points themselves. This makes it a useful sandbox for studying how uncertainty-aware training differs from the standard SVM.

25.2 Observed Feature Vectors and Class Labels

The six observed feature vectors (the overall means, in the notation of section 17) are stored as the rows of the matrix $\mathbf{X} \in \mathbb{R}^{6\times 2}$, and the corresponding class labels form the vector $\mathbf{y} \in \{+1, -1\}^6$:

$$\mathbf{X} = \begin{bmatrix} -1.8 & 0.0 \\ -2.0 & 1.5 \\ -1.0 & 2.5 \\ 1.5 & -1.0 \\ 1.0 & 1.0 \\ 2.0 & 2.0 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} +1 \\ +1 \\ +1 \\ -1 \\ -1 \\ -1 \end{bmatrix}. \tag{75}$$

The first three rows of $\mathbf{X}$ form the positive class, the last three form the negative class.

25.3 Per-Sample GMM Uncertainties

Each observed point is accompanied by a GMM with between 5 and 6 components. The i-th training example is therefore a tuple

$$\left(\left\{\pi_i^{(m)},\, \boldsymbol{\mu}_i^{(m)},\, \boldsymbol{\Sigma}_i^{(m)}\right\}_{m=1}^{M_i},\ y_i\right), \tag{76}$$

matching the SVM-GMU training-set definition in eq. (59). The component means are placed along curved arcs, and each component covariance is elongated roughly along the local tangent of the arc so that the mixture of ellipses traces out a banana or a crescent rather than a single fat ellipse.

We now list the six GMMs explicitly.

Sample $i = 1$ $(y_1 = +1)$. Five components with mixing weights and means

$$\boldsymbol{\pi}_1 = \begin{bmatrix} 0.10 \\ 0.20 \\ 0.40 \\ 0.20 \\ 0.10 \end{bmatrix}, \qquad \left\{\boldsymbol{\mu}_1^{(m)}\right\}_{m=1}^{5} = \left\{ \begin{bmatrix} -4.2 \\ -0.3 \end{bmatrix}, \begin{bmatrix} -3.8 \\ 0.3 \end{bmatrix}, \begin{bmatrix} -3.0 \\ 0.5 \end{bmatrix}, \begin{bmatrix} -2.2 \\ 0.3 \end{bmatrix}, \begin{bmatrix} -1.8 \\ -0.3 \end{bmatrix} \right\}, \tag{77}$$

and full covariance matrices

$$\begin{aligned} \boldsymbol{\Sigma}_1^{(1)} &= \begin{bmatrix} 0.08 & -0.05 \\ -0.05 & 0.15 \end{bmatrix}, & \boldsymbol{\Sigma}_1^{(2)} &= \begin{bmatrix} 0.10 & -0.03 \\ -0.03 & 0.12 \end{bmatrix}, & \boldsymbol{\Sigma}_1^{(3)} &= \begin{bmatrix} 0.18 & 0.00 \\ 0.00 & 0.06 \end{bmatrix}, \\ \boldsymbol{\Sigma}_1^{(4)} &= \begin{bmatrix} 0.10 & 0.03 \\ 0.03 & 0.12 \end{bmatrix}, & \boldsymbol{\Sigma}_1^{(5)} &= \begin{bmatrix} 0.08 & 0.05 \\ 0.05 & 0.15 \end{bmatrix}. \end{aligned} \tag{78}$$

Sample $i = 2$ $(y_2 = +1)$. Six components with mixing weights and means

$$\boldsymbol{\pi}_2 = \begin{bmatrix} 0.08 \\ 0.15 \\ 0.22 \\ 0.30 \\ 0.15 \\ 0.10 \end{bmatrix}, \qquad \left\{\boldsymbol{\mu}_2^{(m)}\right\}_{m=1}^{6} = \left\{ \begin{bmatrix} -2.2 \\ 0.6 \end{bmatrix}, \begin{bmatrix} -1.8 \\ 1.1 \end{bmatrix}, \begin{bmatrix} -1.4 \\ 1.5 \end{bmatrix}, \begin{bmatrix} -0.8 \\ 1.7 \end{bmatrix}, \begin{bmatrix} -0.4 \\ 1.5 \end{bmatrix}, \begin{bmatrix} -0.1 \\ 1.1 \end{bmatrix} \right\}, \tag{79}$$

and full covariance matrices

$$\begin{aligned} \boldsymbol{\Sigma}_2^{(1)} &= \begin{bmatrix} 0.07 & 0.04 \\ 0.04 & 0.13 \end{bmatrix}, & \boldsymbol{\Sigma}_2^{(2)} &= \begin{bmatrix} 0.09 & 0.03 \\ 0.03 & 0.10 \end{bmatrix}, & \boldsymbol{\Sigma}_2^{(3)} &= \begin{bmatrix} 0.14 & 0.02 \\ 0.02 & 0.07 \end{bmatrix}, \\ \boldsymbol{\Sigma}_2^{(4)} &= \begin{bmatrix} 0.18 & 0.00 \\ 0.00 & 0.05 \end{bmatrix}, & \boldsymbol{\Sigma}_2^{(5)} &= \begin{bmatrix} 0.12 & -0.02 \\ -0.02 & 0.07 \end{bmatrix}, & \boldsymbol{\Sigma}_2^{(6)} &= \begin{bmatrix} 0.08 & -0.04 \\ -0.04 & 0.12 \end{bmatrix}. \end{aligned} \tag{80}$$

Sample $i = 3$ $(y_3 = +1)$. Five components with mixing weights and means

$$\boldsymbol{\pi}_3 = \begin{bmatrix} 0.10 \\ 0.20 \\ 0.40 \\ 0.20 \\ 0.10 \end{bmatrix}, \qquad \left\{\boldsymbol{\mu}_3^{(m)}\right\}_{m=1}^{5} = \left\{ \begin{bmatrix} -0.6 \\ 2.2 \end{bmatrix}, \begin{bmatrix} -0.2 \\ 2.7 \end{bmatrix}, \begin{bmatrix} 0.3 \\ 3.0 \end{bmatrix}, \begin{bmatrix} 0.8 \\ 3.4 \end{bmatrix}, \begin{bmatrix} 1.1 \\ 3.8 \end{bmatrix} \right\}, \tag{81}$$

and full covariance matrices

$$\begin{aligned} \boldsymbol{\Sigma}_3^{(1)} &= \begin{bmatrix} 0.14 & -0.06 \\ -0.06 & 0.08 \end{bmatrix}, & \boldsymbol{\Sigma}_3^{(2)} &= \begin{bmatrix} 0.12 & -0.07 \\ -0.07 & 0.10 \end{bmatrix}, & \boldsymbol{\Sigma}_3^{(3)} &= \begin{bmatrix} 0.10 & -0.05 \\ -0.05 & 0.14 \end{bmatrix}, \\ \boldsymbol{\Sigma}_3^{(4)} &= \begin{bmatrix} 0.08 & 0.03 \\ 0.03 & 0.16 \end{bmatrix}, & \boldsymbol{\Sigma}_3^{(5)} &= \begin{bmatrix} 0.10 & 0.06 \\ 0.06 & 0.15 \end{bmatrix}. \end{aligned} \tag{82}$$

Sample $i = 4$ ($y_4 = -1$). Six components with mixing weights and means

$$\boldsymbol{\pi}_4 = \begin{bmatrix} 0.08 \\ 0.17 \\ 0.25 \\ 0.25 \\ 0.17 \\ 0.08 \end{bmatrix}, \qquad \left\{\boldsymbol{\mu}_4^{(m)}\right\}_{m=1}^{6} = \left\{ \begin{bmatrix} -0.9 \\ -0.3 \end{bmatrix}, \begin{bmatrix} -0.5 \\ -0.7 \end{bmatrix}, \begin{bmatrix} -0.1 \\ -1.0 \end{bmatrix}, \begin{bmatrix} 0.4 \\ -1.2 \end{bmatrix}, \begin{bmatrix} 0.8 \\ -1.1 \end{bmatrix}, \begin{bmatrix} 1.1 \\ -0.8 \end{bmatrix} \right\}, \tag{83}$$

and full covariance matrices

$$\begin{aligned} \boldsymbol{\Sigma}_4^{(1)} &= \begin{bmatrix} 0.07 & 0.04 \\ 0.04 & 0.12 \end{bmatrix}, & \boldsymbol{\Sigma}_4^{(2)} &= \begin{bmatrix} 0.09 & 0.03 \\ 0.03 & 0.10 \end{bmatrix}, & \boldsymbol{\Sigma}_4^{(3)} &= \begin{bmatrix} 0.15 & 0.00 \\ 0.00 & 0.07 \end{bmatrix}, \\ \boldsymbol{\Sigma}_4^{(4)} &= \begin{bmatrix} 0.15 & 0.00 \\ 0.00 & 0.07 \end{bmatrix}, & \boldsymbol{\Sigma}_4^{(5)} &= \begin{bmatrix} 0.09 & -0.03 \\ -0.03 & 0.10 \end{bmatrix}, & \boldsymbol{\Sigma}_4^{(6)} &= \begin{bmatrix} 0.12 & -0.05 \\ -0.05 & 0.09 \end{bmatrix}. \end{aligned} \tag{84}$$

Sample $i = 5$ ($y_5 = -1$). Five components with mixing weights and means

$$\boldsymbol{\pi}_5 = \begin{bmatrix} 0.10 \\ 0.20 \\ 0.40 \\ 0.20 \\ 0.10 \end{bmatrix}, \qquad \left\{\boldsymbol{\mu}_5^{(m)}\right\}_{m=1}^{5} = \left\{ \begin{bmatrix} 1.3 \\ 0.7 \end{bmatrix}, \begin{bmatrix} 1.7 \\ 0.3 \end{bmatrix}, \begin{bmatrix} 2.3 \\ 0.1 \end{bmatrix}, \begin{bmatrix} 2.9 \\ 0.3 \end{bmatrix}, \begin{bmatrix} 3.3 \\ 0.7 \end{bmatrix} \right\}, \tag{85}$$

and full covariance matrices

$$\begin{aligned} \boldsymbol{\Sigma}_5^{(1)} &= \begin{bmatrix} 0.09 & 0.05 \\ 0.05 & 0.14 \end{bmatrix}, & \boldsymbol{\Sigma}_5^{(2)} &= \begin{bmatrix} 0.12 & 0.03 \\ 0.03 & 0.10 \end{bmatrix}, & \boldsymbol{\Sigma}_5^{(3)} &= \begin{bmatrix} 0.16 & 0.00 \\ 0.00 & 0.07 \end{bmatrix}, \\ \boldsymbol{\Sigma}_5^{(4)} &= \begin{bmatrix} 0.12 & -0.03 \\ -0.03 & 0.10 \end{bmatrix}, & \boldsymbol{\Sigma}_5^{(5)} &= \begin{bmatrix} 0.09 & -0.05 \\ -0.05 & 0.14 \end{bmatrix}. \end{aligned} \tag{86}$$

Sample $i = 6$ ($y_6 = -1$). Six components with mixing weights and means

$$\boldsymbol{\pi}_6 = \begin{bmatrix} 0.08 \\ 0.17 \\ 0.25 \\ 0.25 \\ 0.17 \\ 0.08 \end{bmatrix}, \qquad \left\{\boldsymbol{\mu}_6^{(m)}\right\}_{m=1}^{6} = \left\{ \begin{bmatrix} 2.4 \\ 2.3 \end{bmatrix}, \begin{bmatrix} 2.8 \\ 2.0 \end{bmatrix}, \begin{bmatrix} 3.3 \\ 2.0 \end{bmatrix}, \begin{bmatrix} 3.8 \\ 2.1 \end{bmatrix}, \begin{bmatrix} 4.2 \\ 2.5 \end{bmatrix}, \begin{bmatrix} 4.4 \\ 3.0 \end{bmatrix} \right\}, \tag{87}$$

and full covariance matrices

$$\begin{aligned} \boldsymbol{\Sigma}_6^{(1)} &= \begin{bmatrix} 0.08 & 0.05 \\ 0.05 & 0.11 \end{bmatrix}, & \boldsymbol{\Sigma}_6^{(2)} &= \begin{bmatrix} 0.12 & 0.03 \\ 0.03 & 0.08 \end{bmatrix}, & \boldsymbol{\Sigma}_6^{(3)} &= \begin{bmatrix} 0.15 & 0.00 \\ 0.00 & 0.06 \end{bmatrix}, \\ \boldsymbol{\Sigma}_6^{(4)} &= \begin{bmatrix} 0.13 & -0.02 \\ -0.02 & 0.07 \end{bmatrix}, & \boldsymbol{\Sigma}_6^{(5)} &= \begin{bmatrix} 0.09 & -0.04 \\ -0.04 & 0.10 \end{bmatrix}, & \boldsymbol{\Sigma}_6^{(6)} &= \begin{bmatrix} 0.07 & -0.05 \\ -0.05 & 0.12 \end{bmatrix}. \end{aligned} \tag{88}$$

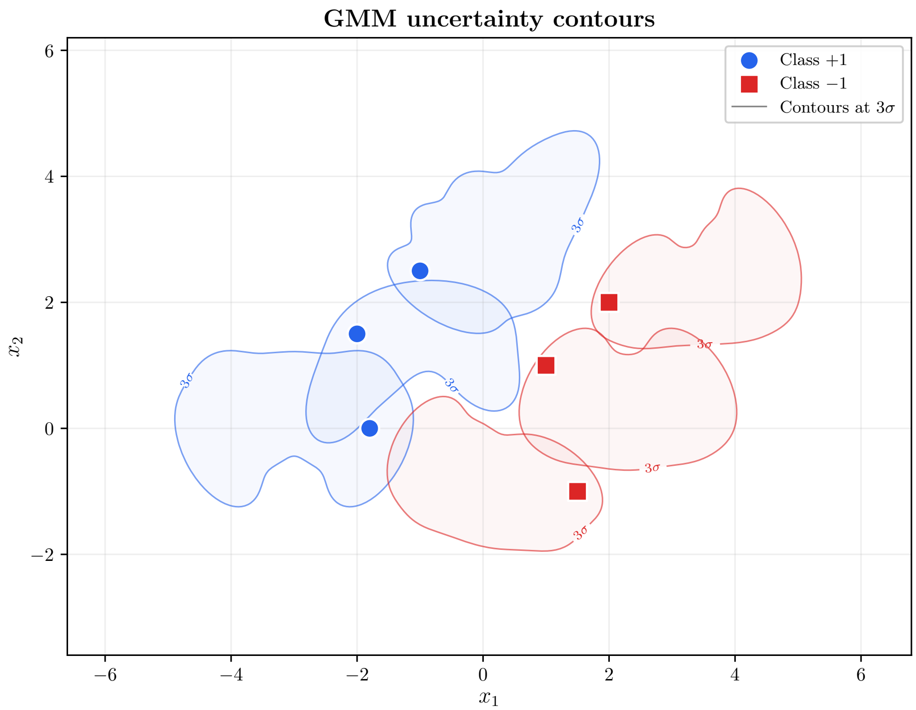

Figure 1: The six per-sample GMM uncertainty distributions of the `close_separable` dataset, visualized as 3σ iso-density contours. Blue contours correspond to class $+1$, red contours to class -1. Blue dots and red boxes mark the observed feature vectors (the rows of $\mathbf{X}$ in eq. (75)). The banana and crescent shapes cannot be represented by single Gaussians.

25.4 Visualizing the GMM Uncertainty

Before fitting any classifier, we can get a sense of the dataset by plotting the GMM density of each sample. Figure 1 shows the 3σ iso-density contours of each of the six mixtures, colored by class (blue for $+1$, red for -1). The markings (blue dots and red boxes) mark the observed feature vectors (the rows of $\mathbf{X}$). The banana and crescent shapes are clearly visible and, crucially, extend toward the decision region between the two classes, which is what makes them interesting from a classification point of view: a Gaussian could not faithfully represent where probability mass actually lies.

What a 3σ iso-density contour means. An *iso-density contour* is a curve along which the probability density $f_{\mathbf{x}_i}(\mathbf{x})$ takes a single constant value, exactly like a contour line on a topographic map joins points of equal altitude. Stepping inward across these curves, the density rises toward the peaks of the mixture; stepping outward, it falls toward zero. The label "3σ" selects one particular contour: the one drawn so that the region it encloses holds about 99.7% of the probability mass. The name borrows the one-dimensional rule of thumb that a Gaussian places roughly 99.7% of its mass within three standard deviations (3σ) of its mean. So in plain terms the 3σ contour of a sample outlines "where that sample almost certainly is"; the banana and crescent curves in fig. 1 are exactly these almost-certain regions, and they lean toward the gap between the classes.

26 Experiment 1: Fitting SVM-GMU on the `close_separable` Dataset

26.1 Training Setup

We train SVM-GMU on the dataset of section 25 using the Pegasos-style SGD described in algorithm 2. The hyperparameters are:

- Regularization parameter $\lambda = 10^{-2}$.
- Number of iterations $T = 1000$.
- Mini-batch size $k = 6$, equal to n (i.e., full-batch SGD, since n is small).
- Random seed 42 for reproducibility.

For comparison, we also train a standard linear SVM on the same observed points by invoking the same estimator with the uncertainty description omitted. As shown in section 22, this reduces the objective exactly to the standard hinge-loss SVM objective eq. (9). The standard SVM is trained with $T = 5000$ iterations (the standard hinge loss has flat regions on the correct side of the margin, so convergence of SGD can take slightly longer).

26.2 Learned Parameters and Convergence

The SGD trajectory of SVM-GMU converges rapidly: the objective value (the regularized loss $\mathcal{J}_{\text{GMU}}(\mathbf{w}, b)$ of eq. (67) that training is driving down, so that "flat" means SGD has essentially found the minimum) drops from roughly 0.0508 at iteration 50 to 0.0472 at iteration 250 and stays essentially flat afterward (the value at iteration 1000 is 0.047228). The learned parameters are

$$\mathbf{w}_{\text{GMU}} = \begin{bmatrix} -1.2527 \\ 1.2589 \end{bmatrix}, \qquad b_{\text{GMU}} = -0.9470. \tag{89}$$

The standard SVM converges to a different objective-value plateau (around 0.00366, reflecting the much smaller regularized hinge loss attained by a classifier that is free to ignore the heavy tails of the GMMs), with learned parameters

$$\mathbf{w}_{\text{SVM}} = \begin{bmatrix} -0.8106 \\ 0.2575 \end{bmatrix}, \qquad b_{\text{SVM}} = -0.4547. \tag{90}$$

Because the SVM-GMU objective is strongly convex (theorem 11.1), the boundary eq. (89) is its *unique* global minimizer and does not depend on the random seed. Refitting over 30 independent seeds changes the orientation of $\mathbf{w}_{\text{GMU}}$ by at most 4×10^{-5} degrees, a seed-to-seed boundary drift of essentially zero. The parameters reported here are therefore representative of the method, not an artifact of a single lucky draw. Both models classify all six observed points correctly. Their decision scores $\mathbf{w}^\intercal \mathbf{x}_i + b$ on the training points are listed in table 2. Recall that the decision score is the signed number whose *sign* is the predicted class (positive for $+1$, negative for -1) and whose *magnitude* measures how far the point sits from the boundary along the direction of $\mathbf{w}$, a large positive score means "confidently class $+1$". The sign matches the class label in every row, but the magnitudes differ substantially because SVM-GMU is responding to the *entire* GMM density, not just the mean.

Table 2: Decision scores $\mathbf{w}^\intercal \mathbf{x}_i + b$ of SVM-GMU and the standard SVM on the six observed points of the `close_separable` dataset. Both classifiers predict the correct class (sign) on every point, but their magnitudes differ because SVM-GMU is shaped by the full GMM densities.

i	y_i	SVM-GMU score	Standard SVM score
1	$+1$	1.3078	1.0044
2	$+1$	3.4466	1.5528
3	$+1$	3.4528	0.9997
4	-1	-4.0849	-1.9281
5	-1	-0.9408	-1.0078
6	-1	-0.9346	-1.5609

26.3 Visualizing the SVM-GMU Boundary

Figure 2 overlays the learned SVM-GMU decision boundary $\mathbf{w}_{\text{GMU}}^\intercal \mathbf{x} + b_{\text{GMU}} = 0$ (solid black) and its margin lines $\mathbf{w}_{\text{GMU}}^\intercal \mathbf{x} + b_{\text{GMU}} = \pm 1$ (dashed) on top of the GMM contours of fig. 1. The classifier has rotated its normal to be roughly perpendicular to the arc connecting the two class clouds, which is exactly what we expect given how the uncertainty is oriented in this region.

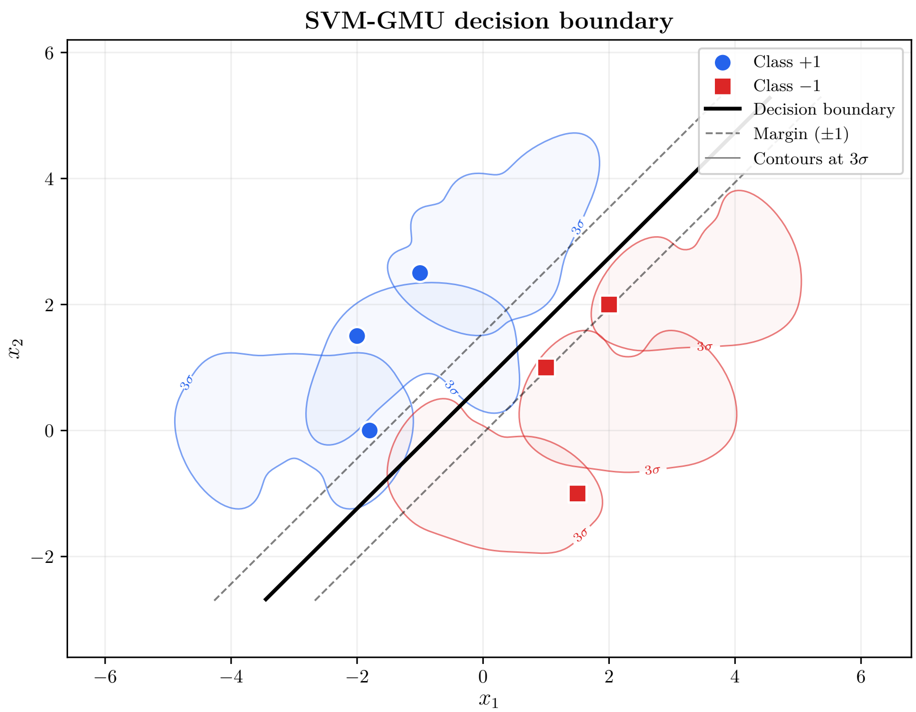

Figure 2: SVM-GMU decision boundary (solid black line) and margin lines (dashed) overlaid on the 3σ GMM contours of the `close_separable` dataset. Learned parameters: $\mathbf{w}_{\text{GMU}} = \begin{bmatrix} -1.2527 & 1.2589 \end{bmatrix}^\intercal$, $b_{\text{GMU}} = -0.9470$.

26.4 Comparison with the Standard SVM

Figure 3 shows both decision boundaries on the same plot: SVM-GMU in solid black and the standard SVM in dashed orange. Even though the two classifiers agree on the *labels* of all six observed points, their boundaries are visibly different in orientation and position. The standard SVM responds only to the six observed $\mathbf{x}_i$'s and picks the maximum-margin line that separates those six points. The SVM-GMU boundary, in contrast, is shaped by the full GMM densities: where a GMM extends heavily toward the opposite class (as several of the banana/crescent arcs do), the SVM-GMU pushes the boundary away from the *mass* of that extension, not from the single point that nominally "is" the sample. Concretely, SVM-GMU rotates its normal roughly 27.5° away from the standard SVM normal and places the boundary farther to the right, responding to the rightward-leaning arcs of the positive class and the downward/leftward-leaning arcs of the negative class. This is precisely what an uncertainty-aware classifier should do; the comparison makes the effect visible rather than abstract.

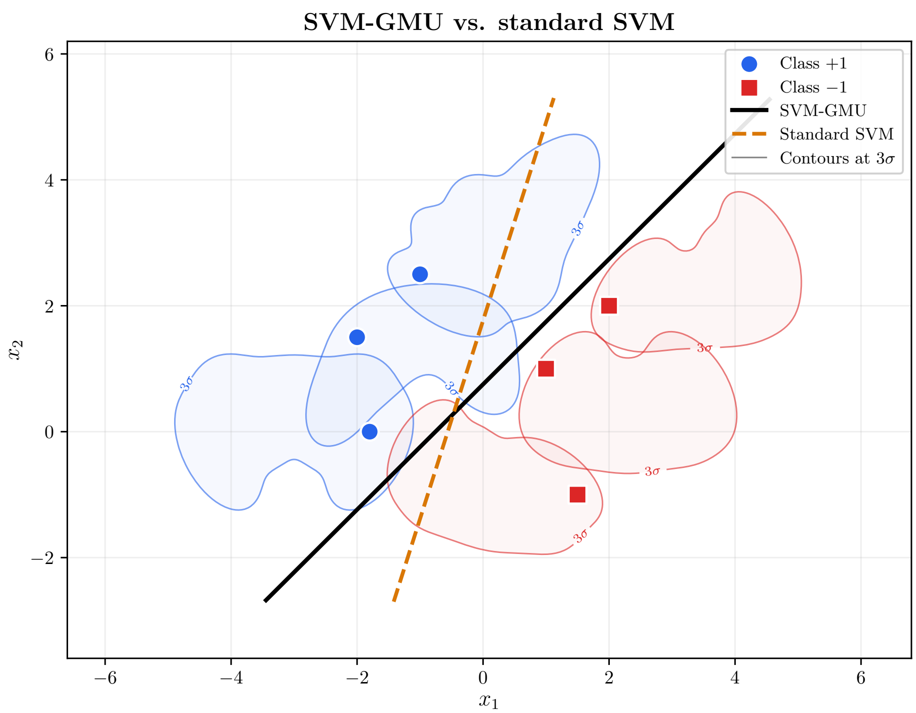

Figure 3: Side-by-side comparison of the SVM-GMU boundary (solid black) and the standard linear SVM boundary (dashed orange) on the `close_separable` dataset, overlaid on the 3σ GMM contours. Both classifiers are trained with the same $\lambda = 10^{-2}$ and random seed. The learned parameters are given in eqs. (89) and (90).

27 Experiment 2: Monte Carlo Convergence of Standard SVM to SVM-GMU

27.1 Motivation

Recall that the SVM-GMU objective eq. (60) is a regularized expected hinge loss,

$$\mathcal{J}_{\mathrm{GMU}}(\mathbf{w}, b) = \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_{i=1}^{n} \underbrace{\int_{\mathbb{R}^d} \max(0,\ 1 - y_i(\mathbf{w}^\intercal \mathbf{x} + b))\, f_{\mathbf{x}_i}(\mathbf{x})\ \mathrm{d}\mathbf{x}}_{\mathcal{L}_{\mathrm{GMU}\,i}(\mathbf{w}, b)},$$

which can be written as

$$\mathcal{J}_{\mathrm{GMU}}(\mathbf{w}, b) = \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_{i=1}^{n} \mathbb{E}_{\mathbf{x}\sim f_{\mathbf{x}_i}}[\max(0,\ 1 - y_i\,(\mathbf{w}^\intercal \mathbf{x} + b))]\,, \tag{91}$$

where each expectation is taken over the per-sample GMM density $f_{\mathbf{x}_i}$. A natural question (and a natural sanity check) is: if we replaced each expectation by a Monte Carlo average over N IID draws from $f_{\mathbf{x}_i}$, how close would the resulting *standard* SVM come to the closed-form SVM-GMU? By the strong law of large numbers,

$$\frac{1}{N}\sum_{k=1}^{N} \max\Big(0,\ 1 - y_i\,\Big(\mathbf{w}^\intercal \mathbf{x}_i^{(k)} + b\Big)\Big) \xrightarrow[N\to\infty]{\text{a.s.}} \mathbb{E}_{\mathbf{x}\sim f_{\mathbf{x}_i}}[\max(0,\ 1 - y_i\,(\mathbf{w}^\intercal \mathbf{x} + b))]\,, \tag{92}$$

so the empirical hinge loss of a standard SVM trained on the $nN = 6N$ sampled points (each labeled by its parent's y_i) converges to eq. (91) pointwise in $(\mathbf{w}, b)$. Under the convexity of the objective established in section 20, the *minimizers* converge as well. This experiment verifies that prediction empirically on the `close_separable` dataset.

What "Monte Carlo", "IID", and the law of large numbers mean here. Equation eq. (92) is the engine behind this whole experiment, so it is worth unpacking in plain language. The expectation $\mathbb{E}_{\mathbf{x}\sim f_{\mathbf{x}_i}}[\cdot]$ on the right is the *true* average hinge loss taken over the entire uncertainty cloud of example i, exactly the quantity SVM-GMU computes in closed form. We cannot average over an infinite cloud by hand, so we *estimate* that average: we draw a finite batch of N sample points from the cloud and take their ordinary average (the left-hand sum). Estimating an average by drawing random samples and averaging them is called a *Monte Carlo* estimate, named after the casino, because it relies on random draws. Those draws are *IID*, independent and identically distributed: each of the N points is generated the same way and independently of the others, like rolling the same die over and over. The *law of large numbers* then guarantees that this sample average closes in on the true expectation as $N \to \infty$; the annotation "a.s." (almost surely) is the precise way of saying "with probability 1", that is, guaranteed except for freak events of total probability zero. Because this holds for every fixed boundary $(\mathbf{w}, b)$ (the phrase "pointwise in $(\mathbf{w}, b)$"), and because both objectives are convex with a single minimum (section 20), the *location* of the minimum converges too: the boundary the standard SVM settles on from the samples drifts toward the boundary SVM-GMU finds in closed form. The rest of this section measures that drift.

27.2 Experimental Procedure

The reference SVM-GMU is trained as in section 26, yielding the parameters in eq. (89) (denoted $\mathbf{w}_{\mathrm{ref}}$, b_{ref} below). For each value of $N \in \{1, 5, 25, 100, 500, 2000, 5000, 20000\}$ and each of $R = 30$ independent random seeds, we repeat the following procedure.

1. For each of the $n = 6$ observed points, draw N IID samples from $f_{\mathbf{x}_i}$ by picking a component index $m \in \{1, \ldots, M_i\}$ according to $\boldsymbol{\pi}_i$ and then drawing a point from $\mathcal{N}\left(\boldsymbol{\mu}_i^{(m)}, \boldsymbol{\Sigma}_i^{(m)}\right)$.

2. Label each sample by its parent's y_i.

3. Stack the $6N$ labeled samples into a dataset and train a *standard* linear SVM on it (i.e., invoke the estimator without any `sample_uncertainty`, which reduces to the usual hinge-loss objective). The regularization parameter is $\lambda = 10^{-2}$, as for the reference SVM-GMU; the variation across the R seeds enters through the random draws in step 1.

4. Record the learned $(\mathbf{w}_N, b_N)$ and then compute three boundary-disagreement metrics against $(\mathbf{w}_{\text{ref}}, b_{\text{ref}})$.

For each N we then summarize the R seeds by the median of each metric, together with the 25th–75th percentile inter-quartile band that the figures display.

27.3 Boundary-Disagreement Metrics

To quantify how close a learned boundary is to the reference SVM-GMU boundary, we use three scalar metrics.

Why we first normalize by $\|\mathbf{w}\|$. A subtlety underlies all three metrics: the same boundary line can be written with many different $(\mathbf{w}, b)$ pairs. Multiplying both $\mathbf{w}$ and b by any positive constant leaves the line $\mathbf{w}^\intercal\mathbf{x} + b = 0$ unchanged (for example $2\mathbf{w}^\intercal\mathbf{x} + 2b = 0$ is the very same line), so the raw entries of $\mathbf{w}$ and b are not directly comparable between two classifiers; one may simply have happened to use a longer $\mathbf{w}$. To strip out this arbitrary scale we divide through by the length $\|\mathbf{w}\|$, rescaling the normal vector to length one. After this normalization the decision function $\widehat{\mathbf{w}}^\intercal\mathbf{x} + b/\|\mathbf{w}\|$ equals the genuine *signed distance* from $\mathbf{x}$ to the boundary (positive on one side, negative on the other), a purely geometric quantity on which two classifiers can be compared fairly. The three metrics are:

- **Angle between normals:**

$$\theta_N = \arccos(\widehat{\mathbf{w}}_N^\intercal \widehat{\mathbf{w}}_{\text{ref}}), \qquad \widehat{\mathbf{w}} = \frac{\mathbf{w}}{\|\mathbf{w}\|}, \tag{93}$$

  reported in degrees. This measures how much the two boundary *orientations* disagree. The dot product of two unit vectors is exactly the cosine of the angle between them, so arccos turns it back into that angle: $\theta_N = 0^\circ$ means the two normals point the same way and the boundaries are perfectly parallel, while a larger angle means their orientations have diverged.

- **Offset difference:**

$$\Delta_N^{\text{offset}} = \left| \frac{b_N}{\|\mathbf{w}_N\|} - \frac{b_{\text{ref}}}{\|\mathbf{w}_{\text{ref}}\|} \right|. \tag{94}$$

  Since $-\frac{b}{\|\mathbf{w}\|}$ is the signed distance from the origin to the hyperplane, this is the displacement of one boundary relative to the other along their shared normal direction.

- **Grid RMS:** the root-mean-square difference between the two *unit-normalized* decision functions, evaluated on a dense grid. Writing

$$\hat{f}(\mathbf{x}) = \widehat{\mathbf{w}}^\intercal\mathbf{x} + \frac{b}{\|\mathbf{w}\|}, \qquad \widehat{\mathbf{w}} = \frac{\mathbf{w}}{\|\mathbf{w}\|}, \tag{95}$$

for the unit-normalized decision function of a boundary $(\mathbf{w}, b)$, so that $\hat{f}_N$ is this function built from the learned boundary $(\mathbf{w}_N, b_N)$ and $\hat{f}_{\text{ref}}$ is the same function built from the reference SVM-GMU boundary $(\mathbf{w}_{\text{ref}}, b_{\text{ref}})$, and letting $\{\mathbf{x}_g\}_{g=1}^{G}$ be the $G = 60 \times 60 = 3600$ points of a regular grid over the plotting window $[-5, 5] \times [-4, 5]$,

$$\Delta_N^{\text{grid}} = \sqrt{\frac{1}{G} \sum_{g=1}^{G} \left(\hat{f}_N(\mathbf{x}_g) - \hat{f}_{\text{ref}}(\mathbf{x}_g) \right)^2}. \tag{96}$$

In simple terms: because $\hat{f}(\mathbf{x})$ is exactly the signed distance from the point $\mathbf{x}$ to the boundary, this metric goes to every point of the window, asks how differently the two boundaries "see" that point (how far each one says the point is from it), squares those differences, averages them, and takes the square root. It is therefore the typical size of the disagreement between the two boundaries across the whole window, expressed as a single number. Unlike the angle and the offset, which isolate orientation and position separately, the grid RMS folds both effects into one overall summary; it is zero only when the two boundaries coincide everywhere in the window.

If the Monte Carlo argument above is correct, all three metrics should tend to zero as $N \to \infty$.

27.4 Results

Table 3 reports the median of each metric over the $R = 30$ seeds for every value of N in the sweep, alongside the total training-set size $n_{\text{total}} = 6N$ used to fit each standard SVM. The trend is clear: by $N = 20000$ (i.e., $n_{\text{total}} = 120000$) the median angle is within 0.12°, the median offset difference is below 2×10^{-3}, and the median grid RMS is ≈ 0.007. Averaging over seeds removes the spurious wiggles that any single run would show: the residual run-to-run scatter is now carried by the inter-quartile bands of fig. 4, which contract steadily as N grows, rather than appearing as non-monotonicities in a point estimate.

Table 3: Convergence of the standard-SVM-on-samples boundary to the reference SVM-GMU boundary, as a function of the number of Monte Carlo draws N per observed point. $n_{\text{total}} = 6N$ is the size of the training set fed to the standard SVM. Values are medians over $R = 30$ random seeds; the inter-quartile bands are shown in fig. 4.

N	n_{total}	θ_N (deg)	Δ_N^{offset}	Grid RMS
1	6	6.4025	0.4054	0.5868
5	30	5.8808	0.1591	0.3654
25	150	2.0682	0.0937	0.1499
100	600	0.9299	0.0457	0.0899
500	3000	0.2316	0.0168	0.0292
2000	12000	0.2367	0.0090	0.0171
5000	30000	0.2399	0.0080	0.0161
20000	120000	0.1185	0.0017	0.0074

Figure 4 plots the three metrics against N on a logarithmic horizontal axis. All three curves trend toward zero as N grows, which is exactly the pattern predicted by eq. (92) and the convexity of the objective.

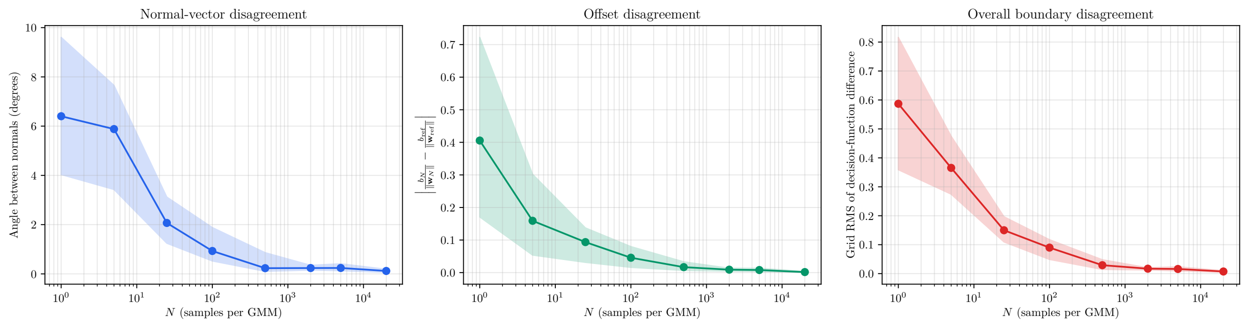

Figure 4: Boundary-disagreement metrics between the standard SVM fit to $6N$ Monte Carlo samples and the reference SVM-GMU, as a function of N. *Left:* angle between the two weight-vector normals (degrees). *Middle:* offset difference $\left|\frac{b_N}{\|\mathbf{w}_N\|} - \frac{b_{\text{ref}}}{\|\mathbf{w}_{\text{ref}}\|}\right|$. *Right:* RMS difference of the unit-normalized decision functions over a dense grid. Solid lines are the median over $R = 30$ seeds and the shaded bands span the 25th to 75th percentile. All three tend to zero as N grows, confirming that the standard-SVM-on-samples minimizer converges to the SVM-GMU minimizer.

Figure 5 shows the visual counterpart of the metrics table: one panel per value of N, each depicting

1. the GMM contours of the original six observed points,
2. the sampled cloud used to fit that panel's standard SVM (all $6N$ points are shown),
3. the reference SVM-GMU boundary as a dashed green line, and
4. the standard SVM boundary as a solid black line.

As N increases from 1 to 20000, the cloud fills out the GMM densities and the standard-SVM boundary pulls toward the SVM-GMU reference.

27.5 Interpretation

The experiment confirms, empirically, what section 14 argues on theoretical grounds: the SVM-GMU objective is nothing more than the $N \to \infty$ limit of a standard hinge-loss objective evaluated on samples drawn from the per-sample GMMs. In the opposite direction, it also highlights the computational value of the closed-form objective. Reaching a median grid-RMS of ≈ 0.007 required $N = 20000$ samples per observed point, i.e., a training set of 120000 points. SVM-GMU attains the same fit directly from the $n = 6$ original examples and their GMM descriptions, with no sampling at all. In higher dimensions the gap between the closed-form and sampling approaches widens sharply (as noted in the concentration bound eq. (57)), so this advantage becomes essential rather than merely convenient.

28 Experiment 3: Does the Mixture Structure Matter?

28.1 Motivation

The SVM-GSU framework of section 9.5 already handles per-sample uncertainty via a single Gaussian per example. A natural question is: how much does the *mixture* structure of SVM-GMU actually buy us, beyond using the best single-Gaussian approximation of each mixture? We answer

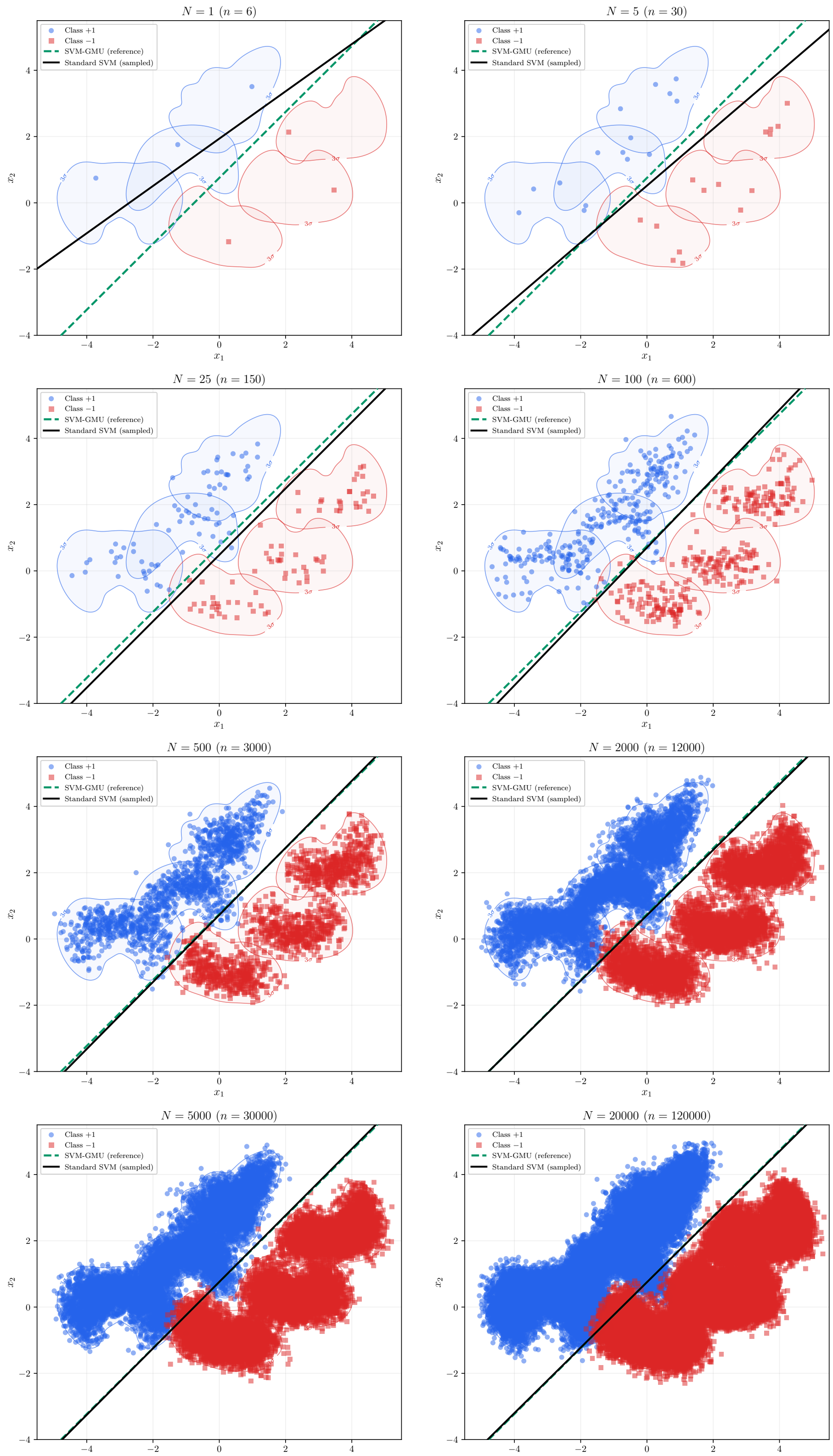

Figure 5: Convergence of the standard-SVM-on-samples boundary to the SVM-GMU boundary on the `close_separable` dataset.

this by comparing SVM-GMU against SVM-GSU on the `close_separable` dataset, where each GMM is replaced by a moment-matched single Gaussian.

28.2 Moment-Matched Single-Gaussian Approximation

For a GMM with weights $\left\{\pi_i^{(m)}\right\}$, component means $\left\{\boldsymbol{\mu}_i^{(m)}\right\}$, and component covariances $\left\{\boldsymbol{\Sigma}_i^{(m)}\right\}$ for $m = 1, \ldots, M_i$, the moment-matched single Gaussian $\mathcal{N}\left(\bar{\boldsymbol{\mu}}_i, \bar{\boldsymbol{\Sigma}}_i\right)$ has the same overall mean and overall covariance as the mixture. The overall mean is the weighted average of component means,

$$\bar{\boldsymbol{\mu}}_i = \sum_{m=1}^{M_i} \pi_i^{(m)}\, \boldsymbol{\mu}_i^{(m)}, \tag{97}$$

and the overall covariance is given by the law of total covariance eq. (74), which we restate here in the notation of this experiment:

$$\bar{\boldsymbol{\Sigma}}_i = \underbrace{\sum_{m=1}^{M_i} \pi_i^{(m)}\, \boldsymbol{\Sigma}_i^{(m)}}_{\text{within-component (average component cov.)}} + \underbrace{\sum_{m=1}^{M_i} \pi_i^{(m)} \left(\boldsymbol{\mu}_i^{(m)} - \bar{\boldsymbol{\mu}}_i\right)\left(\boldsymbol{\mu}_i^{(m)} - \bar{\boldsymbol{\mu}}_i\right)^{\mathsf{T}}}_{\text{between-component (cov. of the means)}}. \tag{98}$$

The between-component term is the crucial one for banana- and crescent-shaped mixtures: it inflates the moment-matched covariance along the direction in which the component means spread out, turning a curved row of small ellipses into a single large, oriented ellipse.

What "matching moments" means. The *moments* of a distribution are its basic summary numbers: the first moment is the mean (where the distribution is centered) and the second moment is the covariance (how far and in which directions it is spread). A moment-matched Gaussian is the single Gaussian with the *same center and the same spread* as the mixture, the closest a lone Gaussian can come to mimicking it. The *law of total covariance* eq. (98) then says this overall spread splits into two intuitive pieces. The *within-component* term averages how spread out the individual components are on their own; the *between-component* term adds how far the component centers sit from the overall center. A classroom analogy makes this concrete: to describe the spread of students' heights across several classrooms, part of it is the variation *inside* each classroom (within) and part is the variation *between* the classroom averages (between), and the total is the sum of the two. For a banana mixture the component centers are strung along an arc, so the between-component term is large in the arc's direction, and that is exactly what stretches the matched Gaussian into a long ellipse.

Table 4 lists the moment-matched mean $\bar{\boldsymbol{\mu}}_i$ and the two *eigenvalues* of $\bar{\boldsymbol{\Sigma}}_i$ for each of the six samples, along with the anisotropy ratio λ_2/λ_1. The two eigenvalues of a 2×2 covariance have a direct geometric reading: they are the variances along the ellipse's two principal axes (its longest and shortest directions), so $\sqrt{\lambda_1}$ and $\sqrt{\lambda_2}$ are the lengths of the short and long semi-axes of the one-standard-deviation ellipse. Their ratio λ_2/λ_1 (larger over smaller) therefore measures how stretched the ellipse is: a value of 1 is a perfect circle, and the larger it grows the more cigar-shaped the ellipse becomes. Here the anisotropy is between $3.29\times$ and $3.99\times$, confirming that each moment-matched ellipse is heavily elongated, as one would expect from a banana/crescent GMM whose means lie along an arc.

Table 4: Moment-matched single-Gaussian approximations of the six GMMs in the `close_separable` dataset, obtained via eqs. (97) and (98). $\lambda_1 \leq \lambda_2$ are the eigenvalues of $\bar{\boldsymbol{\Sigma}}_i$. The anisotropy ratio λ_2/λ_1 is around 3 to 4 for every sample, which reflects the arc shape of the component means.

i	y_i	$\bar{\boldsymbol{\mu}}_i$	$\lambda_1\left(\bar{\boldsymbol{\Sigma}}_i\right)$	$\lambda_2\left(\bar{\boldsymbol{\Sigma}}_i\right)$	λ_2/λ_1
1	$+1$	$\begin{bmatrix}-3.000 & +0.260\end{bmatrix}^\intercal$	0.188	0.672	3.57
2	$+1$	$\begin{bmatrix}-1.064 & +1.388\end{bmatrix}^\intercal$	0.151	0.551	3.66
3	$+1$	$\begin{bmatrix}+0.290 & +3.020\end{bmatrix}^\intercal$	0.149	0.508	3.40
4	-1	$\begin{bmatrix}+0.142 & -0.944\end{bmatrix}^\intercal$	0.122	0.486	3.99
5	-1	$\begin{bmatrix}+2.300 & +0.300\end{bmatrix}^\intercal$	0.144	0.474	3.29
6	-1	$\begin{bmatrix}+3.509 & +2.214\end{bmatrix}^\intercal$	0.136	0.510	3.74

28.3 Training and Learned Parameters

Using the moment-matched Gaussians as a single-component GMM per sample (weight 1, mean $\bar{\boldsymbol{\mu}}_i$, covariance $\bar{\boldsymbol{\Sigma}}_i$), we fit SVM-GSU with the same hyperparameters as SVM-GMU ($\lambda = 10^{-2}$, $T = 1000$, $k = 6$, seed 42). The learned parameters are:

$$\mathbf{w}_{\text{GMU}} = \begin{bmatrix}-1.2527\\ 1.2589\end{bmatrix}, \qquad b_{\text{GMU}} = -0.9470, \tag{99}$$

$$\mathbf{w}_{\text{GSU}} = \begin{bmatrix}-1.1922\\ 1.2721\end{bmatrix}, \qquad b_{\text{GSU}} = -0.9824. \tag{100}$$

Applying the three boundary-disagreement metrics from section 27 (here comparing the GSU boundary to the GMU boundary) gives:

$$\theta = 1.7157^\circ, \qquad \Delta^{\text{offset}} = 0.0303, \qquad \text{Grid RMS} = 0.0861. \tag{101}$$

Both the SVM-GMU and the moment-matched SVM-GSU boundaries are unique minimizers of strongly convex objectives, so these three numbers are seed-independent: repeating the comparison over 30 seeds leaves them unchanged to within 5×10^{-5} (their seed-to-seed standard deviations are all below 10^{-4}). The disagreement is small but not zero. For context: in the Monte Carlo experiment of section 27, the standard-SVM-on-samples boundary reaches a median grid RMS of this magnitude (≈ 0.09) at around $N = 100$ samples per observed point (i.e., $n_{\text{total}} \approx 600$ total training points, see table 3). So going from a full GMM to a single moment-matched Gaussian costs us about as much boundary fidelity as replacing the closed-form objective with a coarse Monte Carlo approximation.

28.4 Visual Comparison

Figure 6 shows the comparison in three panels. Panel (a) repeats the original GMM contours from fig. 1 for reference. Panel (b) shows the moment-matched single-Gaussian contours at the 3σ level. The loss of fidelity is immediate: each banana/crescent has collapsed into a single oriented ellipse that *covers* roughly the same region but assigns non-trivial probability mass to the interior of the original arc, exactly where the GMM placed almost no density. Panel (c) overlays the SVM-GMU boundary (dashed green) and the SVM-GSU boundary (solid purple) on top of the original GMM contours. The two boundaries differ by about 1.72° in orientation and by ≈ 0.03 in offset, with the SVM-GSU boundary pulled slightly toward the side where the moment-matched ellipses "spill over" more aggressively than the true GMM contours do.

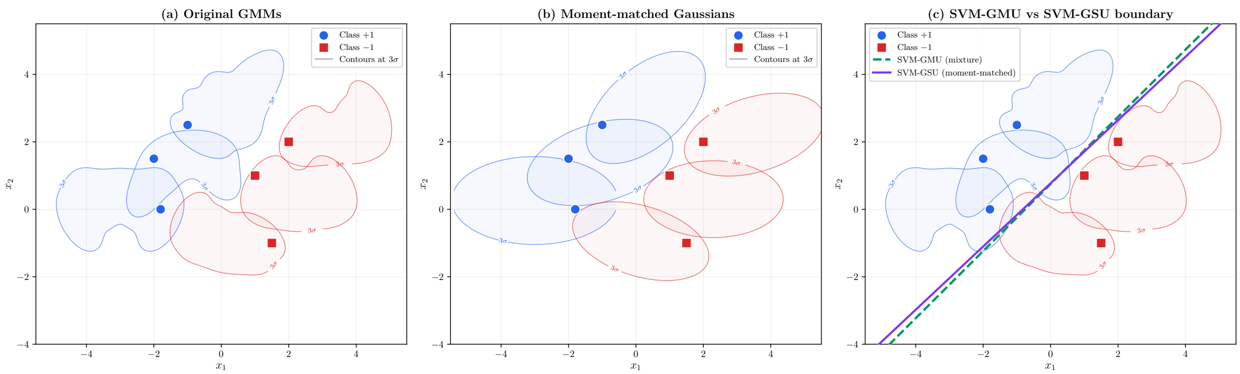

Figure 6: Effect of replacing the per-sample GMMs with their moment-matched single Gaussians. *Panel (a):* original GMM contours (banana and crescent shapes). *Panel (b):* moment-matched single-Gaussian contours at the same 3σ level; each GMM collapses to a tilted ellipse that covers roughly the same region but loses the shape information. *Panel (c):* SVM-GMU boundary (dashed green) vs. SVM-GSU-on-moment-matched boundary (solid purple), overlaid on the original GMM contours. The two boundaries differ by the quantities in eq. (101).

28.5 Interpretation

Two observations stand out.

1. **Moment-matching is a lossy summary even when it preserves location and spread.** Panels (a) and (b) of fig. 6 have identical first and second moments *by construction*, yet they tell completely different stories about where probability mass actually lies. This is a fundamental limitation of any single-Gaussian representation of a genuinely non-elliptical uncertainty: the Gaussian family simply does not contain the shape.

2. **The shape loss propagates to the classifier.** Panel (c) shows that the two classifiers still end up close, but visibly distinct. On this dataset, with banana/crescent GMMs that are only moderately eccentric, the effect is small (grid RMS ≈ 0.086). On datasets where the per-sample uncertainty is more sharply non-Gaussian, or where samples lie closer to the decision boundary, one would expect the gap to grow accordingly.

The practical takeaway is the symmetric one we would want: when the per-sample uncertainty *is* unimodal and roughly elliptical, SVM-GSU and SVM-GMU will agree closely and the simpler GSU model is enough. When the uncertainty is multimodal, curved, or otherwise non-Gaussian, as in the `close_separable` dataset, SVM-GMU earns its keep by modeling the structure directly.

29 Experiment 4: Learning the Uncertainty from Data via Expectation Maximization

29.1 Motivation

The first three experiments assume the per-sample GMMs are handed to us. In a real application they are not: a sensor yields a cloud of noisy measurements, and the mixture that describes each example's uncertainty must be *estimated*. This experiment closes that gap. For each observed point we draw samples from its true GMM, fit a fresh GMM to those samples by Expectation Maximization (EM), and run SVM-GMU on the estimated mixtures. Sweeping the number of samples shows how the learned boundary approaches the closed-form reference of section 26 as more data becomes available. It is the estimation-side counterpart of the Monte Carlo experiment of section 27: there we sampled and fit a standard SVM, here we sample, *re-learn the mixtures*, and fit SVM-GMU.

29.2 Experimental Procedure

The reference SVM-GMU is the one trained in section 26 on the true GMMs, with the parameters in eq. (89). For each value of $N \in \{50, 100, 250, 500, 1000, 2000, 5000, 20000\}$ and each of $R = 30$ independent seeds, we repeat the following.

1. For each of the $n = 6$ observed points, draw N IID samples from its true GMM $f_{\mathbf{x}_i}$ (component picked by $\boldsymbol{\pi}_i$, then a draw from that component's Gaussian).
2. Fit a GMM to those N samples by EM, selecting the number of components $M \in \{1, \ldots, 8\}$ that minimizes the Bayesian Information Criterion (BIC; see section 29.3), with full covariances and several restarts.
3. Label the estimated GMM by its parent's y_i.
4. Fit SVM-GMU on the six estimated GMMs ($\lambda = 10^{-2}$, $T = 1000$, $k = 6$, seed 42) and measure its boundary against the reference using the three metrics of section 27 (angle between normals, offset difference, and grid RMS of the unit-normalized decision functions).

The whole procedure is repeated over $R = 30$ independent seeds, all spawned from one master seed so that the sweep is reproducible. For each N we report the median of each metric with its 25th–75th percentile band; the selected component count M is reported as the per-point *modal* choice, the most frequently selected count, across the seeds (a six-vector, since the six examples are fit independently). Seed-averaging matters more here than anywhere else in this part: a single EM run, with its random restarts and BIC tie-breaks, is noticeably noisier than the pure-sampling experiment of section 27, and a lone run can land on a misleadingly good or bad fit at any given N.

What Expectation Maximization does. Step 2 fits a Gaussian mixture to a cloud of points: it estimates the weights, means, and covariances of the components that best explain where the points fell. *Expectation Maximization* (EM) is the standard algorithm for this. It cannot directly see which component produced each point, so it alternates two steps until the fit stops improving: an *Expectation* step that, given the current components, computes for every point a soft guess of which component it probably came from; and a *Maximization* step that, given those soft guesses, re-estimates each component's weight, mean, and covariance. Each round can only increase the likelihood of the data (the quantity defined just below), so EM climbs steadily toward a good fit. It can, however, climb to a *local* peak that depends on where it started, so in practice one runs it several times from different random starting points (the "restarts" mentioned above) and keeps the best result. The output is an ordinary GMM, ready to be handed to SVM-GMU exactly like the hand-specified mixtures of the earlier experiments.

29.3 Model Selection with the Bayesian Information Criterion

Step 2 of the procedure leaves one choice open: how many Gaussian components should the re-estimated mixture have? Too few and the fit cannot capture the banana or crescent shape; too many and the extra components merely chase sampling noise. We resolve this with the *Bayesian Information Criterion* (BIC), a standard model-selection score.

Definition. For a model fit by maximum likelihood to N data points, the BIC is

$$\mathrm{BIC} = -2 \ln \hat{L} + k \ln N, \tag{102}$$

where $\hat{L}$ is the maximized likelihood of the data under the fitted model (the *likelihood* is how probable the observed points are under the model, so a higher likelihood means the fitted mixture places more of its density where the points actually landed), k is the number of free parameters of the model, and N is the number of data points (here, the size of the sampled cloud for one observed example). The score has two competing terms:

- The *fit term* $-2 \ln \hat{L}$ rewards explaining the data. A better fit means a higher likelihood, which makes this term smaller (more negative). Adding components can only keep the likelihood the same or improve it, so this term decreases as M grows.
- The *complexity penalty* $k \ln N$ punishes using more parameters. It grows with M, since each additional component adds parameters.

Parameter count of a GMM. For a mixture of M Gaussians in d dimensions with full covariances, the free parameters are the $M-1$ independent mixing weights (they sum to one), the Md entries of the means, and the $M\,(d(d+1)/2)$ distinct entries of the symmetric covariance matrices:

$$k = (M-1) + Md + M\,\frac{d(d+1)}{2}. \tag{103}$$

In this experiment $d = 2$, so $k = 6M - 1$: every extra component costs six parameters, and the penalty $k \ln N$ rises accordingly.

Why the smallest BIC is chosen. Lower BIC is better. Selecting the M with the smallest BIC picks the model that best balances fit against complexity. Choosing by likelihood alone would always select the most complex candidate ($M = 8$ here), because more components never hurt the in-sample likelihood; the penalty $k \ln N$ is exactly what prevents that overfitting. Plotted against M, the BIC is typically U-shaped: it falls while added components yield genuine likelihood gains, then rises once further components only add parameters without improving the fit. The minimum is the sweet spot between underfitting and overfitting.

Consistency, and the trend in the M column. The penalty per parameter is $\ln N$, which *grows with the sample size*. This makes BIC consistent: as $N \to \infty$ it recovers the true number of components and resists adding spurious ones. This is exactly the behavior of the M column of table 5. At $N = 50$ the penalty dominates the modest likelihood gains that a few noisy points can demonstrate, so BIC collapses each example to roughly one component, $M = (1, 2, 1, 1, 1, 1)$. At $N = 20000$ the likelihood improvement from the genuine five to six components in each banana or crescent overwhelms the penalty, and BIC recovers counts approaching the truth, $M = (5, 4, 6, 3, 4, 4)$ (the per-point modal selection across the $R = 30$ seeds).

What BIC compares, and what it does not. It is worth being precise about the two distinct comparison steps in this experiment, because they operate at different levels.

1. **BIC (model selection, upstream).** For a single observed example, the N sampled points are fit by eight candidate GMMs, one for each component count $M \in \{1, \ldots, 8\}$ (each with several EM restarts). BIC compares *these eight fitted mixtures* and keeps the one with the smallest score. This is done independently for each of the $n = 6$ observed examples, so per value of N there are $6 \times 8 = 48$ candidate fits and six selections; this is why M is reported as a six-vector rather than a single number.

2. **Boundary metrics (the measured outcome).** The two SVM-GMU boundaries, the true-GMM reference of section 26 and the one fit on the BIC-selected estimated GMMs, are compared by the three boundary-disagreement metrics of section 27 (angle, offset, grid RMS). BIC plays no role in that comparison.

In short, BIC only decides how many Gaussians to use when reconstructing each example's uncertainty; it never sees the classifier. The convergence the experiment measures is then assessed separately by the boundary metrics.

29.4 Results

Table 5 reports the median of the three metrics across the sweep, and figs. 7 and 8 visualize them with their bands. The trend is clear in the orientation and offset of the boundary: the median angle falls to $0.34°$ and the median offset difference to 6×10^{-3} by $N = 20000$, and the median grid RMS ends at 0.018, the same order as the 0.007 reached by the Monte Carlo experiment of section 27 at the same N. The most telling quantity is the number of components BIC selects: at $N = 50$ the modal choice is $M = (1, 2, 1, 1, 1, 1)$, collapsing most examples to a single Gaussian, whereas by $N = 20000$ it is $M = (5, 4, 6, 3, 4, 4)$, approaching the true counts $(5, 6, 5, 6, 5, 6)$. As the data reveals the true mixture structure, the EM-fitted contours sharpen into the banana and crescent shapes and the learned boundary settles onto the reference.

The panels of fig. 8 carry a third boundary for comparison: the *standard* SVM of section 27, fit on the very cloud that the EM step consumed at that N. It converges to the reference as well, so by the end of the sweep all three lines lie almost on top of one another. Placing the two routes side by side at the sample budgets the two sweeps share ($N \in \{100, 500, 2000, 5000, 20000\}$) is instructive, because it shows that per drawn point the sampling route is the more accurate of the two: from $N = 500$ upward it sits closer to the reference on both the angle and the grid RMS, and at $N = 20000$ it reaches a median grid RMS of 0.0074 (table 3) against the 0.0176 of the EM route. Only at $N = 100$ do the two trade places, the EM route edging ahead on grid RMS (0.0880 against 0.0899) while still trailing on angle. This ordering is expected rather than troubling: the EM route pays an estimation error the sampling route never incurs, since it must recover each mixture's weights, means, and covariances from the cloud before the classifier sees anything at all. Section 29.5 takes up what the EM route buys in exchange.

Table 5: Disagreement between the SVM-GMU boundary fit on EM-estimated GMMs and the true-GMM reference, as a function of the number of EM training samples N per observed point ($n_{\text{total}} = 6N$). The three metrics are medians over $R = 30$ seeds; the inter-quartile bands are shown in fig. 7. The column M lists the per-point *modal* BIC-selected component count across the seeds for the six observed points $(M_1, \ldots, M_6)$; the true counts are $(5, 6, 5, 6, 5, 6)$.

N	n_{total}	θ_N (deg)	Δ_N^{offset}	Grid RMS	M (BIC)
50	300	2.0024	0.0520	0.1156	$(1, 2, 1, 1, 1, 1)$
100	600	1.4264	0.0419	0.0880	$(2, 2, 1, 1, 1, 1)$
250	1500	0.8401	0.0251	0.0606	$(3, 2, 1, 1, 1, 2)$
500	3000	1.0427	0.0180	0.0553	$(3, 3, 1, 2, 2, 2)$
1000	6000	0.8527	0.0100	0.0431	$(3, 3, 2, 2, 3, 3)$
2000	12000	0.8306	0.0158	0.0424	$(3, 3, 3, 2, 3, 3)$
5000	30000	0.5570	0.0079	0.0285	$(5, 4, 3, 3, 3, 3)$
20000	120000	0.3413	0.0064	0.0176	$(5, 4, 6, 3, 4, 4)$

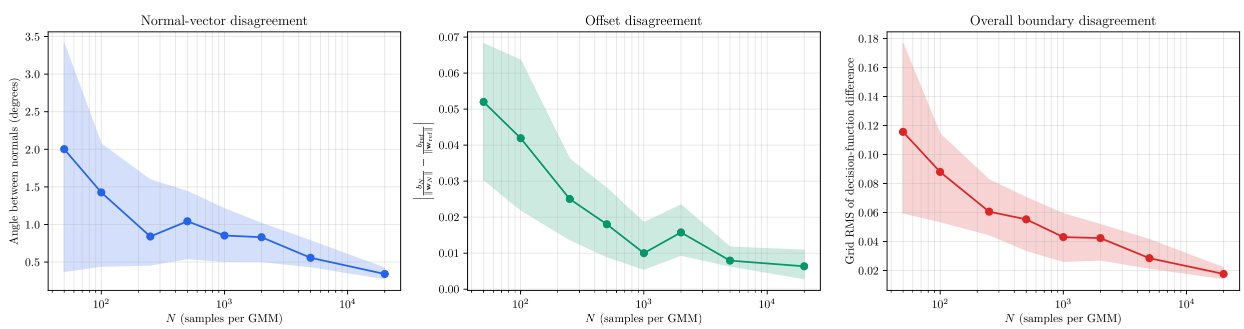

Figure 7: Boundary-disagreement metrics between SVM-GMU fit on EM-estimated GMMs and the true-GMM reference, versus the number of EM samples N per observed point. *Left:* angle between the weight-vector normals. *Middle:* offset difference. *Right:* grid RMS of the unit-normalized decision functions. Solid lines are the median over $R = 30$ seeds and the shaded bands span the 25th to 75th percentile. The orientation and offset converge steadily toward zero; the grid RMS is noisier but ends near 0.018.

29.5 Interpretation

Three points stand out. First, SVM-GMU does not require the uncertainty to be known in closed form: when each mixture is estimated from data by EM with BIC choosing the component count, the resulting classifier converges to the one trained on the true mixtures. This is the bridge from the synthetic dataset to a realistic pipeline: collect repeated noisy measurements, fit a GMM per example, and feed those to SVM-GMU. Second, seed-averaging is what makes the convergence trustworthy here. A single run can land on a deceptively good fit at small N, because with little data BIC underfits each example to a near-single Gaussian, which is close to the moment-matched single-Gaussian summary of section 28 and already yields a boundary close to SVM-GMU; an

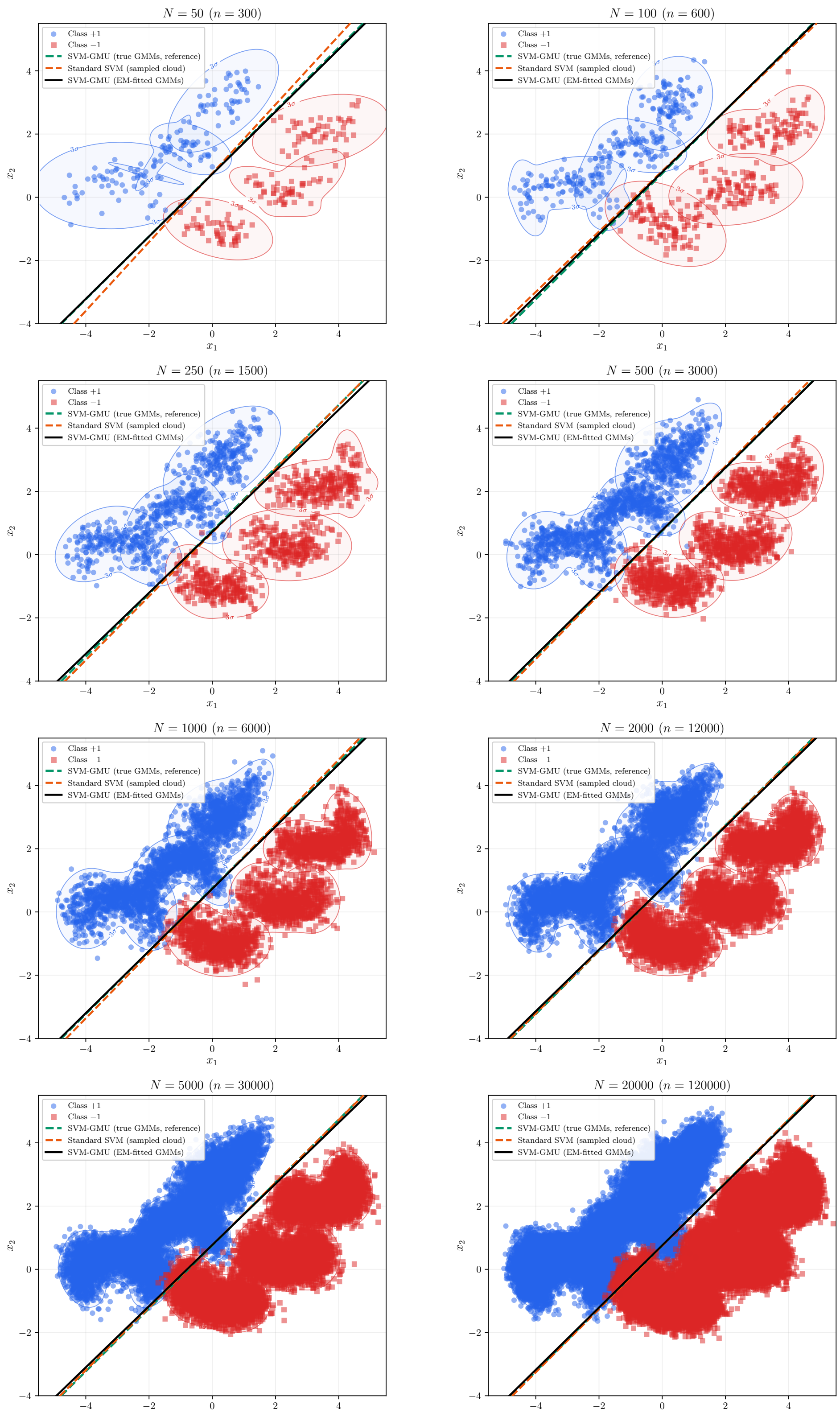

Figure 8: One panel per N: the sampled cloud the EM fits were trained on (all $6N$ points are shown), the EM-fitted GMM contours (at the 3σ level), the reference SVM-GMU boundary (green dashed), the SVM-GMU boundary fit on the estimated GMMs (black solid), and, for comparison, the standard SVM fit on that same sampled cloud (orange dashed). As N grows the cloud fills out the true banana and crescent shapes, the fitted contours recover them, and both the black and the orange boundaries approach the green reference. The orange line is the Monte Carlo route of section 27 and the black line is the estimate-then-solve route of this section; the panels show the two consuming the very same sampled points.

earlier single-run version of this experiment showed exactly such a spurious dip at $N = 50$. The median over $R = 30$ seeds removes that artifact: the disagreement is now largest at $N = 50$, as it should be, and falls roughly monotonically as N grows and the estimated mixtures genuinely recover the multi-component banana and crescent shapes. The residual run-to-run variability is reported honestly as the inter-quartile band rather than hidden inside a single number.

Third, the standard-SVM boundary now drawn alongside the other two in fig. 8 states the trade-off plainly, and it is worth being explicit that the comparison does not flatter the EM route. In the plane, at a fixed number of draws, training directly on the cloud is the better use of those draws; the EM route spends part of its budget reconstructing the mixtures and is correspondingly further from the reference at the same N. What the EM route buys is therefore not accuracy per sample but a *change of representation*. It compresses each N-point cloud back into a handful of Gaussians, after which the closed-form objective eq. (67) is minimized over the six examples at a cost that does not depend on N at all, and the cloud itself can be thrown away. The sampling route has no such option: the $6N$ points *are* its model, they must all be carried into training, and, as section 30 measures, the budget they demand grows by roughly an order of magnitude per doubling of the dimension until it becomes unaffordable, while the closed-form fit is untouched by that growth. Seen in this light, the two lines converging in these panels is exactly the reassurance we want: EM recovers the same boundary that sampling does, and then hands it to the one formulation of the two that survives the move out of the plane.

30 Experiment 5: How Sampling Scales with Dimension

30.1 Motivation

Section 14 argued, via the concentration bound eq. (57), that drawing samples from each per-sample distribution and training a standard SVM becomes progressively less competitive with the closed-form objective as the dimensionality grows: a Gaussian, and hence a Gaussian mixture, spreads its probability mass ever more thinly in higher dimensions, so ever more samples are needed to represent it faithfully. Section 27 confirmed the $N \to \infty$ equivalence in the plane ($d = 2$); this experiment makes the *dimensional* dependence quantitative. For a range of d we build a genuinely d-dimensional dataset and measure $N^*(d)$, the number of samples per example that a standard SVM needs in order to recover the closed-form SVM-GMU boundary to a fixed angular tolerance.

30.2 A Genuinely d-Dimensional Dataset

The previous four experiments share the planar `close_separable` dataset. This one cannot: its question is about *high-dimensional* uncertainty, so the per-sample distributions must genuinely occupy $\mathbb{R}^d$. The non-Gaussian *shapes* that motivate SVM-GMU, the banana and crescent of section 25, are intrinsically low-dimensional, so rather than extruding them artificially we use a purpose-built family of d-dimensional Gaussian-mixture datasets, one per dimension, in which the per-example uncertainty is a genuine mixture that fills all d coordinates.

For each d we construct $n = 10$ examples, five per class. Example i is an equally weighted mixture of $M = 3$ isotropic Gaussian components, written in the same notation as the general GMM of eq. (58),

$$f_{\mathbf{x}_i}(\mathbf{x}) = \sum_{m=1}^{3} \pi_i^{(m)} \mathcal{N}\Big(\mathbf{x};\ \boldsymbol{\mu}_i^{(m)},\ \boldsymbol{\Sigma}_i^{(m)}\Big)\,. \tag{104}$$

The construction merely fixes what each of those three ingredients is. The mixing weights are equal, $\pi_i^{(m)} = \frac{1}{3}$; each component covariance is isotropic, $\mathbf{\Sigma}_i^{(m)} = \sigma^2 \mathbf{I}_d$; and each component mean is the example center pushed a fixed distance in a random direction,

$$\boldsymbol{\mu}_i^{(m)} = \mathbf{c}_i + s\,\mathbf{u}_i^{(m)}, \tag{105}$$

where $\mathbf{c}_i \in \mathbb{R}^d$ is the example center, each $\mathbf{u}_i^{(m)} \in \mathbb{R}^d$ is an independent random unit vector (a direction of length one), and s is the component spread. In words, eq. (105) says: start at the center $\mathbf{c}_i$ and step a distance s along the random direction $\mathbf{u}_i^{(m)}$ to place the m-th component. The two class centers are separated along the first coordinate by $\pm\delta$ and given a small isotropic jitter across all coordinates, whose off-axis part makes the optimal boundary genuinely d-dimensional rather than aligned with a single axis. The classes are placed close enough that their uncertainty overlaps, so that, exactly as in section 25, the boundary depends on the full per-example distribution and not merely on the observed means. We use $\delta = 2$, jitter standard deviation 0.5, $s = 0.9$, and $\sigma = 1.1$.

Reading the construction. A few terms in eqs. (104) and (105) are worth spelling out. A covariance of the form $\sigma^2 \mathbf{I}_d$, a number σ^2 times the $d \times d$ identity matrix, describes *isotropic* uncertainty: the same variance σ^2 in every coordinate and no correlation between coordinates, so each component is a perfectly round, fuzzy ball of radius about σ in $\mathbb{R}^d$ rather than a stretched ellipse. A *unit vector* $\mathbf{u}_i^{(m)}$ is just a direction, an arrow of length one; multiplying it by the spread s places the m-th component's center a distance s from the example center $\mathbf{c}_i$ along that random direction. So each example is three round balls of uncertainty, scattered a short hop apart around $\mathbf{c}_i$ in three random directions of the d-dimensional space and blended in equal thirds. The parameter δ is how far apart the two classes sit (small, so they nearly touch), and the "jitter" is a little extra randomness in the class centers so that the dividing surface tilts through all d axes instead of lining up with a single one.

Figure 9 traces, step by step, how a single example center $\mathbf{c}_i$ is generated in the code, making explicit what is decided first and where the randomness and the class identity enter.

Fix the constants (once for the whole sweep): $\delta = 2$ and the jitter standard deviation 0.5 are the two that build $\mathbf{c}_i$; the other two, $s = 0.9$ and $\sigma = 1.1$, are not used until *after* $\mathbf{c}_i$ exists (they build the component means and covariances).

Choose the class $y_i \in \{+1, -1\}$ this example belongs to (the loop variable `cls` in the code).

Draw a random vector $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$: each of the d coordinates independent, mean 0, standard deviation 1.

Scale by the jitter: $\mathbf{c}_i \leftarrow 0.5\,\mathbf{z}$, so $\mathbf{c}_i \sim \mathcal{N}(\mathbf{0},\, 0.5^2\mathbf{I}_d)$, a small isotropic scatter about the origin in every coordinate.

Shift the first coordinate only: $c_{i,1} \leftarrow c_{i,1} + y_i\,\delta$, which adds $+2$ for class $+1$ and -2 for class -1.

Result: $\mathbf{c}_i \sim \mathcal{N}\big((y_i \cdot 2,\; 0, \ldots, 0),\; 0.5^2\mathbf{I}_d\big)$, centered at ± 2 on the first axis and at 0 on the others. It then feeds the component means $\boldsymbol{\mu}_i^{(m)} = \mathbf{c}_i + s\,\mathbf{u}_i^{(m)}$ and covariances $\boldsymbol{\Sigma}_i^{(m)} = \sigma^2\mathbf{I}_d$.

Figure 9: How the example center $\mathbf{c}_i$ is generated in the high-dimensional construction of section 30.2. The order runs top to bottom: the constants are fixed first, then the class label, then a random Gaussian draw, then a scale by the jitter, and finally the $\pm\delta$ shift on the first coordinate alone. The randomness enters at the draw step; the class identity enters only at the final shift, and only on coordinate 1. Each of the $n = 10$ examples repeats the random steps independently. Of the four constants, only δ and the jitter shape $\mathbf{c}_i$; the spread s and the component width σ act one stage later.

Two features make this the right construction for the question at hand. First, it is genuinely high-dimensional: the uncertainty has variance σ^2 in every one of the d directions, and the optimal weight vector uses all of them. Second, it is a *genuine mixture* ($M = 3 > 1$), so the closed-form reference is literally SVM-GMU, not its single-Gaussian special case. In high dimensions a three-component mixture is only mildly non-Gaussian, but that is deliberate: the role of this experiment is to measure the *dimensional scaling* of the sampling cost, whereas the value of the mixture's *shape* is what sections 26 and 28 demonstrate in the plane, where non-Gaussian structure lives.

30.3 Protocol and a Dimension-Agnostic Metric

The grid RMS of section 27 evaluates the decision functions on a dense grid over the plane, which is infeasible in high dimensions. We replace it with a Monte Carlo decision-function RMS that captures the same quantity but works at any d: we draw a cloud $\{\mathbf{x}_g\}_{g=1}^{G}$ of $G = 10 \times 2000 = 20000$ points from the per-example mixtures themselves and compute

$$\Delta^{\mathrm{mc}} = \sqrt{\frac{1}{G}\sum_{g=1}^{G}\Big(\hat{f}_N(\mathbf{x}_g) - \hat{f}^{d}_{\mathrm{ref}}(\mathbf{x}_g)\Big)^2}, \tag{106}$$

with $\hat{f}$ the unit-normalized decision function defined in eq. (95). The two decision functions being compared are: $\hat{f}_N$, built from the boundary $(\mathbf{w}_N, b_N)$ that a *standard* SVM learns from the $10N$-point sample cloud (the sampling-based approximation under test); and $\hat{f}^d_{\text{ref}}$, built from the closed-form SVM-GMU reference boundary $(\mathbf{w}^d_{\text{ref}}, b^d_{\text{ref}})$ described in the next paragraph (the exact target). The superscript d signals that this reference is refit afresh in each dimension of the sweep. The angle θ and the offset Δ^{offset} are dimension-agnostic already and carry over unchanged. Of these, the angle θ drives the samples-to-match summary $N^*(d)$ defined below, while the decision-function RMS Δ^{mc} and the offset difference Δ^{offset} are reported as functions of d in figs. 12 and 13.

The closed-form reference $(\mathbf{w}^d_{\text{ref}}, b^d_{\text{ref}})$ is obtained by minimizing the SVM-GMU objective eq. (67) on the analytic mixtures; it is exact at every d and never samples. For each dimension $d \in \{2, 3, 5, 10, 20, 35, 50\}$ and each of $R = 30$ seeds (each seed drawing a fresh dataset and fresh samples), we sweep N over a geometric ladder (a sequence whose values grow by a roughly constant multiplicative factor, here about $\times 3$ per step, so a handful of "rungs" span several orders of magnitude) $\{10, 30, 100, 300, 1000, 3000, 10^4, 3 \times 10^4\}$, fit a standard SVM to the $10N$-point cloud at each rung, and record θ against the reference. We define

$$N^*(d) = \min\{N \text{ on the ladder} : \theta \le \tau\}, \tag{107}$$

the smallest number of samples per example at which the standard SVM matches the closed-form boundary to within τ degrees, and report its median over the R seeds for both $\tau = 2^\circ$ and the stricter $\tau = 1^\circ$. The ladder is capped at $N_{\max} = 3 \times 10^4$; when a seed never reaches the tolerance within the cap we record its N^* as having "exceeded the cap".

Why the sampler struggles. The closed-form reference uses the analytic mixtures directly, so it incurs no sampling error at any d. A standard SVM, by contrast, must reconstruct each example's d-dimensional distribution from finitely many draws. As d grows there are more directions in which the empirical cloud can fail to pin the boundary down, and the mixture's mass spreads more thinly, so the number of samples needed to reach a fixed angular tolerance grows steeply. This is exactly the difficulty the concentration bound eq. (57) anticipates.

30.4 Results

Table 6 reports $N^*(d)$ for both tolerances, and fig. 10 plots them on a logarithmic vertical axis. The growth is steep and orderly. At $\tau = 2^\circ$ the median samples-to-match climbs from 30 at $d = 2$ to 10^4 at $d = 20$, roughly an order of magnitude for every doubling of the dimension. By $d = 35$ the median has reached 3×10^4, the last rung of the ladder, with 12 of the 30 seeds already unable to reach the tolerance even there, and at $d = 50$ *every one* of the 30 seeds exceeds the ceiling. At the stricter $\tau = 1^\circ$ the wall arrives far sooner. Already at $d = 10$ half the seeds fail to reach the tolerance within 3×10^4 samples, which puts the median itself past the ceiling, and from $d = 20$ onward not a single seed reaches it. The closed-form SVM-GMU, by contrast, attains the exact boundary at every d from the ten examples and their mixture descriptions, with no sampling at all.

Remark 30.1 (Reading a censored median)**.** *The $\tau = 1^\circ$ entry at $d = 10$ deserves a word, because it is the one place where the summary is delicate. Of the 30 seeds, 15 reach the tolerance (two at $N = 3000$, seven at 10^4, six at 3×10^4) and 15 do not reach it at all within the ladder. Averaging only the seeds that succeeded would report 10^4, but that is the median of the successes, not of the experiment: it silently discards precisely the 15 hardest runs, which are the evidence that sampling is failing. Ranking the censored seeds where they belong, above every seed that finished, puts the* *median past the ceiling, which is what table 6 records. The general point is worth carrying to any table of this kind: with right-censored data, dropping the runs that did not finish reports the best half of the evidence as though it were all of it.*

Table 6: Samples-to-match $N^*(d)$: the smallest number of Monte Carlo samples per example at which a standard SVM matches the closed-form SVM-GMU boundary to within τ degrees, on the genuinely d-dimensional Gaussian-mixture datasets of eq. (104). Values are medians over $R = 30$ seeds. A seed that never reaches the tolerance within the ladder ceiling $N_{\max} = 3 \times 10^4$ is *censored* rather than discarded: its N^* is unknown but certainly larger than every rung, so it is ranked above every seed that did reach the tolerance. The median is therefore taken over all 30 seeds, and the entry "$> 3 \times 10^4$" marks the dimensions where the median itself falls in the censored region, that is, where at least half the seeds never reach the tolerance at all. The inter-quartile bands are shown in fig. 10.

d	$N^*(d)$, $\tau = 2^\circ$	$N^*(d)$, $\tau = 1^\circ$
2	30	300
3	300	2000
5	1000	3000
10	3000	$> 3 \times 10^4$
20	10000	$> 3 \times 10^4$
35	30000	$> 3 \times 10^4$
50	$> 3 \times 10^4$	$> 3 \times 10^4$

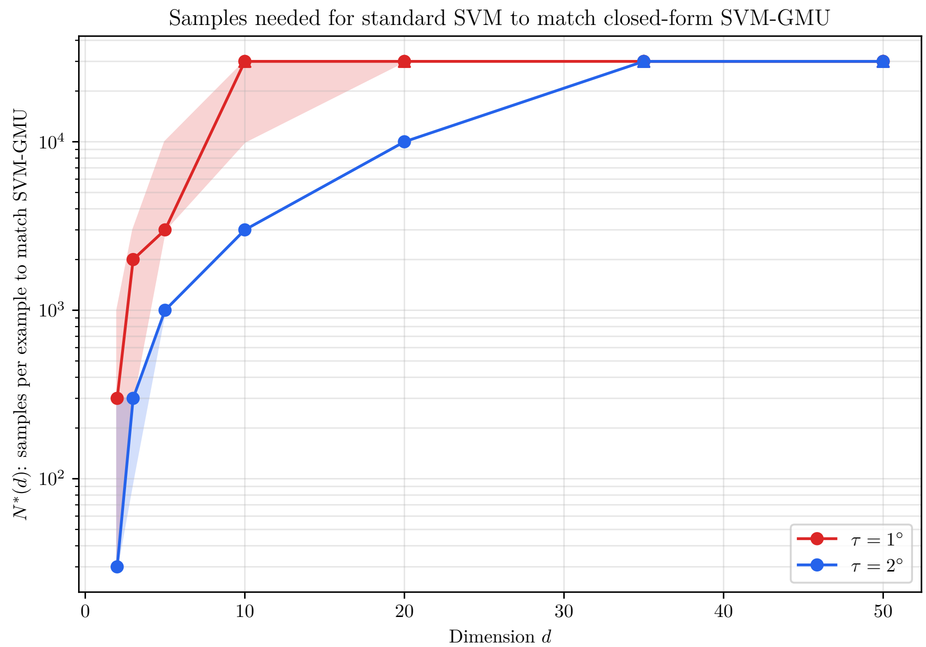

Figure 10: Samples-to-match $N^*(d)$ versus dimension d on a logarithmic vertical axis, for tolerances $\tau = 1°$ and $\tau = 2°$. Solid markers are the median over $R = 30$ seeds and the shaded bands span the 25th to 75th percentile, both computed with censored seeds ranked above every seed that reached the tolerance. Upward triangles at the ceiling mark dimensions where some seeds never reach the tolerance within $N_{\max} = 3 \times 10^4$. Where a median or a band edge is itself censored it is known only to lie *above* the ceiling, so it is drawn *at* the ceiling: the flat stretches at 3×10^4 are therefore lower bounds, not plateaus, and the true curves keep climbing beyond the top of the figure. $N^*(d)$ grows by roughly an order of magnitude per doubling of d, while the closed-form SVM-GMU pays nothing.

What a "sample budget" is, and two ways to read the same trade-off. A *sample budget* is just the number of random draws N we allow ourselves *per example* when feeding the standard SVM; with $n = 10$ examples a budget of N means $10N$ sampled points in total. We call it a budget because every draw has a price (memory to hold it and time to train on it), so in practice one sets a ceiling and lives within it, exactly as one lives within a fixed amount of money. The experiment probes a single trade-off between how much we spend (the budget N) and how hard the problem is (the dimension d), and there are two equally valid ways to read it. Table 6 fixes the *quality* we demand (the angular tolerance τ) and asks how large a budget $N^*(d)$ is needed to reach it, like asking "how much money would it take to furnish this house to a decent standard?". The fixed-budget view does the mirror image: it pins the budget at $N = 1000$ or $N = 10^4$ and watches the quality slip as d grows, like asking "with only a thousand dollars in hand, how well can I furnish the house?". The same picture explains *why* the quality slips: a higher-dimensional distribution spreads its probability mass over ever more directions, just as a bigger house has ever more rooms, so a fixed budget is stretched thinner and each room is furnished more sparsely. One further reason to show both views: when the dimension is so high that even the largest budget on our ladder (3×10^4) still cannot reach the tolerance, the first view can only report "off the top of the ladder", whereas the fixed-budget view still returns an honest number for how far off the boundary is.

Figure 11 carries out this second, fixed-budget reading for $N = 1000$ and $N = 10^4$ samples per example: the median angle to the reference rises from 0.59° at $d = 2$ to 6.93° at $d = 50$ for $N = 1000$, and from 0.27° to 2.91° for $N = 10^4$, whereas the closed-form SVM-GMU sits at exactly 0° for every d.

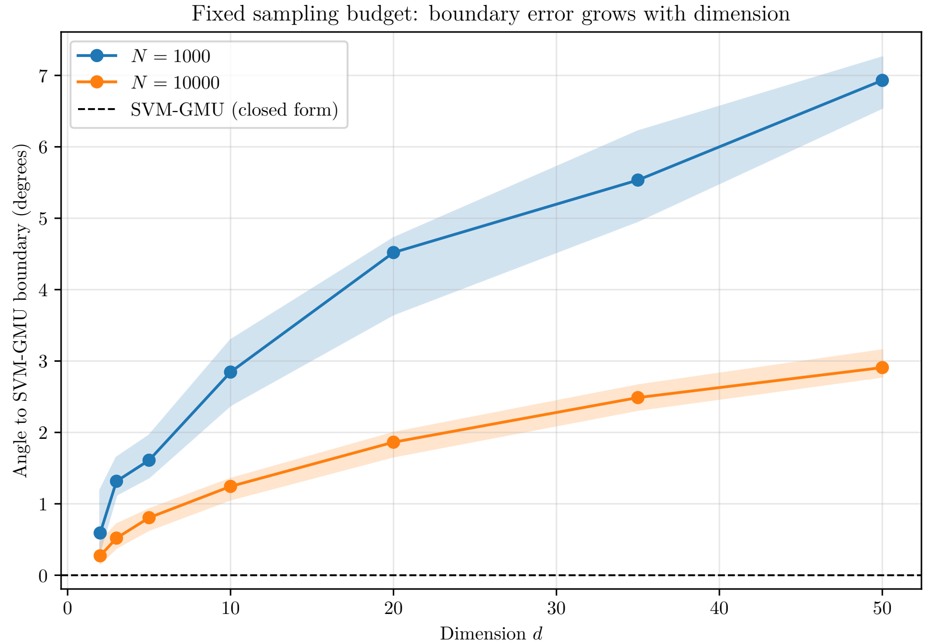

Figure 11: Fixed-budget view: median angle (over $R = 30$ seeds) between the standard-SVM-on-samples boundary and the closed-form SVM-GMU reference, as a function of dimension d, at two sample budgets N. The dashed line at 0° is the closed-form SVM-GMU, exact at every dimension. At a fixed budget the sampling error grows steadily with d.

Figure 12 reports the same fixed-budget reading through the dimension-agnostic decision-function RMS Δ^{mc} of eq. (106) rather than the angle, and it tells an identical story: at $N = 1000$ samples per example the median Δ^{mc} to the reference rises from 0.024 at $d = 2$ to 0.141 at $d = 50$, and at $N = 10^4$ from 0.012 to 0.061, whereas the closed-form SVM-GMU sits at exactly 0 for every d. That the orientation-only angle and the whole-space decision-function RMS grow in lockstep confirms that the disagreement is a genuine boundary effect and not an artifact of looking at a single facet of it.

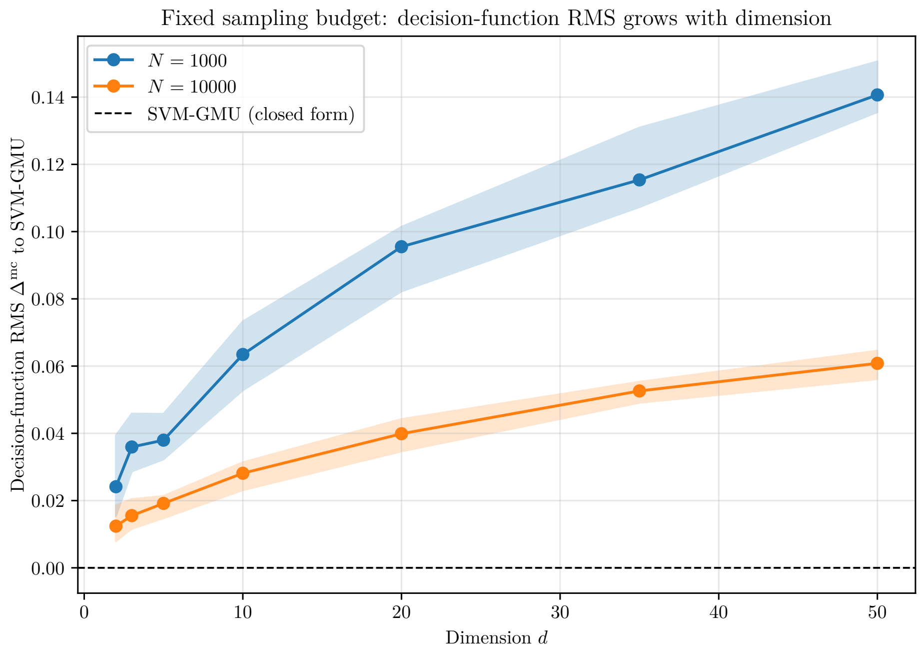

Figure 12: Fixed-budget view through the Monte Carlo decision-function RMS Δ^{mc} of eq. (106): median Δ^{mc} (over $R = 30$ seeds) between the standard-SVM-on-samples boundary and the closed-form SVM-GMU reference, as a function of dimension d, at two sample budgets N. The dashed line at 0 is the closed-form SVM-GMU, exact at every dimension. As with the angle in fig. 11, the sampling error grows steadily with d at a fixed budget.

The third boundary-disagreement metric, the offset difference Δ^{offset}, completes the picture and behaves qualitatively differently from the other two. Figure 13 plots it in the same fixed-budget style. It shrinks with the sample budget, from a median of roughly 0.01 to 0.02 at $N = 1000$ down to roughly 0.005 to 0.008 at $N = 10^4$, but, unlike the angle and the decision-function RMS, it shows no systematic growth with d, staying small and essentially flat across the whole range $d = 2$ to 50. This is exactly what one would expect: the offset is the single scalar $b/\|\mathbf{w}\|$, which finitely many samples estimate about equally well regardless of dimension, whereas the orientation $\mathbf{w}$ has d components that become progressively harder to pin down. The dimensional cost of sampling therefore concentrates in the boundary's *orientation* (the angle, and through it the decision-function RMS), not in its *position*.

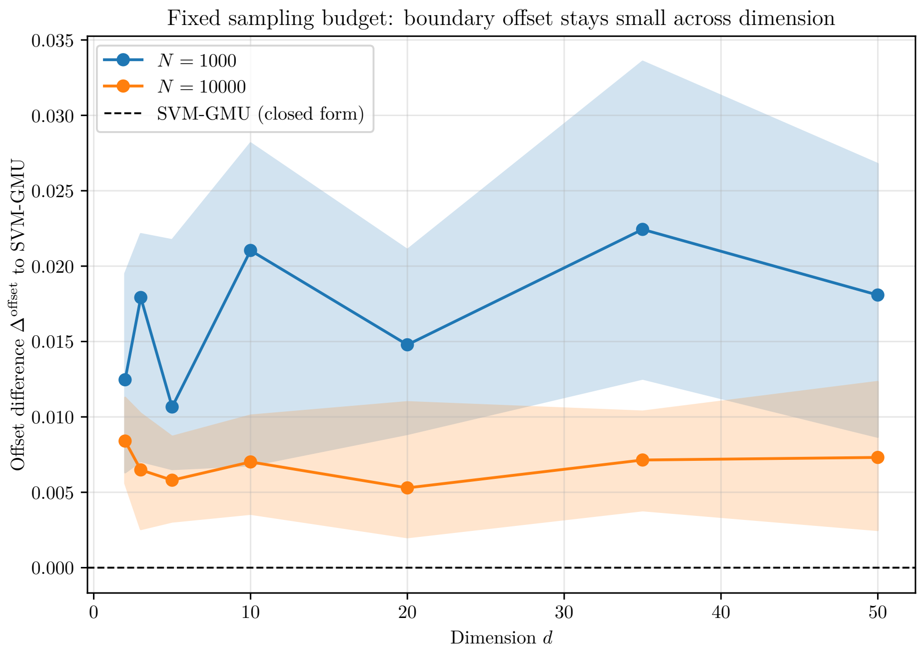

Figure 13: Fixed-budget view of the offset difference Δ^{offset}: median (over $R = 30$ seeds) of the difference in signed origin-to-boundary distance between the standard-SVM-on-samples boundary and the closed-form SVM-GMU reference, as a function of dimension d, at two sample budgets N. The dashed line at 0 is the closed-form SVM-GMU. In contrast to the angle (fig. 11) and the decision-function RMS (fig. 12), the offset stays small and roughly flat across d: the difficulty of high-dimensional sampling shows up in the boundary's orientation, not its position.

30.5 Interpretation

The experiment turns the qualitative warning of section 14 into a measured curve. Sampling from the per-sample mixtures and training a standard SVM is a viable substitute for the closed-form objective only in low dimensions; the sample budget needed to match SVM-GMU grows by about an order of magnitude per doubling of the dimensionality, so by a few dozen dimensions even 3×10^4 samples per example are not enough to reach a two-degree agreement. SVM-GMU sidesteps the issue entirely by evaluating the expected hinge loss in closed form eq. (67), at a cost independent of any sampling and, for diagonal (here isotropic) covariances, scaling only as $\mathcal{O}(M_i\, d)$ per example (section 21). In genuinely high-dimensional problems this is the difference between a tractable fit and an infeasible one, which is precisely the regime where uncertainty-aware classification is most often needed.

31 Experiment 6: SVM-GMU versus SVM-GSU on Real Images

31.1 What this experiment adds

The first five experiments used synthetic data, where we built each example's uncertainty by hand and could therefore check the method against a known ground truth. This final experiment moves to a real, widely used image dataset, the MNIST collection of handwritten digits, and asks the most practical question of all: *on real data, does modeling each example's uncertainty as a Gaussian* ***mixture*** *(SVM-GMU) actually classify better than modeling it as a single Gaussian (SVM-GSU),* *and better than ignoring the uncertainty altogether?* The answer turns out to be yes, but only when two conditions hold, and understanding those two conditions is as instructive as the result itself.

31.2 The two conditions for the mixture to matter

Before the setup, it is worth stating plainly what we are looking for and why, because a naive version of this experiment fails, and the reason it fails is exactly the reason the careful version succeeds.

Condition 1: the uncertainty must be genuinely non-Gaussian. Recall the central message of section 28: when an example's uncertainty cloud is a single blob, a single Gaussian already describes it well, and SVM-GMU and SVM-GSU agree. A mixture only earns its keep when the cloud has a shape a single Gaussian cannot mimic, for example two separate lobes with a gap between them. So if we want to see SVM-GMU beat SVM-GSU, we must construct per-image uncertainty that is genuinely multi-lobed, not just a fuzzy ball.

Condition 2: the test data must reflect the uncertainty. Modeling uncertainty during training buys one thing: *robustness*, a decision boundary that stays correct even when an example is perturbed within its uncertainty cloud. Robustness only pays off if the data we are eventually tested on is itself perturbed. Here is an analogy. Suppose you practice free throws while someone waves their arms to distract you. That practice makes you robust to distraction, but it slightly hurts your form on a perfectly calm court, where a player who only ever practiced in silence will edge you out. The extra preparation helps only when the real game is also noisy. The original SVM-GSU paper saw exactly this: its uncertainty-aware classifier helped most on the deliberately polluted test images (the rotated and translated MNIST variants D1 to D5), and barely at all on the pristine originals. We therefore evaluate on test images that carry the same uncertainty as the training images, not on clean ones.

31.3 Building the dataset: digits that wobble two ways

We take the two MNIST digit classes that are most easily confused once rotated, the digit `4` (which we label $+1$) and the digit `9` (labeled -1). For each class we draw a handful of clean training images.

The per-image uncertainty is created by *data augmentation*, the modeling route mentioned in section 32. But rather than the usual small symmetric wobble, we use a deliberately **bimodal** augmentation that satisfies Condition 1. Each time we generate a noisy copy of a training image, we flip a fair coin and rotate the image either by $+30°$ *or* by $-30°$, then add a small random jitter to that angle (a Gaussian of standard deviation $6°$) and a one-pixel random translation. Concretely, think of a photograph of a digit taken by someone who always tilts their head a firm $30°$ to the left or to the right, never holding it straight. The set of images this process produces has *two* clusters, a left-tilted lobe and a right-tilted lobe, with a hole in the middle where the upright digit would sit but almost never does.

This two-lobed cloud is the worst possible case for a single Gaussian, which is precisely why we chose it. A single Gaussian fit to "$+30°$ or $-30°$" is forced to center itself at $0°$ (the average of the two tilts), the one orientation the digit is almost never in, and to spread itself wide enough to cover both lobes. It therefore pours most of its modeled probability into the empty middle. A two-component mixture, by contrast, simply places one component on each lobe and describes the cloud faithfully. For each training image we generate 100 such augmented copies, giving a cloud of 100 points that stands in for "all the ways this digit might plausibly have looked."

31.4 From 784 pixels to 20 numbers: dimensionality reduction

Each MNIST image is $28 \times 28 = 784$ pixels, so each augmented copy is a point in $\mathbb{R}^{784}$. Fitting Gaussians and covariances directly in 784 dimensions is wasteful, because the augmentation only ever moves an image along a few meaningful directions (those corresponding to rotation and shift). We therefore compress the data with *Principal Component Analysis* (PCA), a standard technique that finds the handful of directions along which the data varies most and re-expresses every point using just its coordinates along those directions. We keep the top 20 such directions, fit on the pooled augmentation clouds of the training images (so the retained directions are exactly the ones the wobble lives in), and from here on every image, clean or augmented, is a point in $\mathbb{R}^{20}$. In the language of the rest of this report, the feature dimension is $d = 20$. The analogy is describing a person not by listing the color of all 784 pixels of their photograph but by 20 well-chosen summary numbers (roughly: how tilted, how shifted, how thick the strokes, and so on).

31.5 Five classifiers on a ladder of uncertainty modeling

The heart of the experiment is to feed the *same* classifier (the closed-form estimator of Part IV) five increasingly faithful descriptions of each image's uncertainty cloud, and to see how classification accuracy responds as the description improves. All five are trained by the same Pegasos-style solver (algorithm 2) with the same regularization; the only thing that changes from one to the next is how the cloud is summarized.

1. **Standard SVM (no uncertainty).** The cloud is ignored entirely; the classifier sees only the single clean image as a point. This is the ordinary linear SVM of Part I and our weakest baseline. In the notation of section 22 it is the case $\mathbf{\Sigma}_i \to \mathbf{0}$.

2. **Isotropic-uncertainty SVM (LSVM-iso).** The cloud is summarized by a single *round* Gaussian: one variance, the same in every direction, like a fuzzy ball centered on the cloud. This is the kind of uncertainty modeling used by earlier methods that handle noise isotropically, and the variant the SVM-GSU paper compares against.

3. **SVM-GSU (single full Gaussian).** The cloud is summarized by a single Gaussian whose mean and full covariance are *moment-matched* to the cloud, exactly the construction of section 28.2. This is the best a single ellipse can do, and it is the method we most want to beat.

4. **SVM-GMU, structural mixture (no fitting).** A two-component mixture placed *by hand* at the two known rotation modes: one Gaussian on the $+30°$ lobe and one on the $-30°$ lobe, each obtained by moment-matching a small batch of augmentations near that mode, with equal weights. No iterative fitting is involved; we exploit the fact that we *know* how the cloud was generated, in the same spirit as the analytic banana mixtures of section 25.

5. **SVM-GMU, EM mixture (learned).** A Gaussian mixture fit to the 100-point cloud by Expectation Maximization, with the number of components chosen by the Bayesian Information Criterion (BIC) over $M \in \{1, \ldots, 8\}$, exactly the estimation pipeline of sections 29 and 29.3. This is the realistic case in which we do *not* assume we know the cloud's structure but learn it from the points. On this data BIC typically selects around seven or eight components, because

each lobe has internal spread from the jitter and translation; the cloud is genuinely rich, not merely two dots.

The five form a *ladder*: each rung describes the same uncertainty a little more faithfully than the one below, from ignoring it (rung 1), to a round blob (rung 2), to a single oriented ellipse (rung 3), to a hand-built two-lobe mixture (rung 4), to a learned mixture (rung 5). If richer modeling of the uncertainty genuinely helps, accuracy should climb monotonically up the ladder.

31.6 Testing on wobbly digits

To satisfy Condition 2 of section 31.2, we do not test on clean digits. Each test image is passed through the *same* bimodal augmentation, producing ten noisy copies, and the classifier must label each copy. Accuracy is then measured over all of these augmented test points. This is the honest test of robustness: the question is not "can you recognize a perfectly upright 4" but "can you recognize a 4 that has been tilted $\pm 30°$, which is the kind of 4 the world actually hands you here." As a control we also measured accuracy on clean test images, and there, exactly as the free-throw analogy predicts, the plain SVM wins and the uncertainty-aware models gain nothing, because the robustness they paid for goes unused. All results below are on the augmented (realistic) test set.

31.7 Experimental protocol

The configuration is as follows. We use $d = 20$ PCA features; 80 training images per class at the main operating point; 100 augmentations per training image; 200 test images per class, each augmented 10 times; and the bimodal rotation magnitude set to $30°$ at the main operating point. All five classifiers share a single fixed regularization parameter $\lambda = 10^{-2}$. Fixing λ rather than tuning it separately for each method is deliberate: it guarantees that any difference in accuracy is due to the *uncertainty model*, not to one method happening to receive a luckier regularization strength. Covariances are kept diagonal in the 20-dimensional space, which (as section 10 notes) is both cheap and, with 100 samples per cloud, reliably estimable.

Following the reporting protocol of Part V, the whole experiment is repeated over $R = 30$ independent random seeds spawned from one master seed, each seed redrawing the images, the augmentations, and the train/test split; we report every metric as its median over the 30 seeds with a 25th to 75th percentile inter-quartile band. We record four standard classification metrics, each capturing a different facet of quality:

- **Accuracy**: the fraction of test points labeled correctly.
- **Macro-F1**: the average over the two classes of the F1 score (the harmonic mean of precision and recall); unlike raw accuracy it refuses to be fooled by a classifier that is good on one class and poor on the other.
- **ROC-AUC**: the probability that a randomly chosen positive example receives a higher decision score $\mathbf{w}^\intercal \mathbf{x} + b$ than a randomly chosen negative one; a measure of how well the classifier *ranks*, where 1.0 is perfect and 0.5 is a coin flip.
- **Average precision**: the area under the precision-recall curve, summarizing how cleanly the positives can be retrieved; it plays the role that the AP and MAP scores played in the video experiments of the original SVM-GSU paper.

Finally, to confirm that the gap between SVM-GMU and SVM-GSU is real and not a fluke of the random draws, we run two *statistical significance tests*, both used by the SVM-GSU paper. McNemar's test looks only at the test points where the two classifiers *disagree* and asks whether the disagreements favor one classifier more lopsidedly than a coin flip would explain. The paired tests (Wilcoxon signed-rank and a paired t-test) look across the 30 seeds and ask whether SVM-GMU's per-seed accuracy is *consistently* above SVM-GSU's. Each test returns a p-value, the probability of seeing a gap this large if the two classifiers were truly equally good; a tiny p-value (well below the customary 0.05 threshold) means the gap is almost certainly genuine.

31.8 Results

Table 7 reports the four metrics for all five classifiers at the main operating point (bimodal magnitude 30°, 80 training images per class), as medians over the 30 seeds. The ladder behaves exactly as hoped: every rung improves on the one below it, on *every one* of the four metrics. Ignoring uncertainty (the standard SVM) is clearly worst; a round isotropic blob helps; a single oriented Gaussian (SVM-GSU) helps more; and the two SVM-GMU mixtures, structural and EM-learned, are best of all, with the learned EM mixture on top.

Table 7: MNIST `4`-versus-`9` results at the main operating point (bimodal rotation 30°, 80 training images per class), on the augmented test set. Each entry is the median over $R = 30$ seeds; bold marks the best in each column. Accuracy climbs monotonically up the modeling ladder, and SVM-GMU (EM) is best on all four metrics.

Classifier	**Accuracy**	**Macro-F1**	**ROC-AUC**	**Avg. precision**
Standard SVM (no uncertainty)	0.767	0.765	0.840	0.802
LSVM-iso (isotropic Gaussian)	0.818	0.818	0.893	0.875
SVM-GSU (single full Gaussian)	0.836	0.836	0.907	0.898
SVM-GMU (structural mixture)	0.842	0.842	0.916	0.902
SVM-GMU (EM mixture)	**0.849**	**0.849**	**0.920**	**0.909**

The headline comparison is SVM-GMU (EM) against SVM-GSU. The accuracy gap is modest in size, about 1.3 percentage points (0.849 versus 0.836), which is exactly what the theory leads us to expect: section 28 already found that for a *linear* classifier the boundary moves only slightly when a mixture replaces a moment-matched Gaussian (there, an orientation change of about 1.7°). What makes the result convincing is not the size of the gap but its *consistency*. SVM-GMU (EM) beat SVM-GSU on all 30 of the 30 seeds, and the significance tests are emphatic: the paired Wilcoxon test gives $p \approx 2 \times 10^{-6}$, the paired t-test $p < 10^{-6}$, and McNemar's test a median $p \approx 2 \times 10^{-4}$. By every standard the difference is statistically significant, not an artifact of lucky sampling. (As a sanity check on Condition 1, BIC selected a median of about 7.6 components per cloud, confirming that the uncertainty really is richly multi-lobed and not something a single Gaussian could have captured.)

Figure 14 shows what happens as we dial the bimodal rotation magnitude from 15° up to 45°, that is, as the per-image uncertainty becomes more pronounced and the two lobes pull farther apart. Two trends appear together. First, the standard SVM, which ignores the wobble, collapses, its median accuracy falling from about 0.84 at 15° to about 0.65 at 45°, because a boundary tuned to upright digits is increasingly wrong for sharply tilted ones. The uncertainty-aware models degrade far more gracefully. Second, SVM-GMU (EM) stays on top at every magnitude. This is the quantitative form of Condition 2: the more the test data is perturbed, the more the robustness bought by uncertainty modeling is worth, and the further the uncertainty-aware ladder pulls ahead of the plain SVM.

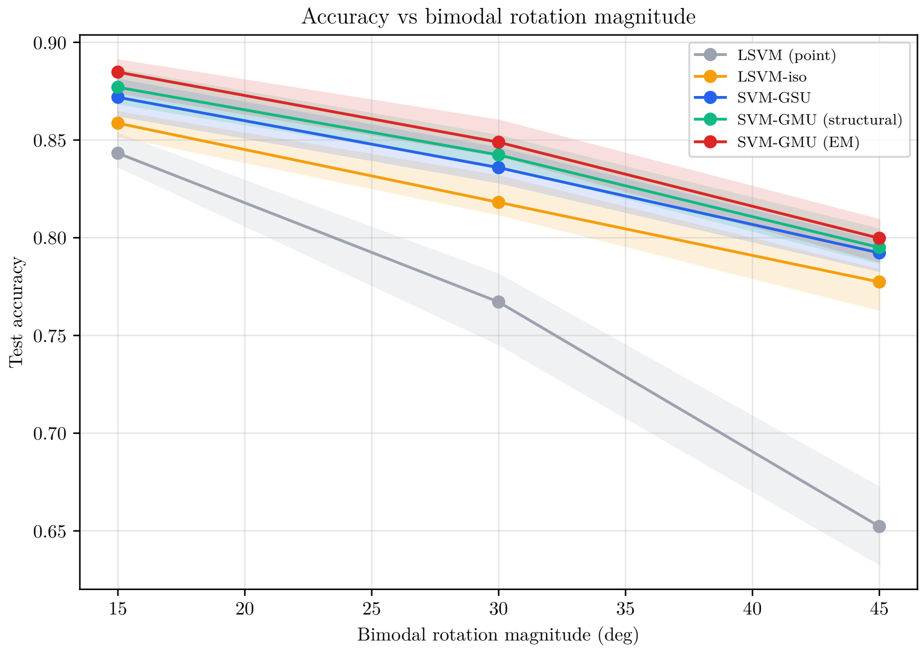

Figure 14: MNIST `4`-versus-`9` accuracy on the augmented test set as a function of the bimodal rotation magnitude, for all five classifiers. Solid lines are medians over $R = 30$ seeds and the shaded bands span the 25th to 75th percentile. As the wobble grows, the standard SVM (which ignores it) collapses while the uncertainty-aware models stay robust; SVM-GMU (EM) is best at every magnitude.

Figure 15 repeats the comparison as a function of the number of training images per class (25, 80, 200), at a fixed rotation magnitude. The ordering of the ladder is preserved at every training-set size, and SVM-GMU (EM) remains best throughout; more training data lifts all five classifiers without changing their ranking. This rules out the worry that the advantage is an artifact of working in the small-sample regime.

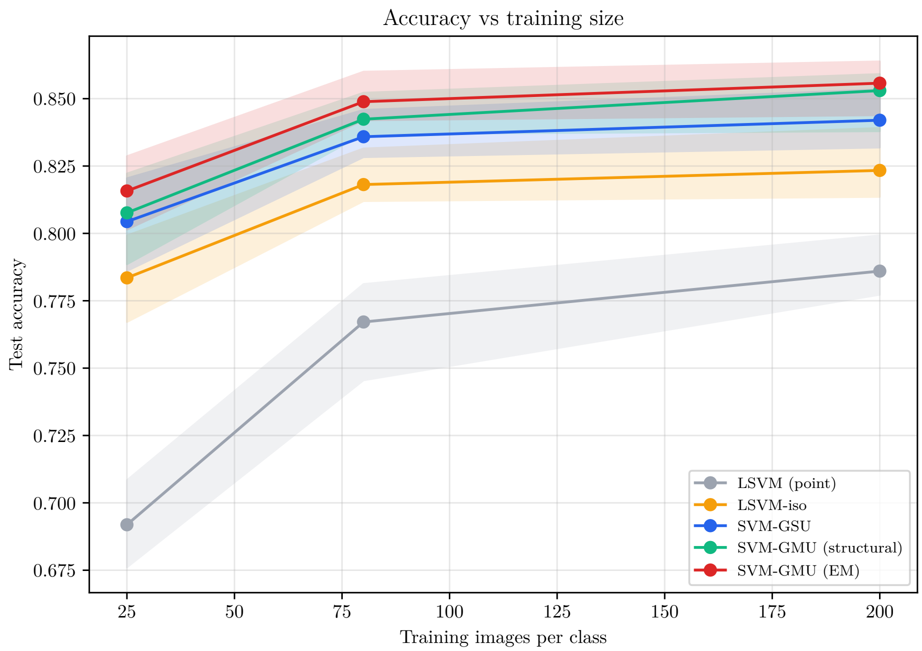

Figure 15: MNIST 4-versus-9 accuracy on the augmented test set as a function of the number of training images per class, at bimodal rotation magnitude 30°. Medians over $R = 30$ seeds with 25th to 75th percentile bands. SVM-GMU (EM) is best at every training-set size, and the whole ladder ordering is stable.

Finally, fig. 16 makes the mechanism visible. It projects the augmentation clouds of one 4 and one 9 into a two-dimensional PCA space and overlays the decision boundaries learned by SVM-GSU and the two SVM-GMU variants. The clouds are visibly two-lobed, and one can see the SVM-GMU boundary sitting slightly differently from the SVM-GSU boundary, responding to where the lobes actually are rather than to the fat single ellipse that a moment-matched Gaussian would draw around them.

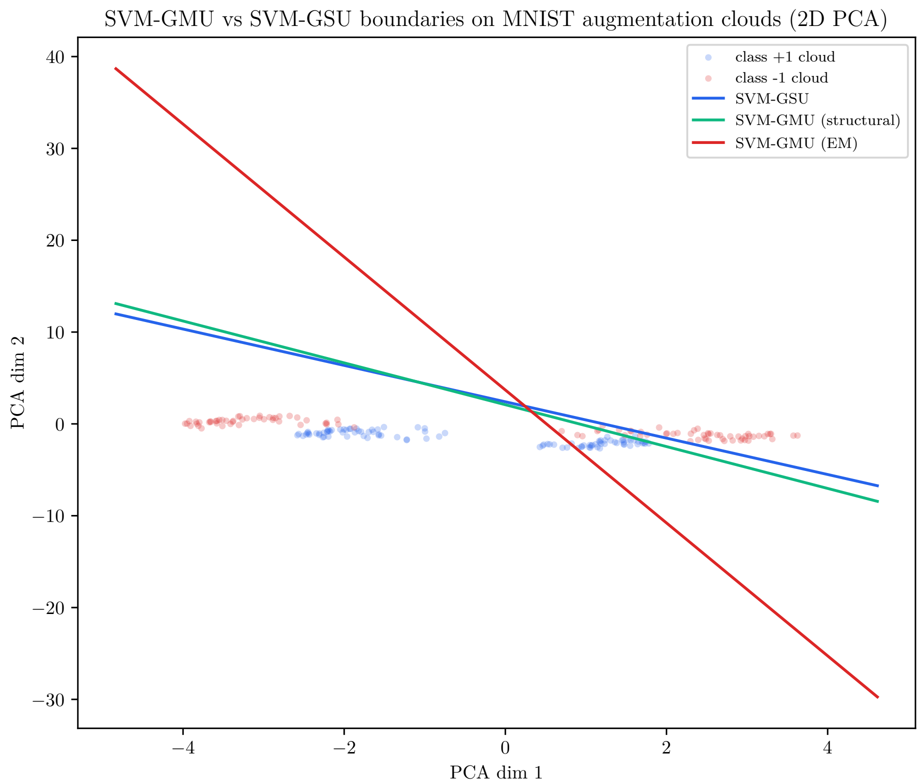

Figure 16: Two-dimensional PCA view of the bimodal augmentation clouds of one `4` (class +1) and one `9` (class −1), with the SVM-GSU and SVM-GMU decision boundaries overlaid. The clouds are clearly two-lobed, the structure that a single Gaussian cannot represent but a mixture can.

31.9 What the numbers mean

The experiment delivers the real-data confirmation the report has been building toward, and it does so with the honesty that the two conditions of section 31.2 demand.

Why the mixture wins. SVM-GMU beats SVM-GSU because the bimodal cloud is genuinely non-Gaussian. The single moment-matched Gaussian of SVM-GSU is forced to spend probability on the upright orientation the digit is almost never in, which nudges its decision boundary slightly away from the optimum; the mixture spends its probability where the digit actually is, on the two lobes, and places its boundary slightly better. Repeated over 30 seeds the slightly-better boundary wins every time.

Why the gap is small but solid. As section 28 explained, a linear classifier only ever sees the projection of the uncertainty onto the single direction $\mathbf{w}$, and along one direction a mixture and its moment-matched Gaussian cannot differ by very much. So the accuracy gap between SVM-GMU and SVM-GSU is inherently modest. But because the experiment controls for everything else (same solver, same fixed λ, same features, same seeds), the modest gap is also extremely stable, significant at $p \approx 2 \times 10^{-6}$ and present on all four metrics, at every rotation magnitude, and at every training-set size.

The honest caveat. This advantage is not free or universal. It appears only when both conditions hold: the per-example uncertainty must be genuinely multimodal (Condition 1), and the data the classifier is judged on must itself carry that uncertainty (Condition 2). When the uncertainty is unimodal, SVM-GMU and SVM-GSU coincide, as section 28 showed; and when the test data is clean, even rich uncertainty modeling helps nothing, and the plain SVM wins, exactly as the SVM-GSU paper found on its pristine MNIST originals. The practical lesson is the symmetric one we would want: model the uncertainty as richly as it truly is, no richer and no poorer, and reap the benefit precisely in the regime, noisy multimodal data, where uncertainty-aware classification is needed in the first place.

Part VI
Discussion and Open Questions

32 How to Obtain the GMM Parameters in Practice?

The GMM parameters $\left\{\pi_i^{(m)}, \boldsymbol{\mu}_i^{(m)}, \boldsymbol{\Sigma}_i^{(m)}\right\}$ can be obtained from domain knowledge, by fitting a GMM to multiple measurements via the EM algorithm, from data augmentation, or from generative models with mixture posteriors.

33 Choosing the Number of Components M_i

Standard model selection criteria (BIC, AIC, cross-validation) apply. Even $M_i = 2$ or 3 dramatically increases expressiveness, as shown by the banana/crescent shapes in the `close_separable` experiments (Part V), which use $M_i \in \{5, 6\}$.

34 Potential for Kernelization

Kernelization remains open: the closed-form loss depends on $\mathbf{w}^\intercal \boldsymbol{\Sigma}_i^{(m)} \mathbf{w}$, which does not directly map to a kernel formulation.

35 Relationship to Other Non-Gaussian Uncertainty Models

Alternatives include sampling-based methods (which scale poorly, as the concentration bound eq. (57) and the convergence experiment of section 27 both show), robust optimization (more conservative), and other distributional families. GMMs offer the best flexibility due to their universal approximation property while preserving all computational advantages of the SVM-GSU framework.

Appendices

A Full Proof of theorem 9.1

We give a complete proof. The goal is to evaluate

$$I^+(\mathbf{a}, b') = \int_{\Omega^+} (\mathbf{a}^\intercal \mathbf{x} + b')\, f_{\mathbf{x}}(\mathbf{x})\, \mathrm{d}\mathbf{x}\,, \tag{108}$$

where $\Omega^+ = \{\mathbf{x} \in \mathbb{R}^d : \mathbf{a}^\intercal \mathbf{x} + b' \geq 0\}$ and $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$. The case I^- follows by a symmetric argument.

We proceed by a sequence of four changes of variables, each bringing the integral closer to a standard form.

A.1 Step 1: Translation to Center the Gaussian

Let $\mathbf{y} = \mathbf{x} - \boldsymbol{\mu}$, so $\mathbf{x} = \mathbf{y} + \boldsymbol{\mu}$ and $\mathrm{d}\mathbf{x} = \mathrm{d}\mathbf{y}$. The Gaussian becomes

$$f_{\mathbf{x}}(\mathbf{y} + \boldsymbol{\mu}) = \frac{1}{(2\pi)^{d/2}\, |\boldsymbol{\Sigma}|^{1/2}} \exp\!\left(-\frac{1}{2}\mathbf{y}^\intercal \boldsymbol{\Sigma}^{-1} \mathbf{y}\right), \tag{109}$$

which is the density of $\mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma})$. The integrand becomes

$$\mathbf{a}^\intercal(\mathbf{y} + \boldsymbol{\mu}) + b' = \mathbf{a}^\intercal \mathbf{y} + \underbrace{(\mathbf{a}^\intercal \boldsymbol{\mu} + b')}_{d_{\boldsymbol{\mu}}} = \mathbf{a}^\intercal \mathbf{y} + d_{\boldsymbol{\mu}}. \tag{110}$$

The halfspace transforms to

$$\Omega_1^+ = \{\mathbf{y} \in \mathbb{R}^d : \mathbf{a}^\intercal \mathbf{y} + d_{\boldsymbol{\mu}} \geq 0\}. \tag{111}$$

So

$$I^+ = \frac{1}{(2\pi)^{d/2}\, |\boldsymbol{\Sigma}|^{1/2}} \int_{\Omega_1^+} (\mathbf{a}^\intercal \mathbf{y} + d_{\boldsymbol{\mu}}) \exp\!\left(-\frac{1}{2}\mathbf{y}^\intercal \boldsymbol{\Sigma}^{-1} \mathbf{y}\right) \mathrm{d}\mathbf{y}\,. \tag{112}$$

A.2 Step 2: Diagonalize the Covariance

Since $\boldsymbol{\Sigma} \in \mathbb{S}^d_{++}$, it has an orthonormal eigendecomposition:

$$\boldsymbol{\Sigma} = \mathbf{U}\mathbf{D}\mathbf{U}^\intercal, \tag{113}$$

where $\mathbf{U}$ is orthogonal ($\mathbf{U}^\intercal \mathbf{U} = \mathbf{I}$, so $|\det(\mathbf{U})| = 1$) and $\mathbf{D}$ is diagonal with positive entries (the eigenvalues of $\boldsymbol{\Sigma}$). Let $\mathbf{z} = \mathbf{U}^\intercal \mathbf{y}$, so $\mathbf{y} = \mathbf{U}\mathbf{z}$ and $\mathrm{d}\mathbf{y} = |\det(\mathbf{U})|\, \mathrm{d}\mathbf{z} = \mathrm{d}\mathbf{z}$.

Define $\mathbf{a}_1 = \mathbf{U}^\intercal \mathbf{a}$. Then

$$\mathbf{a}^\intercal \mathbf{y} = \mathbf{a}^\intercal \mathbf{U}\mathbf{z} = (\mathbf{U}^\intercal \mathbf{a})^\intercal \mathbf{z} = \mathbf{a}_1^\intercal \mathbf{z}, \tag{114}$$

and

$$\mathbf{y}^\intercal \boldsymbol{\Sigma}^{-1} \mathbf{y} = \mathbf{z}^\intercal \mathbf{U}^\intercal (\mathbf{U}\mathbf{D}\mathbf{U}^\intercal)^{-1} \mathbf{U}\mathbf{z} = \mathbf{z}^\intercal \mathbf{U}^\intercal \mathbf{U}\mathbf{D}^{-1}\mathbf{U}^\intercal \mathbf{U}\mathbf{z} = \mathbf{z}^\intercal \mathbf{D}^{-1} \mathbf{z}. \tag{115}$$

Also $|\boldsymbol{\Sigma}|^{1/2} = |\mathbf{D}|^{1/2}$. The integral becomes

$$I^+ = \frac{1}{(2\pi)^{d/2}\, |\mathbf{D}|^{1/2}} \int_{\Omega_2^+} (\mathbf{a}_1^\intercal \mathbf{z} + d_{\boldsymbol{\mu}}) \exp\!\left(-\frac{1}{2}\mathbf{z}^\intercal \mathbf{D}^{-1} \mathbf{z}\right) \mathrm{d}\mathbf{z}\,, \tag{116}$$

where $\Omega_2^+ = \{\mathbf{z} : \mathbf{a}_1^\intercal \mathbf{z} + d_{\boldsymbol{\mu}} \geq 0\}$.

A.3 Step 3: Rescale to Standardize

Let $\mathbf{v} = \mathbf{D}^{-1/2}\mathbf{z}$, so $\mathbf{z} = \mathbf{D}^{1/2}\mathbf{v}$ and $\mathrm{d}\mathbf{z} = |\mathbf{D}|^{1/2}\,\mathrm{d}\mathbf{v}$. Define $\mathbf{a}_2 = \mathbf{D}^{1/2}\mathbf{a}_1$. Then

$$\mathbf{a}_1^\intercal \mathbf{z} = \mathbf{a}_1^\intercal \mathbf{D}^{1/2}\mathbf{v} = (\mathbf{D}^{1/2}\mathbf{a}_1)^\intercal \mathbf{v} = \mathbf{a}_2^\intercal \mathbf{v}, \tag{117}$$

and

$$\mathbf{z}^\intercal \mathbf{D}^{-1}\mathbf{z} = \mathbf{v}^\intercal \mathbf{D}^{1/2}\mathbf{D}^{-1}\mathbf{D}^{1/2}\mathbf{v} = \mathbf{v}^\intercal \mathbf{v}. \tag{118}$$

The factors $|\mathbf{D}|^{1/2}$ cancel, yielding

$$I^+ = \frac{1}{(2\pi)^{d/2}} \int_{\Omega_3^+} (\mathbf{a}_2^\intercal \mathbf{v} + d_{\boldsymbol{\mu}}) \exp\left(-\frac{1}{2}\|\mathbf{v}\|^2\right) \mathrm{d}\mathbf{v}\,, \tag{119}$$

where $\Omega_3^+ = \{\mathbf{v} : \mathbf{a}_2^\intercal \mathbf{v} + d_{\boldsymbol{\mu}} \geq 0\}$.

Note that

$$\|\mathbf{a}_2\|^2 = \mathbf{a}_2^\intercal \mathbf{a}_2 = \mathbf{a}_1^\intercal \mathbf{D}\mathbf{a}_1 = \mathbf{a}^\intercal \mathbf{U}\mathbf{D}\mathbf{U}^\intercal \mathbf{a} = \mathbf{a}^\intercal \boldsymbol{\Sigma}\mathbf{a}, \tag{120}$$

so $\|\mathbf{a}_2\| = \sqrt{\mathbf{a}^\intercal \boldsymbol{\Sigma}\mathbf{a}}$. Recall from eq. (23) that $d_{\boldsymbol{\Sigma}} = \sqrt{2\mathbf{a}^\intercal \boldsymbol{\Sigma}\mathbf{a}} = \sqrt{2}\|\mathbf{a}_2\|$.

A.4 Step 4: Rotation to Isolate One Coordinate

Now we have a standard Gaussian (with identity covariance) and a linear function times it. Pick an orthogonal matrix $\mathbf{B}$ such that

$$\mathbf{B}\mathbf{a}_2 = \|\mathbf{a}_2\|\mathbf{e}_d, \tag{121}$$

where $\mathbf{e}_d = (0, 0, \ldots, 0, 1)^\intercal$ is the d-th standard basis vector. Such a $\mathbf{B}$ exists (for example, a Householder reflection).

Let $\mathbf{m} = \mathbf{B}\mathbf{v}$, so $\mathbf{v} = \mathbf{B}^\intercal \mathbf{m}$ and $\mathrm{d}\mathbf{v} = \mathrm{d}\mathbf{m}$. Then

$$\mathbf{a}_2^\intercal \mathbf{v} = \mathbf{a}_2^\intercal \mathbf{B}^\intercal \mathbf{m} = (\mathbf{B}\mathbf{a}_2)^\intercal \mathbf{m} = \|\mathbf{a}_2\|\mathbf{e}_d^\intercal \mathbf{m} = \|\mathbf{a}_2\|\, m_d, \tag{122}$$

and

$$\|\mathbf{v}\|^2 = \mathbf{v}^\intercal \mathbf{v} = \mathbf{m}^\intercal \mathbf{B}\mathbf{B}^\intercal \mathbf{m} = \mathbf{m}^\intercal \mathbf{m} = \|\mathbf{m}\|^2. \tag{123}$$

The halfspace becomes $\Omega_4^+ = \{\mathbf{m} : \|\mathbf{a}_2\| m_d + d_{\boldsymbol{\mu}} \geq 0\}$, equivalently $m_d \geq -d_{\boldsymbol{\mu}}/\|\mathbf{a}_2\|$.

So eq. (119) becomes

$$I^+ = \frac{1}{(2\pi)^{d/2}} \int_{\mathbb{R}^{d-1}} \int_{-d_{\boldsymbol{\mu}}/\|\mathbf{a}_2\|}^{\infty} (\|\mathbf{a}_2\| m_d + d_{\boldsymbol{\mu}}) \exp\left(-\frac{1}{2}\|\mathbf{m}\|^2\right) \mathrm{d}m_d\; \mathrm{d}m_1 \cdots \mathrm{d}m_{d-1}\,. \tag{124}$$

A.5 Step 5: Separate and Evaluate the 1D Integral

Write $\|\mathbf{m}\|^2 = m_d^2 + \sum_{j=1}^{d-1} m_j^2$. The integrand factors and the $m_1, \ldots, m_{d-1}$ integrals each give $\sqrt{2\pi}$ (they are integrals of a standard Gaussian over all of $\mathbb{R}$):

$$\int_{\mathbb{R}^{d-1}} \exp\left(-\frac{1}{2}\sum_{j=1}^{d-1} m_j^2\right) \prod_{j=1}^{d-1} \mathrm{d}m_j = (2\pi)^{(d-1)/2}. \tag{125}$$

So

$$I^+ = \frac{(2\pi)^{(d-1)/2}}{(2\pi)^{d/2}} \int_c^\infty (\|\mathbf{a}_2\| t + d_{\boldsymbol{\mu}}) \exp\left(-\frac{t^2}{2}\right) \mathrm{d}t = \frac{1}{\sqrt{2\pi}} \int_c^\infty (\|\mathbf{a}_2\| t + d_{\boldsymbol{\mu}}) \exp\left(-\frac{t^2}{2}\right) \mathrm{d}t\,, \tag{126}$$

where $c = -d_{\boldsymbol{\mu}}/\|\mathbf{a}_2\|$. We split this into two 1D integrals:

$$I^+ = \frac{\|\mathbf{a}_2\|}{\sqrt{2\pi}} \underbrace{\int_c^\infty t\, e^{-t^2/2}\, \mathrm{d}t}_{T_1} + \frac{d_{\boldsymbol{\mu}}}{\sqrt{2\pi}} \underbrace{\int_c^\infty e^{-t^2/2}\, \mathrm{d}t}_{T_2}. \tag{127}$$

Evaluating T_1. Since $\frac{\mathrm{d}}{\mathrm{d}t}\left(-e^{-t^2/2}\right) = t\,e^{-t^2/2}$:

$$T_1 = \left[-e^{-t^2/2}\right]_c^{\infty} = 0 - \left(-e^{-c^2/2}\right) = e^{-c^2/2}. \tag{128}$$

Evaluating T_2. Using the substitution $u = t/\sqrt{2}$, $\mathrm{d}t = \sqrt{2}\,\mathrm{d}u$:

$$T_2 = \int_c^{\infty} e^{-t^2/2}\ \mathrm{d}t = \sqrt{2}\int_{c/\sqrt{2}}^{\infty} e^{-u^2}\ \mathrm{d}u\,. \tag{129}$$

Now we express this in terms of the error function. From eq. (24), $\int_0^x e^{-u^2}\ \mathrm{d}u = \frac{\sqrt{\pi}}{2}\mathrm{erf}(x)$, and $\int_0^{\infty} e^{-u^2}\ \mathrm{d}u = \frac{\sqrt{\pi}}{2}$. Therefore

$$\int_{c/\sqrt{2}}^{\infty} e^{-u^2}\ \mathrm{d}u = \int_0^{\infty} e^{-u^2}\ \mathrm{d}u - \int_0^{c/\sqrt{2}} e^{-u^2}\ \mathrm{d}u = \frac{\sqrt{\pi}}{2} - \frac{\sqrt{\pi}}{2}\mathrm{erf}\left(\frac{c}{\sqrt{2}}\right) = \frac{\sqrt{\pi}}{2}\left(1 - \mathrm{erf}\left(\frac{c}{\sqrt{2}}\right)\right). \tag{130}$$

So

$$T_2 = \sqrt{2}\cdot\frac{\sqrt{\pi}}{2}\left(1 - \mathrm{erf}\left(\frac{c}{\sqrt{2}}\right)\right) = \sqrt{\frac{\pi}{2}}\left(1 - \mathrm{erf}\left(\frac{c}{\sqrt{2}}\right)\right). \tag{131}$$

A.6 Step 6: Combine and Simplify

Putting it together:

$$I^+ = \frac{\|\mathbf{a}_2\|}{\sqrt{2\pi}}e^{-c^2/2} + \frac{d_{\boldsymbol{\mu}}}{\sqrt{2\pi}}\cdot\sqrt{\frac{\pi}{2}}\left(1 - \mathrm{erf}\left(\frac{c}{\sqrt{2}}\right)\right). \tag{132}$$

Simplify the second term: $\frac{1}{\sqrt{2\pi}}\cdot\sqrt{\frac{\pi}{2}} = \frac{1}{2}$. So

$$I^+ = \frac{\|\mathbf{a}_2\|}{\sqrt{2\pi}}e^{-c^2/2} + \frac{d_{\boldsymbol{\mu}}}{2}\left(1 - \mathrm{erf}\left(\frac{c}{\sqrt{2}}\right)\right). \tag{133}$$

Now substitute back. Recall $c = -d_{\boldsymbol{\mu}}/\|\mathbf{a}_2\|$, and $\|\mathbf{a}_2\| = d_{\boldsymbol{\Sigma}}/\sqrt{2}$, so $c = -\sqrt{2}\,d_{\boldsymbol{\mu}}/d_{\boldsymbol{\Sigma}}$, and

$$\frac{c}{\sqrt{2}} = -\frac{d_{\boldsymbol{\mu}}}{d_{\boldsymbol{\Sigma}}}, \qquad \frac{c^2}{2} = \frac{d_{\boldsymbol{\mu}}^2}{d_{\boldsymbol{\Sigma}}^2}. \tag{134}$$

Using the oddness of erf, $\mathrm{erf}(-x) = -\mathrm{erf}(x)$:

$$1 - \mathrm{erf}\left(-\frac{d_{\boldsymbol{\mu}}}{d_{\boldsymbol{\Sigma}}}\right) = 1 + \mathrm{erf}\left(\frac{d_{\boldsymbol{\mu}}}{d_{\boldsymbol{\Sigma}}}\right). \tag{135}$$

Also

$$\frac{\|\mathbf{a}_2\|}{\sqrt{2\pi}} = \frac{d_{\boldsymbol{\Sigma}}/\sqrt{2}}{\sqrt{2\pi}} = \frac{d_{\boldsymbol{\Sigma}}}{2\sqrt{\pi}}. \tag{136}$$

Therefore

$$I^+ = \frac{d_{\boldsymbol{\mu}}}{2}\left(1 + \mathrm{erf}\left(\frac{d_{\boldsymbol{\mu}}}{d_{\boldsymbol{\Sigma}}}\right)\right) + \frac{d_{\boldsymbol{\Sigma}}}{2\sqrt{\pi}}\exp\left(-\frac{d_{\boldsymbol{\mu}}^2}{d_{\boldsymbol{\Sigma}}^2}\right). \tag{137}$$

This is exactly eq. (21) with the + sign. The I^- case follows by repeating the argument with the halfspace $\mathbf{a}^\intercal\mathbf{x} + b' \le 0$; equivalently, one can use the identity $I^+ + I^- = \mathbb{E}[\mathbf{a}^\intercal\mathbf{x} + b'] = d_{\boldsymbol{\mu}}$ to recover I^-. $\square$

References

Cirel'son, Boris S, Ildar A Ibragimov, and Vladimir N Sudakov (2006). "Norms of Gaussian sample functions". In: *Proceedings of the Third Japan—USSR Symposium on Probability Theory.* Springer, pp. 20–41.

Tzelepis, Christos, Vasileios Mezaris, and Ioannis Patras (2017). "Linear Maximum Margin Classifier for Learning from Uncertain Data". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 40.12, pp. 2948–2962.